TECHPIOASSET — ENTERPRISE ITAM REDESIGN

Developer Engineering Pack

Data Model, APIs, RBAC, Module Logic, Architecture & Roadmap

Discipline extract from the 86-page master blueprint

Pack	Developer Engineering Pack
Audience	Backend & full-stack engineers, tech leads, QA engineers
Product	TechpioAsset — multi-tenant SaaS (Web + Mobile + REST API)
Version	2.0 — target-state redesign
Master document	TechpioAsset — Enterprise ITAM Redesign Blueprint (86 pp, BRD + FRD + Technical SRS)
Baseline	Monorepo at `E:\TechpioAsset` · live at https://piotask.com

SCOPE OF THIS PACK Everything an engineer needs to build the redesign: business rules, the full 13-role RBAC + permission taxonomy, the normalized data model / ERD / status split, License Management enforcement logic & APIs plus the hardware data model, Procurement & Inventory entities/validations/APIs, the Maintenance model with notification & audit matrices, the technical SRS (architecture, API design, security, auth, offline sync, queue/outbox, caching) with web module/screen trees, and the phased delivery roadmap. Visual dashboards & wireframes live in the Design Pack.

Contents

1 Requirements & Business Rules (BRD)

2 Roles, Permission Model & Taxonomy

3 Data Model, ERD & Status Redesign

4 License Management — Logic, Seat-Limit Enforcement, APIs + Hardware Data Model

5 Procurement & Inventory — Entities, Validations & APIs

6 Maintenance, Notification Matrix & Audit Matrix

7 Technical SRS — Architecture, API, Security, Data + Module/Screen Trees

8 Delivery Roadmap & Phase Plan

PART 1

Requirements & Business Rules (BRD)

Part A — Business Requirement Document (BRD)

A.1 Business Objectives (measurable)

#	Objective	Baseline (typical pre-redesign)	Target (12 months post-GA)	Measurement / KPI
O1	Establish a single source of truth for all IT & office assets across every tenant	~70% assets tracked, spreadsheets in parallel	≥ 99% of in-scope assets in CMDB with owner, location, state	Reconciliation coverage % (discovered vs. recorded)
O2	Reduce asset request → fulfilment cycle time	9–14 business days	≤ 3 business days (standard kit), ≤ 5 (non-standard)	Median time from request submit to "Active/In-Use"
O3	Eliminate invoice/PO/GRN mismatches reaching Finance	~12% exceptions manually resolved	≤ 2% exceptions; 0 silent overpayments	3-way match pass rate; overpayment $ prevented
O4	Guarantee license compliance (no over-deployment)	Unknown seat position	100% of paid SKUs with real-time seat ledger; 0 breaches	Seats consumed ≤ seats owned, per SKU, per tenant
O5	Cut hardware TCO through lifecycle visibility	No warranty/AMC alerting	≥ 20% reduction in emergency/out-of-warranty spend	Out-of-warranty repair spend %; asset utilisation %
O6	Make onboarding "day-one ready"	40% new hires wait > 1 day for kit	≥ 95% provisioned before start date	% new hires "ready on day 0"
O7	Guarantee clean offboarding (no orphaned assets/licenses)	8–15% assets unrecovered	≤ 1% unrecovered after 30 days; 0 licenses left active	Recovery rate; active-license-after-exit count
O8	Audit & compliance readiness	2–3 week audit prep	Evidence pack generated in < 1 hour	Time to produce audit evidence; audit findings count
O9	Multi-tenant isolation & scale	Single-scope	0 cross-tenant data leaks; 500+ tenants	Tenant isolation test pass; P95 API latency at scale

A.2 Scope

In Scope

- Hardware assets (endpoints, mobile, network, servers, peripherals, IoT/OT) and consumables/stock.
- Software & SaaS licenses, subscriptions, seat allocation, and entitlement tracking.
- Full lifecycle: request → procure → receive → deploy → operate → maintain → transfer → return → retire → dispose.
- Configurable multi-step approval workflows (existing, extended).
- Deterministic 3-way invoice verification (PO ↔ GRN ↔ Invoice), extended from existing invoice verification.
- Employee onboarding/offboarding orchestration triggered by HRIS.
- QR/barcode/asset-tag generation and mobile scan (Expo app).
- RBAC by permission strings + data scopes (existing), multi-tenant SaaS.
- Immutable audit log, warranty/AMC/contract management, compliance & GDPR data handling.
- Reporting, dashboards, and audit evidence packs.

Out of Scope (this phase)

- Deep endpoint agent-based discovery/patching (integrate with Intune/Lansweeper rather than rebuild).
- Full ITSM incident/problem/change management (integrate with ServiceNow/Freshservice; expose CMDB).
- Financial GL/ERP posting (integrate to NetSuite/SAP/QuickBooks; TechpioAsset raises PR/PO, does not post journals).
- HR system of record (consume HRIS events; not replace it).
- Physical disposal logistics/e-waste hauling (record certificates; do not operate logistics).
- Endpoint security/EDR, MDM enrollment engine (integrate).

A.3 Stakeholders — RACI

Legend: **R** Responsible · **A** Accountable · **C** Consulted · **I** Informed

Activity ↓ / Role →	Employee / Requester	Line Manager	IT Asset Mgr	Procurement	Finance / AP	Vendor	Security/Compliance	Auditor	Tenant Admin	HR
Raise asset request	R/A	C	I	I	I	–	I	–	I	I
Approve request (managerial)	I	R/A	C	I	I	–	I	–	I	I
Finance approval (over threshold)	I	I	C	C	R/A	–	I	–	I	–
Create Purchase Request/PO	I	I	C	R/A	C	I	I	–	I	–
Goods receipt (GRN)	–	–	R/A	C	I	C	I	–	I	–
Invoice 3-way match	I	–	C	C	R/A	I	I	I	I	–
Asset creation & tagging	I	–	R/A	I	I	–	C	–	I	–
Assignment & acceptance	R (accept)	I	A	–	–	–	I	–	I	–
Maintenance/repair	I	I	R/A	C	I	C	I	–	I	–
Transfer/return	R/C	C	R/A	I	I	–	I	–	I	–
Retirement & disposal	I	I	R	C	C	C	A	I	A	–
License seat grant/revoke	I	C	R/A	C	C	C	C	–	I	I

Activity ↓ / Role →	Employee / Requester	Line Manager	IT Asset Mgr	Procurement	Finance / AP	Vendor	Security/Compliance	Auditor	Tenant Admin	HR
Onboarding orchestration	I	C	R	C	I	–	C	–	A	R/A
Offboarding & clearance	I (return)	C	R	I	C	–	C	I	A	R/A
Compliance/audit evidence	–	–	C	–	C	–	R/A	R	I	–
Tenant config & RBAC	–	–	C	–	–	–	C	–	R/A	–

A.4 Key User Journeys (narrative)

J1 — Employee gets a laptop. Priya (Marketing) opens the self-service catalogue, requests the "Standard Laptop Kit." The system pre-checks entitlement/stock, routes to her manager, then to Finance because the kit exceeds the $1,500 threshold. On approval, Procurement issues a PO (or fulfils from stock if available). On receipt/allocation, an asset record is created, QR-tagged, and assigned to Priya. She receives a notification, opens the mobile app, scans the QR, reviews terms, and e-signs acceptance. Asset state → **Active/In-Use**; audit log records the chain.

J2 — IT onboards a new hire. HRIS emits a `new_hire` event 5 days before start. TechpioAsset instantiates an onboarding case from the role's provisioning template (kit + licenses + access), pre-allocates stock or triggers procurement, grants license seats (respecting seat ceilings), stages device assignment, and tracks a checklist to "day-0 ready." New hire e-signs acceptance on or before day 1.

J3 — Finance verifies an invoice. Vendor invoice arrives (email/EDI/upload). The deterministic engine performs a 3-way match: PO lines ↔ GRN quantities ↔ invoice lines, within tolerance. Clean matches auto-approve for payment scheduling; exceptions (price/qty/tax variance, missing GRN, duplicate) are routed to AP with the exact failed rule cited. Cost, once verified and written, is immutable on the asset.

J4 — Auditor runs compliance. An external auditor with a read-only, time-boxed scope runs the "License Compliance & Asset Custody" evidence pack: seat ledgers per SKU, custody chain per asset (with e-signatures), disposal certificates, and the immutable audit trail. Output is a signed, exportable evidence bundle produced in minutes.

A.5 Core Business Processes

1. **Demand & Request Management** — self-service catalogue, entitlement pre-check, standard vs. non-standard kits.
2. **Approval Orchestration** — configurable multi-step, threshold-conditional, delegation & SLA escalation.
3. **Procurement** — PR → PO → vendor → receipt, or fulfil-from-stock.
4. **Financial Verification** — deterministic 3-way match, exception handling, cost write-once.
5. **Inventory & Stock** — stock-in, reservations, reorder points, consumables.
6. **Asset Lifecycle** — creation, tagging, assignment, acceptance, operation, maintenance, transfer, return, retirement, disposal.
7. **License & Entitlement Management** — seat ledgers, allocation/reclaim, renewal, true-up.
8. **Warranty/AMC/Contract Management** — coverage, alerts, claims.
9. **Employee Lifecycle Orchestration** — onboarding/offboarding via HRIS.
10. **Compliance, Audit & Reporting** — immutable trail, GDPR, evidence packs.
11. **Multi-Tenancy & Administration** — tenant isolation, RBAC, data scopes, retention policy.

A.6 Business Rules Catalogue

BR#	Category	Rule	Enforcement
BR-01	RBAC	Every action is gated by a permission string; the effective permission set is the intersection of role grants and the actor's data scope (tenant, org unit, location, cost centre).	API guard; deny-by-default
BR-02	RBAC	A user may never act on an asset/record outside their data scope, even with the permission string.	Row-level scope filter
BR-03	Multi-tenancy	All data is tenant-partitioned; no query may cross tenant boundaries. Tenant ID is derived from the authenticated principal, never from client input.	Middleware-injected tenant filter; isolation tests
BR-04	Approvals	Requests route through the tenant's configured multi-step chain; each step must be approved by a principal distinct from the requester (segregation of duties).	Workflow engine; SoD check
BR-05	Approvals — Threshold	If estimated cost ≥ tenant Finance threshold (configurable, default $1,500), a mandatory Finance approval step is inserted after managerial approval. Multiple bands may add CFO/exec steps.	Conditional routing on cost band
BR-06	Approvals — Delegation/SLA	Pending approvals escalate after the configured SLA; delegation preserves SoD (a delegate cannot approve their own request).	Timer + escalation
BR-07	Cost — Write-once	Verified asset acquisition cost is immutable once written (post 3-way match). Corrections require a new, linked adjustment record with reason + approver; the original is never mutated.	Append-only cost ledger
BR-08	Invoice Verification	An invoice is payable only when PO ↔ GRN ↔ Invoice match on quantity, unit price, currency, and tax within tolerance (default 0% qty, ±2% price). Any failure blocks auto-approval and cites the exact failed rule.	Deterministic match engine
BR-09	Invoice Verification	Duplicate invoice (same vendor + invoice number, or same PO+amount fingerprint) is auto-blocked.	Idempotency/fingerprint check
BR-10	Goods Receipt	An asset may not reach "Active/In-Use" without a GRN (or a documented stock-issue reference for fulfil-from-stock).	State gate
BR-11	License — Seat Limit	Seats consumed for a SKU must never exceed seats owned per tenant. Grant is rejected when the ledger is exhausted; only reclaim or purchase increases availability.	Seat ledger constraint
BR-12	License — Reclaim	On unassignment/return/offboarding, the seat is reclaimed to the pool atomically; no seat is left "orphaned active."	Transactional reclaim
BR-13	License — Renewal/True-up	Subscriptions nearing expiry raise renewal tasks; over-deployment discovered at true-up creates a compliance exception, never a silent grant.	Renewal scheduler
BR-14	Warranty/AMC	Every asset carries warranty and (optional) AMC coverage dates; expiry within the alert window (default 30/60/90 days) generates renewal/claim tasks. Out-of-warranty repairs require explicit approval.	Coverage calendar + alerts

BR#	Category	Rule	Enforcement
BR-15	Acceptance	Assignment is not complete until the assignee e-signs acceptance of custody terms; unaccepted assignments expire and revert to stock after the configured window.	E-signature gate
BR-16	Immutability/Audit	Every state change, approval, cost write, assignment, and disposal is recorded in an append-only, tamper-evident audit log (hash-chained). Records are never updated or deleted.	Append-only + hash chain
BR-17	Data Retention/GDPR	Personal data (assignee identity, signatures) is retained per policy; on lawful erasure request, PII is pseudonymised while preserving the immutable asset/custody chain. Retention windows are per-tenant and per-data-class.	Retention engine; pseudonymisation
BR-18	Disposal	Retirement → disposal requires a sanctioned data-sanitisation record (wipe/destroy certificate) and, for regulated tenants, a disposal certificate before final "Disposed" state.	Disposal gate
BR-19	Offboarding Gate	Offboarding cannot be marked complete while any asset is outstanding or any paid license remains active for the leaver, unless a documented exception is approved by an authorised role.	Clearance gate + exception log
BR-20	Uniqueness/Integrity	Asset tag, serial number, and QR payload are unique within a tenant; a serial cannot be assigned to two active asset records.	Unique constraints
BR-21	State Machine	Asset state transitions follow the defined lifecycle graph; illegal transitions (e.g., Disposed → In-Use) are rejected.	Guarded state machine
BR-22	Reservation	Stock reserved for an approved request cannot be double-allocated; reservations expire and release if unfulfilled within the window.	Reservation lock
BR-23	Currency/Tax	Costs are stored in tenant base currency with FX rate + original currency retained; tax treatment follows tenant jurisdiction config.	Money type + FX snapshot

---

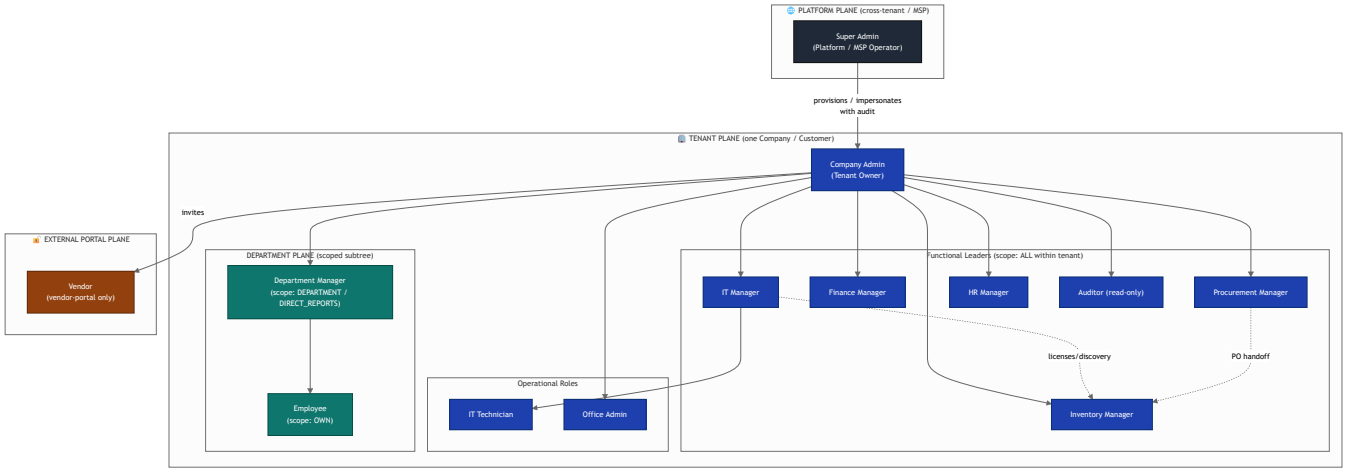
PART 2

Roles, Permission Model & Taxonomy

1. Complete Role Hierarchy

1.1 Governance layering (multi-tenant / MSP)

The platform enforces **three authority planes**. A principal is always anchored to exactly one plane; scopes narrow authority *within* a plane, they never cross a plane boundary upward.



1.2 How the planes work

Plane	Anchored roles	Tenant visibility	Purpose
Platform / MSP	Super Admin	**Cross-tenant** (all customers the operator manages)	Runs the SaaS/MSP fleet: provisions tenants, sets platform policy, aggregates fleet KPIs, supports customers via **audited impersonation**.
Tenant (Company)	Company Admin + all functional/operational roles	**Single tenant only** (hard row-level isolation)	One customer's ITAM world. Company Admin is the sovereign; functional leaders own their module vertically.
Department	Department Manager, Employee	**Subtree of one tenant**	Local, self-service and team-level asset ownership.
External Portal	Vendor	**Only records explicitly shared with them** (their POs, RMAs, contracts)	Zero access to the tenant app; a walled portal keyed to `vendor_id`.

1.3 Scope semantics (the four data scopes)

Every permission a role holds is evaluated against the **narrowest** scope granted for that permission group:

Scope	Row filter applied	Typical holder
`ALL`	All rows in the tenant (Super Admin: all rows in all managed tenants)	Company Admin, functional managers, Auditor (read)
`DEPARTMENT`	`record.department_id ∈ manager's managed departments`	Department Manager, Office Admin (site-scoped)
`DIRECT_REPORTS`	`record.owner_id ∈ {users reporting to me}`	Department Manager (approvals)
`OWN`	`record.owner_id = me` OR `record.requester_id = me`	Employee, and every role for *their personal* assets

Composition rule: effective access = `permission_granted(role) AND scope_filter(role, permission_group)`. A permission with no scope resolves to `OWN` by default (fail-closed). Super Admin's `ALL` is additionally gated by an **MSP tenant-membership** check so an operator only sees tenants under their contract.

1.4 Legacy → canonical mapping (keep existing functionality)

Legacy role (8)	Canonical role	Notes
Admin	Company Admin	1:1
Manager	IT Manager	default; can be reassigned to Department Manager
Asset Manager	Inventory Manager	renamed, scope widened
Staff / User	Employee	scope `OWN`
Finance	Finance Manager	1:1
HR	HR Manager	1:1
Auditor	Auditor	read-only invariant enforced
Guest	Vendor **or** Employee (read)	routed by portal type

New roles introduced by the redesign: **Super Admin (MSP), IT Technician, Procurement Manager, Office Admin, Department Manager.**

---

2. Role Profiles

> Each block: Responsibilities · Data scope · Dashboard · Key permissions · Primary workflows · Reports · Notifications · Mobile · Top KPIs.

2.1 Super Admin — *Platform / MSP Operator (cross-tenant)*

- **Responsibilities:** Provision/suspend tenants; set platform-wide policy, feature flags, SSO/SCIM, data-residency; manage license entitlements sold to tenants; fleet health; billing; audited customer support impersonation.
- **Data scope:** `ALL` across **all managed tenants** (MSP membership-gated).
- **Dashboard:** Fleet map (tenant count, active/suspended), aggregate asset count by tenant, per-tenant health & storage, license-pool consumption across fleet, discovery-agent connectivity %, security/incident feed, MRR/seat usage, impersonation-session log.
- **Key permissions:** `platform:tenant:*`, `platform:policy:*`, `platform:billing:*`, `platform:impersonate:execute`, `admin:*:read (cross-tenant)`, `discovery:connector:manage`.
- **Primary workflows:** Onboard tenant → seed roles/policies → assign entitlements; suspend/offboard with data-export; global patch/agent rollout; escalated support via impersonation.
- **Reports:** Fleet inventory rollup, cross-tenant license compliance, SLA/uptime, per-tenant consumption/billing, security posture.
- **Notifications:** Tenant provisioning events, agent/connector outages, quota breaches, security alerts, failed billing, impersonation start/stop.
- **Mobile:** Fleet health tile, critical-incident push, approve/suspend tenant, kill a runaway job.
- **Top KPIs:** Active tenants, fleet asset count, agent uptime %, cross-tenant compliance %, MRR, open Sev-1s.

Sees across	Can mutate	Cannot
All tenants (contracted)	Platform config, tenant lifecycle	Read tenant business data without an audited impersonation session

2.2 Company Admin — *Tenant Owner*

- **Responsibilities:** Full sovereignty of one tenant: users/roles, org tree, custom roles, integrations, categories/CMDB config, approval chains, branding, retention.
- **Data scope:** `ALL` within tenant.
- **Dashboard:** Total assets by status/category, license compliance gauge, open requests/approvals, maintenance due, spend YTD vs budget, user/seat count, warranty-expiry funnel, audit-exceptions tile.
- **Key permissions:** `admin:*`, `*:*:* (scope ALL, tenant-bounded)` except platform plane; can create **custom roles** and edit approval workflows.
- **Primary workflows:** User/role provisioning; configure procurement approval chains; category & lifecycle setup; integration/SSO setup; period-end audit sign-off.
- **Reports:** Everything tenant-wide — inventory, compliance, spend, utilization, audit.
- **Notifications:** Approval escalations, compliance breaches, contract/warranty expiries, integration failures, unusual-activity alerts.
- **Mobile:** Approve anything, user lock/unlock, exec KPI board, config toggles.
- **Top KPIs:** Total asset value, license compliance %, request cycle time, spend vs budget, audit exceptions.

2.3 IT Manager

- **Responsibilities:** Owns hardware/software estate, discovery, licensing strategy, maintenance program, IT team & queue.
- **Data scope:** `ALL` (tenant) for IT asset/license/discovery/maintenance modules.
- **Dashboard:** Asset health by lifecycle, license compliance & true-up risk, discovery drift (unmanaged devices), maintenance SLA, technician workload, patch/EOL calendar, warranty pipeline.
- **Key permissions:** `assets:*`, `license:*`, `discovery:*`, `maintenance:*`, `inventory:read`, `requests:approve (IT)`, `reports:read`.
- **Primary workflows:** Triage IT requests → assign technician; license true-up; run discovery reconciliation; schedule maintenance/patch cycles; approve IT procurement specs.
- **Reports:** License compliance/position, hardware lifecycle & EOL, discovery reconciliation, maintenance SLA, technician throughput.
- **Notifications:** License overage, EOL/warranty, failed discovery sync, SLA breach, high-priority incident.
- **Mobile:** Approve IT requests, reassign tickets, scan asset, view technician map.
- **Top KPIs:** License compliance %, unmanaged-device %, MTTR, patch coverage %, warranty-at-risk count.

2.4 IT Technician

- **Responsibilities:** Execute assignments — image/deploy, check-in/out, repairs, moves, discovery agent installs.
- **Data scope:** `ALL` (read) for IT assets; **write** scoped to assigned tickets/assets (`DEPARTMENT` or assignment-based).
- **Dashboard:** My queue, assets awaiting deploy, check-outs due back, maintenance tasks today, low-stock consumables, recent scans.
- **Key permissions:** `assets:read`, `assets:update (assigned)`, `assets:checkin`, `assets:checkout`, `maintenance:execute`, `inventory:consume`, `discovery:agent:install`.
- **Primary workflows:** Assign/return asset to user; complete maintenance task; log repair; adjust stock on consume; onboard/offboard device.
- **Reports:** My task history, assets by location, maintenance-completed, check-out aging.
- **Notifications:** New assignment, overdue return, part arrived, escalation.
- **Mobile (heavy):** Barcode/QR/RFID scan, check-in/out, photo-attach condition, offline queue,签 signature capture, geo-stamp.
- **Top KPIs:** Tickets closed/day, on-time task %, overdue returns, mean handle time.

2.5 HR Manager

- **Responsibilities:** Owns joiner-mover-leaver (JML) asset events, personal issuance policy, offboarding asset recovery, acknowledgment/e-sign of custody.
- **Data scope:** `ALL` for **people/HR**; `read` on assets **assigned to employees**; no financial cost visibility beyond issuance.
- **Dashboard:** New hires needing kit, leavers with unreturned assets, acknowledgment-signature status, per-employee asset custody, policy exceptions.
- **Key permissions:** `people:*`, `assets:read (assigned-to-people)`, `requests:create (onboarding)`, `requests:approve (people)`, `reports:read (custody)`.
- **Primary workflows:** Onboarding kit request → issuance; offboarding recovery checklist → clearance; custody acknowledgment e-sign; org-tree updates feeding scopes.

- **Reports:** Custody by employee/department, offboarding recovery, unreturned-on-exit, acknowledgment compliance.
- **Notifications:** New hire, termination date approaching, unreturned asset at exit, unsigned custody.
- **Mobile:** Approve onboarding kits, view employee custody, trigger offboarding, sign-off recovery.
- **Top KPIs:** Onboarding-kit SLA, offboarding recovery %, unreturned-at-exit count, acknowledgment %.

2.6 Finance Manager

- **Responsibilities:** Capitalization, depreciation, budgets, invoices/AP matching, TCO, cost centers, chargeback/showback.
- **Data scope:** `ALL` for **finance/invoices**; `read` on assets/procurement (cost view).
- **Dashboard:** Asset book value & depreciation curve, spend vs budget by cost center, open invoices/3-way-match exceptions, upcoming renewals cost, TCO by category, chargeback summary.
- **Key permissions:** `invoices:*`, `finance:*`, `procurement:read`, `assets:read (financials)`, `license:read (cost)`, `reports:*`.
- **Primary workflows:** PO→invoice 3-way match & approve; run depreciation period-close; budget vs actual review; renewal cost forecasting; cost-center allocation.
- **Reports:** Depreciation schedule, TCO, budget variance, invoice-aging, renewal cost forecast, chargeback/showback.
- **Notifications:** Invoice-match exception, budget overrun, renewal cost due, month-end close tasks.
- **Mobile:** Approve invoices, view spend tile, budget alerts.
- **Top KPIs:** Spend vs budget, net book value, TCO/asset, invoice-match exception rate, renewal cost at risk.

2.7 Procurement Manager

- **Responsibilities:** Requisitions→POs, vendor/catalog management, sourcing, contract terms, receiving oversight.
- **Data scope:** `ALL` for **procurement**; `read` /manage on **vendor-portal** invitations; `read` inventory & assets.
- **Dashboard:** Open requisitions by stage, POs pending/receiving, vendor SLA/lead-time, catalog price trends, contract-renewal pipeline, spend by vendor.
- **Key permissions:** `procurement:*`, `vendor-portal:invite`, `vendor-portal:manage`, `inventory:read`, `invoices:read`, `assets:create (on receipt)`.
- **Primary workflows:** Approve requisition → raise PO → dispatch to vendor portal → track → confirm receipt handoff to Inventory.
- **Reports:** PO cycle time, vendor performance/scorecard, price variance, contract-renewal calendar, spend-by-category.
- **Notifications:** New requisition, PO acknowledged/shipped by vendor, late delivery, contract renewal, price change.
- **Mobile:** Approve requisition/PO, vendor status, receive-confirm.
- **Top KPIs:** PO cycle time, on-time delivery %, price variance, vendor SLA %, contract renewals at risk.

2.8 Inventory Manager

- **Responsibilities:** Stockroom/warehouse, stock levels, GRN/receiving, transfers, cycle counts, consumables, bin/location.
- **Data scope:** `ALL` for **inventory** (optionally `DEPARTMENT` /site-scoped in multi-site tenants).
- **Dashboard:** Stock on hand vs reorder, incoming receipts, low-stock/out-of-stock, transfers in-flight, cycle-count variance, shrinkage, aging stock.
- **Key permissions:** `inventory:*`, `assets:create (from stock)`, `assets:transfer`, `procurement:read (POs to receive)`, `reports:read`.
- **Primary workflows:** Goods receipt (GRN) against PO → stock-in → convert to trackable asset → transfer/issue; cycle count & reconcile; reorder trigger.
- **Reports:** Stock valuation, reorder report, cycle-count variance, movement/turns, shrinkage.
- **Notifications:** Reorder point hit, incoming shipment, count variance, expiry (consumables), transfer received.
- **Mobile (heavy):** Scan receive, bin move, cycle-count mode, stock lookup, adjust qty.
- **Top KPIs:** Stock accuracy %, stockout rate, inventory turns, cycle-count variance, shrinkage %.

2.9 Office Admin

- **Responsibilities:** Site/facility assets (furniture, peripherals, access cards, keys), local issuance, front-desk requests — one or more sites.
- **Data scope:** `DEPARTMENT` (site-scoped) for assets/inventory; `read` broader.
- **Dashboard:** Site asset roster, local check-outs, pending local requests, low office-supply stock, visitor/loaner equipment, upcoming returns.
- **Key permissions:** `assets:read (site)`, `assets:checkin/out (site)`, `inventory:read/consume (site)`, `requests:create`, `requests:approve (site, low-value)`.
- **Primary workflows:** Local asset issuance/return; loaner desk; office-supply replenish request; new-desk setup coordination.
- **Reports:** Site inventory, local check-out aging, supply consumption.
- **Notifications:** Local request, overdue loaner, low supplies, new-hire desk setup.
- **Mobile:** Scan/check-out at desk, approve low-value site request, supply reorder.
- **Top KPIs:** Local request SLA, loaner return %, site asset accuracy, supply stockouts.

2.10 Department Manager

- **Responsibilities:** Approves team asset/software requests; owns department budget consumption; visibility into team custody.
- **Data scope:** `DEPARTMENT` + `DIRECT_REPORTS` (approvals over reports; view over department).
- **Dashboard:** Team assets & value, pending approvals, department spend vs allocation, licenses assigned to team, unreturned by leavers on team, requests in flight.
- **Key permissions:** `requests:approve (DIRECT_REPORTS)`, `assets:read (DEPARTMENT)`, `license:read (DEPARTMENT)`, `people:read (DIRECT_REPORTS)`, `reports:read (DEPARTMENT)`.
- **Primary workflows:** First-level approve/reject team requests; endorse hardware refresh; confirm team custody in audits; department budget check.
- **Reports:** Team custody, department spend, license utilization (team), request approvals log.
- **Notifications:** Approval pending, budget threshold, team member offboarding, audit confirmation due.
- **Mobile:** One-tap approve/reject, team asset list, budget tile.
- **Top KPIs:** Approval turnaround, department spend vs budget, team license utilization, custody-confirmed %.

2.11 Employee

- **Responsibilities:** Requests assets/software; acknowledges custody; reports issues; returns assets.
- **Data scope:** `OWN`.
- **Dashboard:** My assets, my open requests, my licenses, return-due items, my tickets, acknowledgment tasks.
- **Key permissions:** `assets:read (OWN)`, `requests:create`, `requests:read (OWN)`, `maintenance:report (OWN)`, `license:read (OWN)`, `people:read (self)`.
- **Primary workflows:** Raise request → track → receive → acknowledge; report fault; return on exit/refresh.
- **Reports:** My assets/custody statement (self-service).
- **Notifications:** Request status, asset assigned, return due, acknowledgment required, maintenance scheduled for my device.
- **Mobile (heavy):** Self-service catalog request, scan-to-report-issue, e-sign custody, view my kit, check-in return.
- **Top KPIs:** My open requests, items due for return, acknowledgment status.

2.12 Auditor — *read-only invariant*

- **Responsibilities:** Independent assurance — inventory accuracy, license compliance, control effectiveness, evidence collection; **no mutation anywhere**.
- **Data scope:** `ALL` **read-only** (tenant); Super-Auditor variant may be cross-tenant for MSP compliance.
- **Dashboard:** Compliance heatmap, audit-exception register (read), evidence completeness, control-test status, change/access log viewer, discovery-vs-record variance.
- **Key permissions:** `*:*:read` only; `audit:evidence:export`; **explicitly denied** every `create/update/delete/approve` (enforced by a hard invariant, see §3).
- **Primary workflows:** Sample assets → verify existence → log finding (in a separate immutable findings store, not the asset records) → export evidence pack.
- **Reports:** Full read of all reports + audit-trail, access-review, license-compliance, reconciliation, SoD-conflict.
- **Notifications:** New audit period, control-test due, evidence-gap alerts (informational only).
- **Mobile:** Read dashboards, scan-to-verify (records verification event in findings log only), export.
- **Top KPIs:** Inventory accuracy %, open findings, evidence completeness %, compliance score.

2.13 Vendor — *External Portal (walled)*

- **Responsibilities:** External supplier — acknowledges POs, updates shipment/delivery, submits invoices, manages RMAs/warranty claims, maintains catalog entries.
- **Data scope: Only records linked to** `vendor_id` — their POs, contracts, RMAs, invoices. No tenant app access.
- **Dashboard (portal):** POs to acknowledge, shipments in progress, open RMAs, invoice status, contract/SLA summary, catalog items.
- **Key permissions:** `vendor-portal:po:read`, `vendor-portal:po:acknowledge`, `vendor-portal:shipment:update`, `vendor-portal:invoice:submit`, `vendor-portal:rma:manage`, `vendor-portal:catalog:update`.
- **Primary workflows:** Acknowledge PO → update shipping/tracking → submit invoice → handle RMA/warranty.
- **Reports (portal):** My PO history, delivery performance (self), invoice/payment status.
- **Notifications:** New PO, delivery reminder, RMA opened, invoice approved/rejected, contract renewal.
- **Mobile (portal):** Acknowledge PO, update tracking, upload invoice/shipping docs.
- **Top KPIs:** PO acknowledgment time, on-time delivery %, invoice acceptance %, open RMAs.

---

3. Consolidated Permission Matrix

Legend: ✓ = full (create/update/delete/manage within scope) · **R** = read-only · **A** = approve only · blank = no access. Scope abbreviations after some cells: `[ALL]`, `[DEPT]`, `[DR]` =direct reports, `[OWN]`, `[SITE]`, `[V]` =own vendor records.

Permission Group	Super Admin	Company Admin	IT Mgr	IT Tech	HR Mgr	Finance Mgr	Procure Mgr	Inventory Mgr	Office Admin	Dept Mgr	Employee	Auditor	Vendor
assets: (lifecycle, assign)	R[1]	✓[ALL]	✓[ALL]	✓[2]	R[ppl]	R[fin]	R	✓[ALL]	✓[SITE]	R[DEPT]	R[OWN]	R	
license: (SW, entitlements)	R[1]	✓[ALL]	✓[ALL]	R		R[cost]	R	R		R[DEPT]	R[OWN]	R	
procurement: (req, PO, catalog)	R[1]	✓[ALL]	A(IT)			R[cost]	✓[ALL]	R	R	A[DR]	R[OWN]	R	
inventory: (stock, GRN, count)	R[1]	✓[ALL]	R	✓(consume)		R	R	✓[ALL]	✓[SITE]			R	
maintenance: (repair, patch)	R[1]	✓[ALL]	✓[ALL]	✓(exec)		R		R	R[SITE]	R[DEPT]	R(report) [OWN]	R	
requests/approvals:	R[1]	✓[ALL]	A(IT)	R	A(ppl)	A(fin)	A(proc)	R	A[SITE][3]	A[DR]	✓(create) [OWN]	R	
invoices/finance:	R[1]	✓[ALL]	R			✓[ALL]	R	R		R[DEPT]		R	R(submit) [V][4]
people/HR: (org, custody)	R[1]	✓[ALL]		R(assigned)	✓[ALL]				R[SITE]	R[DR]	R(self)	R	
reports:	✓(fleet)	✓[ALL]	R[ALL]	R(own)	R(custody)	✓[ALL]	R[proc]	R[inv]	R[SITE]	R[DEPT]	R[OWN]	R[ALL]	R[V]
audit: (trail, findings, evidence)	✓(platform)	R	R			R						✓(evidence)[5]	
admin/config: (roles, workflow, SSO)	✓(platform)	✓[tenant]	R(IT cfg)		R(HR cfg)	R(fin cfg)	R(proc cfg)	R(inv cfg)				R	
vendor-portal:	✓(oversee)	✓(manage)					✓(invite/mgr)	R				R	✓[V]

Footnotes: 1. Super Admin sees tenant business data **only inside an audited impersonation session**; the "R" reflects that gated read, not ambient access. 2. IT Technician `assets` write is **assignment-scoped** (assets on their ticket/queue), read is tenant-wide. 3. Office Admin approvals are limited to **low-value / site-local** request types (threshold configurable). 4. Vendor `invoices:submit` writes only into their own vendor-portal submission, not tenant AP records. 5. Auditor `audit:evidence` writes go to an **append-only findings store**, never to operational records.

3.1 Runtime-configurable custom roles + read-only invariant

- **Custom roles:** Company Admin (and Super Admin at platform level) can compose new roles at runtime from the permission-string catalog (§5) plus a scope selector per permission group. Custom roles inherit the same evaluation engine — no code deploy. Guardrails: (a) a custom role can never be granted `platform:*`; (b) `admin:role:manage` cannot be self-escalated (a role cannot grant permissions it doesn't itself hold — **no privilege escalation**); (c) SoD conflict warnings surface when e.g. `procurement:po:approve` + `invoices:approve` land on one role.
- **Auditor read-only invariant (hard):** the Auditor archetype (and any role flagged `readonly: true`) is evaluated through a **deny-all-mutations middleware** that rejects every `create|update|delete|approve|execute|manage` action *before* permission resolution — it cannot be overridden by adding permission strings, by a custom role, or by scope. This invariant is enforced at the API gateway, not just the UI, and is covered by a standing test.

---

4. RACI — Core End-to-End Flow

Flow: **Request** → **Procure** → **Receive** → **Assign** → **Maintain** → **Retire** R = Responsible · A = Accountable · C = Consulted · I = Informed. (Roles with no letter are not involved.)

Stage	Employee	Dept Mgr	Office Admin	IT Mgr	Procure Mgr	Inventory Mgr	IT Tech	Finance Mgr	HR Mgr	Vendor	Company Admin	Auditor
1. Request (raise & justify)	**R**	**A**	C	C	I			I	C(JML)		I	I
2. Procure (approve req→PO)	I	C		C(spec)	**R**	I		**A**(budget)		C	I	I
3. Receive (GRN, stock-in)			C(site)	I	C	**R/A**	C	I		**R**(ship)	I	I
4. Assign (issue to user)	I	C	R(site)	A		C	**R**		C(custody)		I	I
5. Maintain (repair/patch)	C(report)	I		**A**	I	C(parts)	**R**	I		C(warranty)		I
6. Retire (decom/dispose)	I	C		**A**	I	C(stock)	**R**	**C**(write-off)	C(recovery)	C(RMA)	I	**C**(evidence)

Reading notes: - **Accountability shifts by domain:** Dept Mgr owns the *request*, Finance owns the *spend decision*, IT Mgr owns *technical assign/maintain/retire*, Inventory owns *physical receipt*. - **Auditor is Consulted/Informed only** at every stage — consistent with the read-only invariant; it validates evidence at retire (asset disposal is the highest audit-risk step). - **Vendor** is Responsible only for the shipment sub-step of Receive and Consulted on warranty/RMA in Maintain/Retire.

---

5. New Permission Taxonomy (implementation-ready)

Format: `module:resource:action` — plus a per-grant `scope` field. Wildcards resolve left-to-right (`assets:*` ⇒ all resources/actions in the assets module, still scope-filtered).

Module	Representative strings	Notes
`assets`	`assets:asset:{create,read,update,delete,checkout,checkin,transfer,dispose}`, `assets:category:manage`, `assets:custody:acknowledge`	Core; `dispose` is SoD-sensitive.
`license`	`license:license:{create,read,update,delete,allocate,reclaim}`, `license:entitlement:manage`, `license:compliance:read`	Software Asset Management.
`procurement`	`procurement:requisition:{create,read,approve}`, `procurement:po:{create,read,approve,dispatch,close}`, `procurement:catalog:manage`, `procurement:contract:manage`	Approval steps map to workflow engine.
`inventory`	`inventory:stock:{read,adjust,transfer}`, `inventory:grn:receive`, `inventory:count:execute`, `inventory:consumable:consume`, `inventory:location:manage`	Warehouse/stockroom.
`discovery`	`discovery:agent:{install,read,manage}`, `discovery:connector:manage`, `discovery:reconcile:execute`, `discovery:unmanaged:read`	Network/agent auto-discovery (Lansweeper/Intune-class).
`maintenance`	`maintenance:workorder:{create,read,execute,close}`, `maintenance:schedule:manage`, `maintenance:patch:deploy`, `maintenance:fault:report`	
`requests`	`requests:request:{create,read,approve,reject,fulfill}`, `requests:workflow:manage`	Self-service catalog.
`invoices` / `finance`	`invoices:invoice:{read,match,approve}`, `finance:depreciation:run`, `finance:budget:manage`, `finance:costcenter:manage`, `finance:chargeback:read`	3-way match, book value.
`people`	`people:employee:{read,manage}`, `people:org:manage`, `people:onboarding:trigger`, `people:offboarding:trigger`	Feeds scope resolution.
`reports`	`reports:report:read`, `reports:builder:manage`, `reports:export`	
`audit`	`audit:trail:read`, `audit:finding:manage`, `audit:evidence:export`, `audit:accessreview:read`	Findings store is append-only.
`admin` / `config`	`admin:user:manage`, `admin:role:manage`, `admin:workflow:manage`, `admin:integration:manage`, `admin:sso:manage`	Tenant config.
`platform`	`platform:tenant:{provision,suspend,offboard}`, `platform:policy:manage`, `platform:billing:manage`, `platform:impersonate:execute`	**Super Admin only** — never grantable to a custom tenant role.
`vendor-portal`	`vendor-portal:po:{read,acknowledge}`, `vendor-portal:shipment:update`, `vendor-portal:invoice:submit`, `vendor-portal:rma:manage`, `vendor-portal:catalog:update`, `vendor-portal:invite`, `vendor-portal:manage`	Split: portal-side (Vendor) vs tenant-side management (Procurement/Admin).

Design invariants baked into the taxonomy: 1. **Fail-closed default** — absence of a grant = deny; missing scope = `OWN`. 2. **Plane isolation** — `platform:*` cannot appear in any tenant/custom role. 3. **No self-escalation** — `admin:role:manage` can only grant a subset of the granter's own effective permissions. 4. **Read-**

only flag — `readonly` roles (Auditor) are mutation-denied at the gateway regardless of strings held. 5. **Scope is orthogonal** — the same string (`assets:asset:read`) yields tenant-wide, department, or own visibility purely by the attached scope, keeping the string catalog small and the model composable across the SME→Enterprise→MSP spectrum.

PART 3

Data Model, ERD & Status Redesign

1. Separated Status Model (the critical redesign)

1.1 Why the single 18-value enum must die

The current `Asset.status` field conflates five *orthogonal* facts into one column: where the asset is in its life, whether it can be given to someone, its physical shape, who owns it, and where it physically sits. Any real query ("show me all *leased* laptops that are *deployed* but in *poor* condition and physically *in Warehouse-BLR*") is impossible against one enum, and every new business case forces yet another enum value (that is how you get to 18).

The redesign models **five independent dimensions**, each a small closed enum or a relation, so state is a *tuple* not a single label.

1.2 The five dimensions

Dimension	Field on `Asset`	Kind	Values / Target	Semantics
Lifecycle	`lifecycleState`	enum `LifecycleState`	PLANNED, IN_PROCUREMENT, IN_STOCK, DEPLOYED, IN_MAINTENANCE, RETIRED, DISPOSED	Where the asset sits in its cradle-to-grave journey. Monotonic-ish; drives depreciation & CMDB visibility.
Availability	`availabilityState`	enum `AvailabilityState`	AVAILABLE, RESERVED, ASSIGNED, IN_TRANSIT, IN_REPAIR, LOST	Can it be handed out *right now*? Drives the request/reservation engine.
Condition	`conditionGrade`	enum `ConditionGrade`	NEW, GOOD, FAIR, POOR, DAMAGED, END_OF_LIFE	Physical/functional health. Backed by an append-only `ConditionLog` history.
Ownership	`ownershipType`	enum `OwnershipType`	OWNED, LEASED, RENTED, BYOD, LOANER	Financial/legal ownership. Drives depreciation vs. lease-liability accounting.
Location	`currentLocationId` **or** `assignedToUserId`	FK (nullable)	→ `Location` / → `User`	Physical placement. Structured hierarchy (Site→Building→Floor→Room→Warehouse→Bin) **or** "with-user".

These are **independent**: `lifecycleState` does not determine `conditionGrade`, ownership is orthogonal to availability, etc. Guard *illegal* combinations with lightweight rules (see §1.4) rather than by re-merging into one enum.

```
enum LifecycleState   { PLANNED IN_PROCUREMENT IN_STOCK DEPLOYED IN_MAINTENANCE RETIRED DISPOSED }
enum AvailabilityState{ AVAILABLE RESERVED ASSIGNED IN_TRANSIT IN_REPAIR LOST }
enum ConditionGrade   { NEW GOOD FAIR POOR DAMAGED END_OF_LIFE }
enum OwnershipType     { OWNED LEASED RENTED BYOD LOANER }
```

1.3 Where each dimension physically lives

- **Lifecycle / Availability / Condition / Ownership** → four scalar enum columns on `Asset` (fast to filter, cheap to index). Each column is `NOT NULL` with a sensible default.
- **Condition history** → separate `ConditionLog` table (append-only) so the *current* `conditionGrade` on `Asset` is a denormalized cache of the latest log row.
- **Lifecycle history** → `AssetLifecycleEvent` (append-only) records every transition with actor, reason, timestamp — the audit spine of the asset.
- **Location** → a normalized `Location` hierarchy table (self-referencing) referenced by `Asset.currentLocationId`. "With-user" is expressed by `Asset.assignedToUserId` being non-null; a `CHECK` ensures an asset is *either* at a location *or* with a user, never ambiguously both for its "current" placement (the assignment record carries the authoritative link).

1.4 Legal-combination guardrails (not a merged enum)

Enforced in the service layer + a few DB `CHECK`s / partial indexes:

- `availabilityState = ASSIGNED` ⇒ an active `AssetAssignment` row must exist (and `assignedToUserId` set).
- `lifecycleState IN (RETIRED, DISPOSED)` ⇒ `availabilityState` must be `LOST` or the asset unassigned; blocks new assignments.
- `lifecycleState = IN_MAINTENANCE` typically pairs with `availabilityState = IN_REPAIR` — warned, not hard-blocked (an asset can be flagged for maintenance while still physically assigned).
- `conditionGrade = END_OF_LIFE` ⇒ recommend transition to `RETIRED` (soft rule surfaced in UI).

1.5 Mapping the OLD 18 statuses → new dimensions

#	Old single status	LifecycleState	AvailabilityState	ConditionGrade	OwnershipType	Location note
1	`PLANNED`	PLANNED	AVAILABLE	NEW	(inherit)	—
2	`ON_ORDER` / `ORDERED`	IN_PROCUREMENT	RESERVED	NEW	(inherit)	vendor
3	`IN_PROCUREMENT`	IN_PROCUREMENT	RESERVED	NEW	(inherit)	vendor
4	`RECEIVED`	IN_STOCK	AVAILABLE	NEW	OWNED	warehouse/bin
5	`IN_STOCK` / `AVAILABLE`	IN_STOCK	AVAILABLE	GOOD	(inherit)	warehouse/bin
6	`RESERVED`	IN_STOCK	RESERVED	GOOD	(inherit)	warehouse/bin
7	`ALLOCATED`	IN_STOCK	RESERVED	GOOD	(inherit)	staging
8	`DEPLOYED` / `IN_USE`	DEPLOYED	ASSIGNED	GOOD	(inherit)	with-user
9	`ASSIGNED`	DEPLOYED	ASSIGNED	GOOD	(inherit)	with-user
10	`IN_TRANSIT` / `SHIPPING`	DEPLOYED	IN_TRANSIT	GOOD	(inherit)	between locations

#	Old single status	LifecycleState	AvailabilityState	ConditionGrade	OwnershipType	Location note
11	`LOANED` / `ON_LOAN`	DEPLOYED	ASSIGNED	GOOD	LOANER	with-user
12	`LEASED`	DEPLOYED	ASSIGNED	GOOD	LEASED	with-user
13	`UNDER_REPAIR` / `IN_REPAIR`	IN_MAINTENANCE	IN_REPAIR	POOR	(inherit)	service center
14	`IN_MAINTENANCE`	IN_MAINTENANCE	IN_REPAIR	FAIR	(inherit)	service center
15	`DAMAGED`	IN_STOCK / IN_MAINTENANCE	IN_REPAIR	DAMAGED	(inherit)	warehouse
16	`LOST` / `MISSING` / `STOLEN`	DEPLOYED	LOST	(last known)	(inherit)	last known
17	`RETIRED` / `DECOMMISSIONED`	RETIRED	AVAILABLE	END_OF_LIFE	(inherit)	warehouse/disposal
18	`DISPOSED` / `SCRAPPED` / `SOLD`	DISPOSED	AVAILABLE	END_OF_LIFE	(inherit)	—

> Migration: a one-time backfill script maps each legacy row via the table above, writes the four enum columns, seeds a synthetic `AssetLifecycleEvent` (`reason = 'legacy-migration'`) and an initial `ConditionLog` . "(inherit)" = carry the existing ownership/condition if already tracked elsewhere, else default OWNED/GOOD.

---

2. New & Extended Entities (by domain)

Common conventions applied to **every** table unless noted: - `id BIGINT GENERATED ALWAYS AS IDENTITY` (or `UUID DEFAULT gen_random_uuid()` for externally-exposed ids) — **PK**. - `tenantId BIGINT NOT NULL` — **FK → Tenant**, on every tenant-scoped table (see §4.1). - `createdAt timestamptz NOT NULL DEFAULT now()` , `updatedAt timestamptz NOT NULL` . - `createdById / updatedById BIGINT` — FK → User. - `deletedAt timestamptz NULL` — soft-delete tombstone (except immutable ledgers, §4.2). - `version INTEGER NOT NULL DEFAULT 0` — optimistic-lock token (§4.3).

2.1 Tenancy & RBAC

Entity	Purpose	Key columns	Relations
Tenant (`Company`)	Root isolation boundary; the MSP/customer org.	`id` , `name` , `slug UNIQUE` , `plan` , `status` , `settings jsonb` , `parentTenantId?` (for org hierarchy)	1→N everything
User	Person acting in the system (staff, requester, approver).	`id` , `tenantId` , `email` , `displayName` , `employeeCode` , `departmentId?` , `managerId?` , `status` , `externalIdpId?`	N→1 Tenant; N→N Role
Role	Named bundle of permissions (system + custom).	`id` , `tenantId?` (NULL = global/system role), `name` , `isSystem bool` , `description`	N→N Permission; N→N User
Permission	Atomic capability, `resource:action` (e.g. `asset:transfer`).	`id` , `key UNIQUE` , `resource` , `action` , `description`	N→N Role
UserRole (join)	Assigns roles to users, optionally scoped.	`userId` , `roleId` , `scopeType?` , `scopeId?`	join
RolePermission (join)		`roleId` , `permissionId`	join
Department	Org unit for cost allocation & approval routing.	`id` , `tenantId` , `name` , `costCenterId?` , `parentId?`	self-ref tree

Custom roles = `Role` rows with `tenantId` set and `isSystem = false` ; system roles have `tenantId IS NULL` .

2.2 Locations & Warehouse

Single self-referencing hierarchy keeps queries uniform; a `type` discriminator distinguishes levels.

Entity	Purpose	Key columns	Relations
Location	Unified physical-place node (site→building→floor→room→warehouse→bin).	`id` , `tenantId` , `parentId?` , `type LocationType` , `code` , `name` , `address jsonb` , `geo point?` , `capacity int?` , `isStorage bool`	self-ref; ← Asset, ← StockLevel
Warehouse *(view/subtype of Location where `type=WAREHOUSE`)*	Storage facility for stock/inventory.	(as Location) `+ manager userId`	← Bin
Bin *(Location `type=BIN`)*	Smallest addressable storage slot.	(as Location) `+ binClass`	← StockLevel

```
enum LocationType { SITE BUILDING FLOOR ROOM WAREHOUSE ZONE BIN }
```

Unique: `(tenantId, parentId, code)` .

2.3 Catalog

Entity	Purpose	Key columns	Relations
Manufacturer	Maker of hardware/software.	`id` , `tenantId?` , `name` , `website` , `supportUrl`	1→N ProductModel
ProductModel	The *model* (e.g. "Dell Latitude 7440"); assets are instances.	`id` , `tenantId?` , `manufacturerId` , `categoryId` , `name` , `sku` , `specs jsonb` , `defaultDepreciationMethod` , `defaultUsefulLifeMonths` , `imageUrl`	N→1 Manufacturer/Category; 1→N Asset
Category	Asset classification tree (Laptop→Ultrabook…).	`id` , `tenantId?` , `parentId?` , `name` , `assetTagPrefix` , `icon`	self-ref; 1→N ProductModel/Asset

Global catalog rows (`tenantId IS NULL`) are shared; tenants may add private models.

2.4 Assets (core)

Entity	Purpose	Key columns	Relations
Asset	The tracked unit. Carries the four status enums (§1).	`id`, `tenantId`, `assetTag`, `serialNumber?`, `productModelId`, `categoryId`, `lifecycleState`, `availabilityState`, `conditionGrade`, `ownershipType`, `currentLocationId?`, `assignedToUserId?`, `purchaseOrderLineId?`, `warrantyEnd?`, `costBasis numeric(14,2)`, `currency char(3)`, `acquiredAt?`, `retiredAt?`, `parentAssetId?`, `customFields jsonb`, `version`, `deletedAt?`	many (hub entity)
AssetLifecycleEvent	Append-only transition ledger.	`id`, `tenantId`, `assetId`, `fromState?`, `toState`, `eventType`, `reason`, `actorId`, `occurredAt`, `metadata jsonb`	N→1 Asset (immutable)
AssetAssignment	Active/historical custody of an asset by a user (or location).	`id`, `tenantId`, `assetId`, `userId?`, `locationId?`, `assignedAt`, `dueReturnAt?`, `returnedAt?`, `status`, `acknowledgedAt?`, `eSignatureId?`	N→1 Asset/User
AssetTransfer	Movement between users/locations/tenants(intra-org).	`id`, `tenantId`, `assetId`, `fromUserId?`, `toUserId?`, `fromLocationId?`, `toLocationId?`, `status`, `shippedAt?`, `receivedAt?`, `carrier?`, `trackingNo?`	N→1 Asset
AssetReturn	Return of an assigned asset to stock.	`id`, `tenantId`, `assetId`, `assignmentId`, `returnedById`, `receivedById?`, `conditionOnReturn ConditionGrade`, `returnedAt`, `notes`	N→1 Asset/Assignment
ConditionLog	Append-only condition history (source of `Asset.conditionGrade`).	`id`, `tenantId`, `assetId`, `grade ConditionGrade`, `assessedById`, `assessedAt`, `notes`, `photoAttachmentId?`	N→1 Asset (immutable)
AssetDocument	Files tied to an asset (invoice scan, warranty, photo).	`id`, `tenantId`, `assetId`, `attachmentId`, `docType`, `title`	N→1 Asset/Attachment
AssetRelationship	Typed graph edges: parent/child, docked-to, peripheral-of, depends-on.	`id`, `tenantId`, `fromAssetId`, `toAssetId`, `relationType`, `note`	N→N Asset (self)

```
enum AssetRelationType { PARENT_CHILD DOCKED_TO PERIPHERAL_OF CONNECTED_TO DEPENDS_ON REPLACES }
```

`parentAssetId` on `Asset` is a fast denormalized shortcut for the common single-parent case; `AssetRelationship` handles the general many-to-many graph (a monitor peripheral-of a laptop, laptop docked-to a dock).

2.5 Hardware Inventory / Discovery

Populated by agents (Intune/Autotask/custom) — mostly overwrite-on-sync, so these are *current-state* tables plus a raw sync log.

Entity	Purpose	Key columns	Relations
HardwareProfile	Discovered spec snapshot (1:1 with Asset).	`id`, `tenantId`, `assetId UNIQUE`, `cpuModel`, `cpuCores`, `ramBytes bigint`, `storageBytes bigint`, `biosVersion`, `chassisType`, `lastSeenAt`	1→1 Asset
OperatingSystemInfo	OS state of the device.	`id`, `tenantId`, `assetId UNIQUE`, `osFamily`, `osVersion`, `build`, `architecture`, `lastBootAt`, `patchLevel`	1→1 Asset
InstalledSoftware	Software found on a device (drives license reconciliation).	`id`, `tenantId`, `assetId`, `name`, `publisher`, `version`, `installDate`, `softwareLicenseId?` (matched)	N→1 Asset; N→1 SoftwareLicense
NetworkInterface	NICs / IP / MAC for the device.	`id`, `tenantId`, `assetId`, `macAddress`, `ipv4?`, `ipv6?`, `interfaceType`, `isPrimary`	N→1 Asset
SecurityPosture	Compliance signals (1:1 with Asset).	`id`, `tenantId`, `assetId UNIQUE`, `bitlockerEnabled bool`, `tpmVersion`, `defenderEnabled bool`, `defenderSignatureAge int`, `firewallEnabled bool`, `encryptionState`, `complianceState`, `assessedAt`	1→1 Asset
DiscoverySource	A connected agent/integration feeding data.	`id`, `tenantId`, `type`, `name`, `credentialsRef`, `status`, `lastSyncAt`	1→N AgentSyncRun
AgentSyncRun	Immutable log of each sync (raw payload ref, counts).	`id`, `tenantId`, `discoverySourceId`, `startedAt`, `finishedAt`, `status`, `assetsSeen int`, `errorsJson jsonb`, `rawPayloadRef`	N→1 DiscoverySource (immutable)

Unique keys: `(tenantId, assetId, macAddress)` on NetworkInterface; `(tenantId, assetId)` unique on the 1:1 profile tables.

2.6 License Management

Entity	Purpose	Key columns	Relations
SoftwareLicense	A license title/agreement the tenant owns.	`id`, `tenantId`, `productModelId?`, `name`, `publisher`, `licenseType`, `metric` (per-seat/per-device/per-core/site), `totalSeats int`, `vendorId?`, `contractId?`, `expiresAt?`, `cost numeric(14,2)`	1→N SeatPool/LicenseAssignment/LicenseKey/Renewal
LicenseEntitlement / SeatPool	Purchased capacity buckets (supports true-up).	`id`, `tenantId`, `softwareLicenseId`, `seatsTotal int`, `seatsReserved int`, `seatsAvailable int` *(computed/maintained)*, `effectiveFrom`, `effectiveTo?`, `sourcePOLineId?`	N→1 SoftwareLicense
LicenseAssignment	Consumes a seat by user **or** device.	`id`, `tenantId`, `softwareLicenseId`, `seatPoolId`, `userId?`, `assetId?`, `installedSoftwareId?`, `assignedAt`, `revokedAt?`, `status`	N→1 License/User/Asset
LicenseKey	Actual product keys/tokens (secret).	`id`, `tenantId`, `softwareLicenseId`, `keyCipher bytea` (encrypted), `keyHash`, `maxActivations int`, `activationsUsed int`, `status`	N→1 SoftwareLicense
LicenseRenewal	Renewal/true-up events & reminders.	`id`, `tenantId`, `softwareLicenseId`, `renewalDate`, `seatsDelta int`, `cost`, `status`, `poId?`	N→1 SoftwareLicense

`XOR` constraint: `LicenseAssignment` must have exactly one of `userId` / `assetId` non-null. Reconciliation compares `InstalledSoftware` counts against `SeatPool.seatsTotal` to flag over-deployment.

2.7 Procurement

Entity	Purpose	Key columns	Relations
PurchaseRequest	Internal ask to buy (pre-PO).	`id`, `tenantId`, `requestorId`, `departmentId?`, `justification`, `status`, `budgetId?`, `workflowInstanceId?`, `neededBy?`	1→N PurchaseRequestLine
PurchaseRequestLine	Line of a PR.	`id`, `prId`, `productModelId?`, `description`, `qty int`, `estUnitCost numeric(14,2)`	N→1 PR
Quotation	Vendor quote against a PR.	`id`, `tenantId`, `prId?`, `vendorId`, `quoteNumber`, `validUntil`, `totalAmount`, `currency`, `status`, `attachmentId?`	N→1 PR/Vendor
PurchaseOrder	Approved order to a vendor.	`id`, `tenantId`, `poNumber`, `vendorId`, `quotationId?`, `status`, `orderDate`, `expectedDate?`, `subtotal`, `tax`, `total`, `currency`, `budgetId?`, `contractId?`	1→N POLine/GoodsReceipt
PurchaseOrderLine	Line of a PO; source of provisioned assets.	`id`, `poId`, `productModelId?`, `description`, `qtyOrdered int`, `qtyReceived int`, `unitPrice`, `taxRate`	1→N Asset (via `purchaseOrderLineId`)
GoodsReceipt	Receiving event against a PO.	`id`, `tenantId`, `poId`, `receivedById`, `receivedAt`, `status`, `warehouseId?`	1→N GoodsReceiptLine
GoodsReceiptLine	Qty received per PO line; spawns assets/stock.	`id`, `grId`, `poLineId`, `qtyReceived int`, `binLocationId?`	N→1 GR/POLine
Vendor	Supplier master.	`id`, `tenantId`, `name`, `code`, `taxId`, `contactJson jsonb`, `rating`, `status`	1→N PO/Contract
VendorContract	Agreement (support/framework/lease) with vendor.	`id`, `tenantId`, `vendorId`, `contractNumber`, `type`, `startDate`, `endDate`, `value`, `terms jsonb`, `attachmentId?`	N→1 Vendor
Budget	Spend envelope per dept/period.	`id`, `tenantId`, `name`, `costCenterId?`, `fiscalYear`, `amountAllocated`, `amountCommitted`, `amountSpent`, `currency`	← PR/PO

2.8 Inventory (bulk / consumable stock)

Distinct from `Asset`: fungible quantities, not individually tracked units.

Entity	Purpose	Key columns	Relations
StockItem	A stock-keeping product (cables, toner, RAM sticks).	`id`, `tenantId`, `sku UNIQUE`, `name`, `categoryId?`, `productModelId?`, `uom`, `isConsumable bool`, `reorderPoint int`, `reorderQty int`	1→N StockLevel/Movement
StockLevel	On-hand per StockItem per location.	`id`, `tenantId`, `stockItemId`, `locationId`, `qtyOnHand int`, `qtyReserved int`, `qtyAvailable int` *(= onHand−reserved, maintained)*	N→1 StockItem/Location
StockMovement	Immutable ledger of every in/out/transfer.	`id`, `tenantId`, `stockItemId`, `fromLocationId?`, `toLocationId?`, `qty int`, `movementType`, `refType`, `refId`, `performedById`, `occurredAt`	N→1 StockItem (immutable)
StockCount / Audit	Cycle-count session & variance.	`id`, `tenantId`, `locationId`, `status`, `startedAt`, `completedAt?`, `countedById`	1→N StockCountLine
StockCountLine	Counted vs system qty per item.	`id`, `stockCountId`, `stockItemId`, `systemQty`, `countedQty`, `variance`	N→1 StockCount
Consumable *(subtype of StockItem where `isConsumable`)*	Depleting items issued to users/tickets.	(as StockItem)	← StockMovement (`ISSUE`)

`qtyAvailable` maintained by trigger/service; `StockLevel` unique on `(tenantId, stockItemId, locationId)`.

2.9 Maintenance

Entity	Purpose	Key columns	Relations
MaintenanceTicket	Corrective/preventive work on an asset.	`id`, `tenantId`, `assetId`, `type` (CORRECTIVE/PREVENTIVE), `priority`, `status`, `reportedById`, `assignedToId?`, `openedAt`, `dueAt?`, `closedAt?`, `amcContractId?`, `warrantyClaimId?`	1→N MaintenanceTask
MaintenanceTask	A step/work item within a ticket.	`id`, `ticketId`, `title`, `status`, `assigneeId?`, `laborHours numeric(6,2)`, `checklistId?`	N→1 Ticket
SparePart	Part consumed in maintenance (links stock).	`id`, `tenantId`, `ticketId`, `stockItemId?`, `assetId?`, `qty int`, `cost numeric(14,2)`	N→1 Ticket/StockItem
WarrantyClaim	Claim raised to vendor/manufacturer.	`id`, `tenantId`, `assetId`, `vendorId?`, `claimNumber`, `status`, `filedAt`, `resolvedAt?`, `outcome`	N→1 Asset
AMCContract	Annual maintenance contract coverage.	`id`, `tenantId`, `vendorId`, `contractNumber`, `startDate`, `endDate`, `coverageJson jsonb`, `value`	N→N Asset (via AMCAssetCoverage)
Checklist	Reusable preventive-maintenance/SOP checklist.	`id`, `tenantId`, `name`, `itemsJson jsonb`, `appliesToCategoryId?`	← MaintenanceTask

2.10 Finance

Entity	Purpose	Key columns	Relations
Invoice	Vendor invoice (AP) or internal chargeback.	`id`, `tenantId`, `invoiceNumber`, `vendorId?`, `poId?`, `issueDate`, `dueDate`, `subtotal`, `tax`, `total`, `currency`, `status`, `costCenterId?`	1→N InvoiceLine/Payment
InvoiceLine	Line item; may tie to asset/PO line.	`id`, `invoiceId`, `description`, `qty`, `unitPrice`, `amount`, `assetId?`, `poLineId?`	N→1 Invoice
Payment	Payment against an invoice (immutable).	`id`, `tenantId`, `invoiceId`, `amount`, `paidAt`, `method`, `reference`, `status`	N→1 Invoice (immutable)
Depreciation	Periodic depreciation schedule/entry per asset.	`id`, `tenantId`, `assetId`, `method` (SL/DB/...); `period date`, `openingValue`, `depreciationAmount`, `closingValue`, `postedAt?`	N→1 Asset (immutable ledger)
CostCenter	Financial bucket for allocation & reporting.	`id`, `tenantId`, `code UNIQUE`, `name`, `parentId?`, `ownerId?`	self-ref; ← Budget/Invoice/Asset

2.11 Governance (cross-cutting)

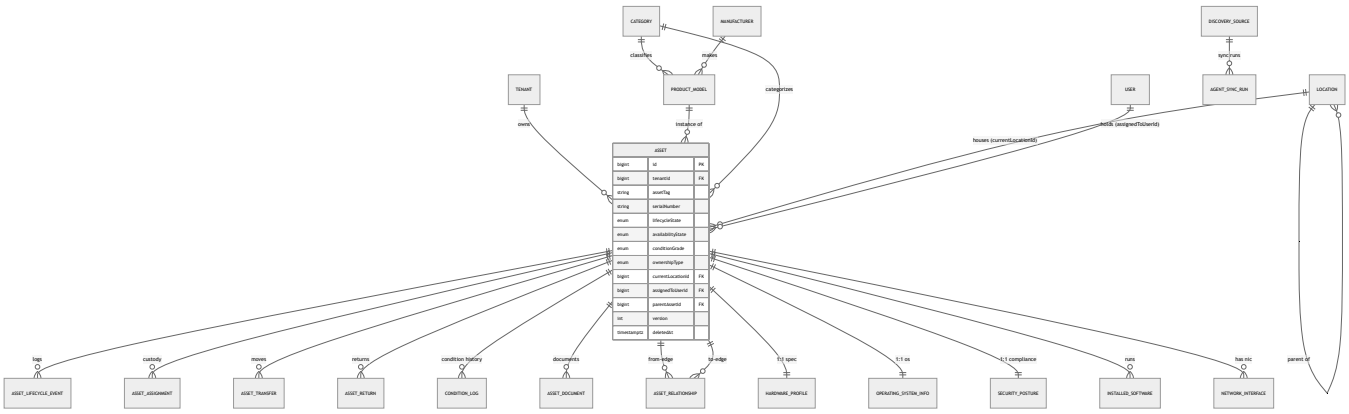
Entity	Purpose	Key columns	Relations
AuditLog	Immutable, tamper-evident change ledger.	`id`, `tenantId`, `actorId?`, `entityType`, `entityId`, `action`, `beforeJson jsonb`, `afterJson jsonb`, `ip`, `occurredAt`, `prevHash`, `rowHash`	(immutable)
Notification	User-facing/event notifications.	`id`, `tenantId`, `userId`, `type`, `title`, `body`, `channel`, `readAt?`, `refType`, `refId`	N→1 User
WorkflowDefinition	Template of an approval/process flow.	`id`, `tenantId`, `name`, `entityType`, `isActive`, `version`	1→N WorkflowStep/Instance
WorkflowStep	Ordered step (approver rule) in a definition.	`id`, `workflowDefinitionId`, `order int`, `name`, `approverRuleJson jsonb`, `slaHours?`	N→1 Definition
WorkflowInstance	A running flow bound to a record.	`id`, `tenantId`, `workflowDefinitionId`, `entityType`, `entityId`, `status`, `currentStepId?`, `startedAt`, `completedAt?`	1→N WorkflowStepInstance
WorkflowStepInstance	Per-step execution + decision.	`id`, `workflowInstanceId`, `stepId`, `assigneeId?`, `decision`, `decidedAt?`, `comment`	N→1 Instance
Attachment	Generic blob metadata (S3/blob store ref).	`id`, `tenantId`, `storageKey`, `fileName`, `mimeType`, `sizeBytes`, `sha256`, `uploadedById`	← many (polymorphic via join tables)
ESignature	Captured signature for acknowledgements.	`id`, `tenantId`, `userId`, `entityType`, `entityId`, `signatureAttachmentId?`, `signedAt`, `ip`, `method`	← Assignment/Return etc.

`AssetRequest` (existing) is retained and now routes through `WorkflowInstance` (its `AssetRequestApproval` rows become `WorkflowStepInstance`s), so approvals are one generic engine rather than request-specific tables.

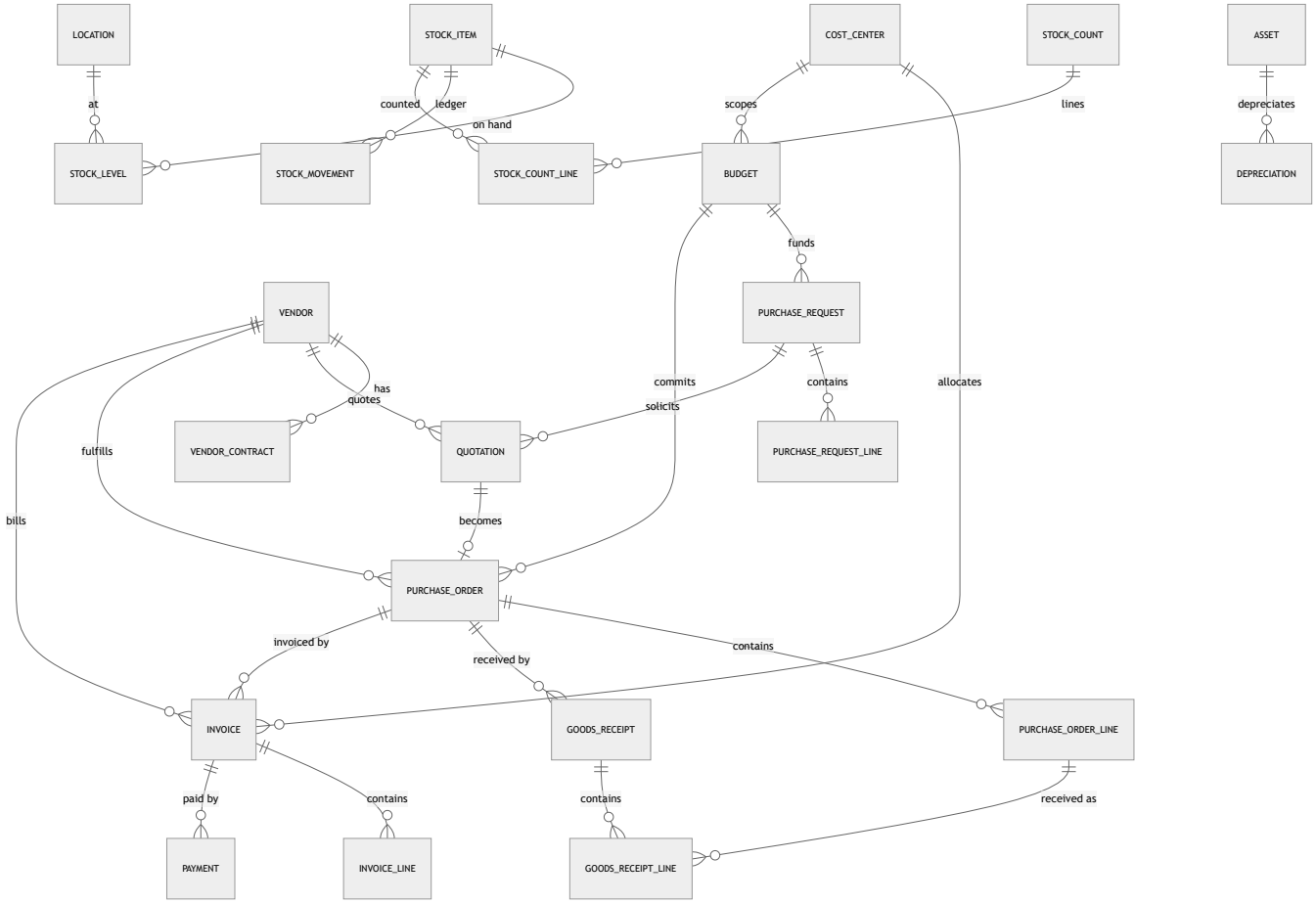
---

3. High-Level ERDs (grouped for readability)

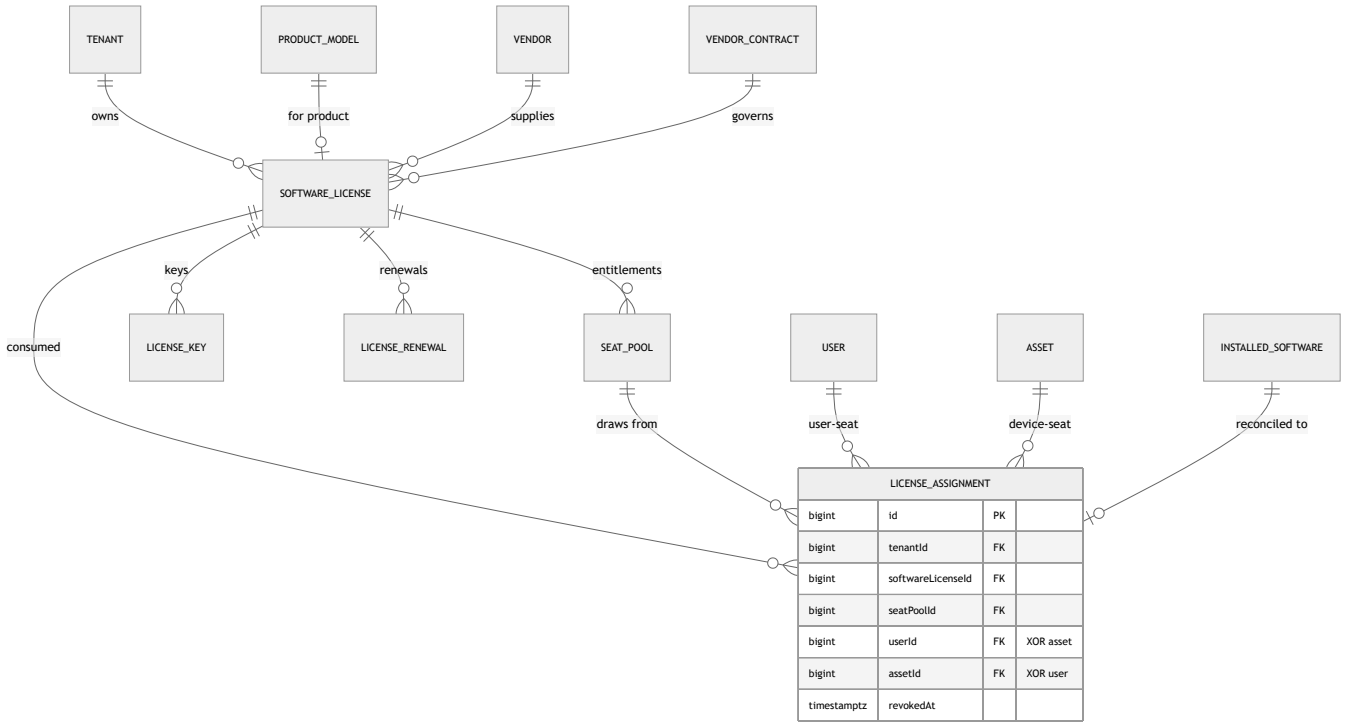
3.1 Assets + Hardware / Discovery



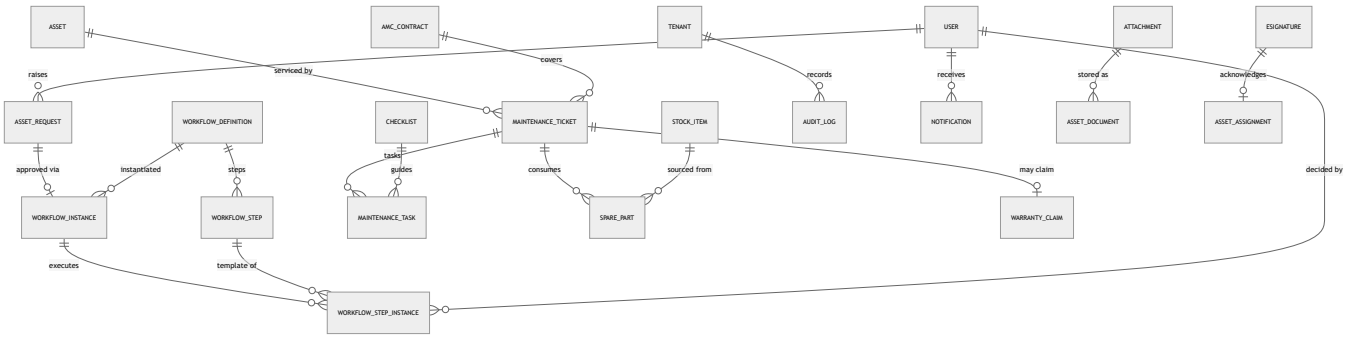
3.2 Procurement + Inventory + Finance



3.3 License Management



3.4 Requests + Maintenance + Governance



---

4. Key Constraints, Integrity & Scale

4.1 Multi-tenancy

- **Every** tenant-scoped table carries `tenantId BIGINT NOT NULL` FK → `Tenant`.
- The Prisma client extension (successor to the `companyId` injector) auto-injects `where: { tenantId }` on reads and `data: { tenantId }` on writes; **no** service code sets it manually.
- **Composite FKs include** `tenantId` so cross-tenant references are structurally impossible, e.g. `Asset.currentLocationId` FK references `Location (tenantId, id)` — a laptop in tenant A can never point at a bin in tenant B.
- **Defense in depth:** enable PostgreSQL **Row-Level Security** with a policy `tenantId = current_setting('app.tenant_id')::bigint`; the connection sets `app.tenant_id` per request. RLS backstops any code path that bypasses the Prisma extension (raw SQL, migrations, analytics).
- All **unique constraints are per-tenant** (see §4.4).

4.2 Soft-delete vs immutable rows

Class	Tables	Rule
Soft-delete	Asset, User, Location, ProductModel, SoftwareLicense, Vendor, Budget, StockItem, Category, WorkflowDefinition… (mutable masters)	`deletedAt timestamptz NULL`; the Prisma extension filters `deletedAt IS NULL` by default. Never physically deleted (preserves FK history).
Immutable / append-only ledgers	AuditLog, AssetLifecycleEvent, ConditionLog, StockMovement, Payment, Depreciation, AgentSyncRun, GoodsReceipt(Line)	**No** `deletedAt`, **no** UPDATE/DELETE. Corrections are new compensating rows. Enforce with `REVOKE UPDATE, DELETE` on the role + a `BEFORE UPDATE/DELETE` trigger that raises.
Tamper-evidence	AuditLog	Hash-chain: `rowHash = sha256(prevHash ‖ canonical(row))`; a gap or mismatch proves tampering.

4.3 Optimistic locking

- Mutable entities carry `version INTEGER NOT NULL DEFAULT 0`.
- Updates use Prisma's `update({ where: { id, version }, data: { …, version: { increment: 1 } } })`; a `count = 0` result ⇒ `409 Conflict` (someone else changed the row). Prevents lost updates on concurrent assignment/transfer/stock operations.
- High-contention counters (`StockLevel.qtyAvailable`, `SeatPool.seatsAvailable`) additionally use **row-level pessimistic locks** (`SELECT … FOR UPDATE`) inside the movement transaction, since versions there would thrash.

4.4 Unique keys (all per-tenant)

Constraint	Definition	Note
Asset tag	`UNIQUE (tenantId, assetTag)`	Human ID; required.
Serial number	partial `UNIQUE (tenantId, serialNumber) WHERE serialNumber IS NOT NULL`	Serial optional (BYOD/legacy) but unique when present.
Invoice number	`UNIQUE (tenantId, invoiceNumber)`	
PO number	`UNIQUE (tenantId, poNumber)`	
License key	`UNIQUE (tenantId, keyHash)`	Store `keyHash` (deterministic) for uniqueness; ciphertext in `keyCipher`.
SKU (stock/model)	`UNIQUE (tenantId, sku)`	
Location code	`UNIQUE (tenantId, parentId, code)`	Unique within a parent.
MAC address	`UNIQUE (tenantId, macAddress)`	Discovery de-dupe.
User email	`UNIQUE (tenantId, email)`	Same person can exist in two tenants.
Permission key	global `UNIQUE (key)`	Permissions are global.
License seat XOR	`CHECK ((userId IS NULL) &lt;&gt; (assetId IS NULL))`	Exactly one consumer.
Asset placement	`CHECK (NOT (currentLocationId IS NOT NULL AND assignedToUserId IS NOT NULL))` *(for current placement)*	Location XOR with-user.

4.5 Indexing guidance for scale

Composite, tenant-first indexes (tenant column always leads so every index is tenant-pruned):

- `Asset`: `(tenantId, lifecycleState, availabilityState)`, `(tenantId, categoryId)`, `(tenantId, assignedToUserId) WHERE assignedToUserId IS NOT NULL`, `(tenantId, currentLocationId)`, `(tenantId, productModelId)`. Partial index `(tenantId, availabilityState) WHERE availabilityState = 'AVAILABLE'` powers

the hot "what can I hand out" query cheaply.

- **Ledgers** (AuditLog, AssetLifecycleEvent, StockMovement, Depreciation, Payment): `(tenantId, entityId/assetId, occurredAt DESC)` for timeline reads. These are the largest tables — **partition by** `RANGE (occurredAt)` **monthly** (or per-tenant hash for very large tenants); prune/archive old partitions.
- `LicenseAssignment`: `(tenantId, softwareLicenseId) WHERE revokedAt IS NULL` (active seats).
- `StockLevel`: unique `(tenantId, stockItemId, locationId)` doubles as the lookup index.
- `Notification`: `(tenantId, userId, readAt) WHERE readAt IS NULL` for unread badges.
- **JSONB** (`Asset.customFields`, `specs`, etc.): `GIN` index only on columns actually queried by key/value; otherwise leave unindexed to save write cost.
- **Covering indexes** via `INCLUDE (assetTag, serialNumber)` on hot list endpoints to enable index-only scans.
- **FK columns are always indexed** (Postgres does not auto-index FKs) — prevents slow cascade checks and lock escalation on parent updates.
- **Text search**: `pg_trgm` GIN index on `Asset.assetTag`, `serialNumber`, `User.displayName` for the global "search box".
- **Maintenance**: schedule `REINDEX` / `ANALYZE` on hot tables; set `fillfactor = 90` on frequently-updated tables (Asset, StockLevel) to keep HOT updates cheap.

Scale posture: soft-delete + partial indexes keep working sets small; append-only ledgers partitioned by time keep the largest tables writable and prunable; tenant-first composite indexes guarantee every plan is tenant-local even at millions of assets across thousands of tenants.

PART 4

License Management — Logic, Seat-Limit Enforcement, APIs + Hardware Data Model

PART A — License Management Module (LMM)

Functional Requirements Document (FRD)

A.1 Purpose

The License Management Module (LMM) is a first-class, multi-tenant subsystem of TechpioAsset that governs the full lifecycle of every software entitlement an organization owns — from procurement request through renewal, reclamation, and retirement. It brings TechpioAsset to parity with ServiceNow SAM Pro, Freshservice, ManageEngine, Lansweeper, and Microsoft Intune/Entra licensing by delivering:

- A **single source of truth** for what is owned, what it cost, what is deployed, and who/what consumes each seat.
- **Hard, transactional seat-limit enforcement** — the module *blocks* over-assignment rather than merely reporting it after the fact.
- **Cost and compliance visibility** — true-up risk, unused-spend recovery, and renewal governance.
- **Automation** — renewal reminders, unused-seat reclamation, duplicate detection, and expiry alerts.

Design Principles

Principle	Implementation
Tenant isolation	Every entity carries `tenant_id`; all queries are tenant-scoped at the repository layer with row-level security.
Seats are a scarce, reserved resource	Assignment is a *reservation* against an atomic counter, never a post-hoc count.
Entitlement ≠ Installation	A license can be assigned to a user, a device, or both; installed software is reconciled against entitlements.
Everything is auditable	Every state change writes an immutable `LicenseAuditEvent`.
Vendor-family aware	License families (M365, Windows, Adobe, VMware, etc.) drive validation, unit-of-assignment, and reconciliation rules.

A.2 Supported License Families

Family	Typical Subscription Types	Unit of Assignment	Reconciliation Signal
Microsoft 365	Subscription (per-user)	User	Entra/Graph `assignedLicenses`, last sign-in
Windows OS	OEM, Volume (KMS/MAK), Subscription (E3/E5)	Device	Activation status, hardware hash
Office (perpetual/volume)	Perpetual, Volume, OpenLicense	Device or User	Installed software inventory
Adobe (CC / Acrobat)	Subscription, Volume (VIP/ETLA)	User	Adobe Admin Console, sign-in
VMware (vSphere/vCenter)	Perpetual, Subscription, OEM	Device / CPU / Core	Host inventory, CPU socket count
Autodesk (AutoCAD, etc.)	Subscription, Named-User	User	Autodesk account, launch telemetry
Visual Studio / MSDN	Subscription (Pro/Enterprise)	User	Sign-in, VS telemetry
Antivirus / EDR	Subscription (per-seat)	Device	Agent check-in
Remote software (TeamViewer, AnyDesk)	Subscription (per-channel/seat)	User or Device	Client heartbeat
Custom software	Any (freeform)	Configurable	Installed software match rule

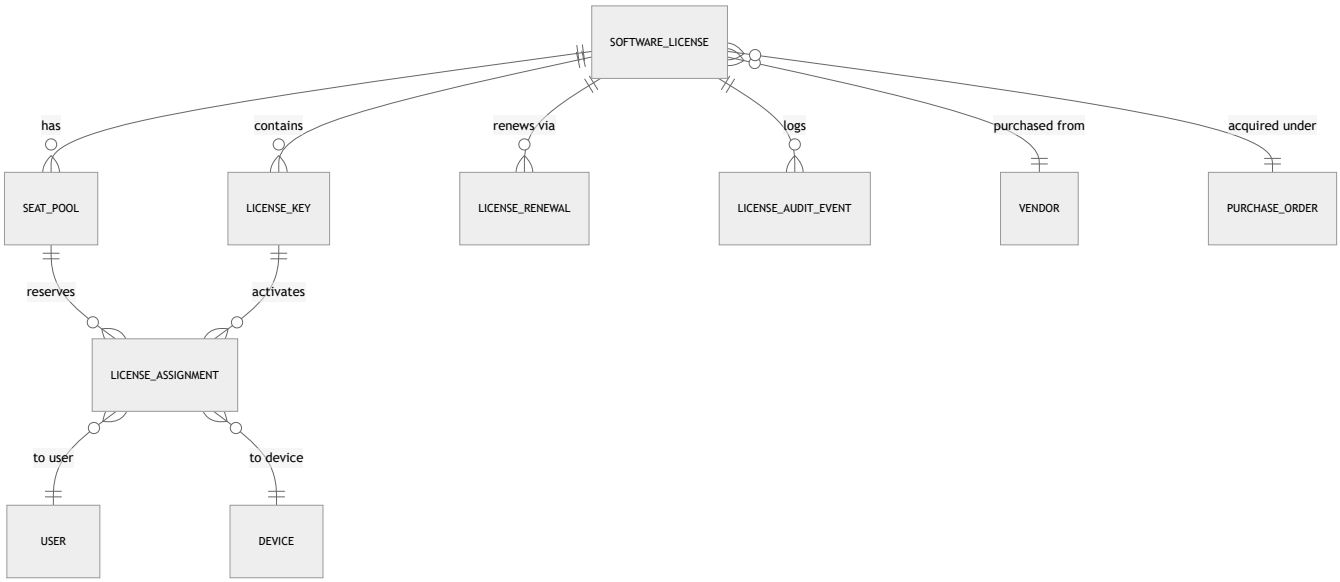
> **Family plug-in model:** each family is a strategy object exposing `unitOfAssignment`, `validationRules`, `reconciliationAdapter`, and `keyFormat`. New families are added without schema change.

A.3 Features (Summary)

1. License catalog with per-family field sets and multiple keys per license.
2. Seat pools / entitlements with derived availability.
3. Assignment to **users and/or devices**, single and **bulk**.
4. **Hard transactional seat-limit enforcement** (the key rule — §A.7).
5. License **transfer** (user→user, device→device, user→device).
6. **Bulk remove / reclaim**.
7. **License history** (full audit timeline per license, seat, user, device).
8. **Expiry & renewal reminders** at 90/60/30/7 days + on-expiry.
9. **Unused-license detection** (assigned but no recent sign-in / device offline).
10. **Duplicate-license detection** (same key, overlapping coverage, over-purchase).
11. Cost, invoice, PO, and vendor linkage.
12. Renewals with auto-renewal flagging and cost roll-forward.
13. Dashboards, reports, notifications, and mobile support.

A.4 Data Model

Entity-Relationship Overview



A.4.1 `SoftwareLicense`

Field	Type	Req	Derived	Notes
`id`	UUID (PK)	Y		
`tenant_id`	UUID (FK)	Y		Tenant isolation
`license_name`	string(200)	Y		e.g. "Microsoft 365 E3"
`family`	enum	Y		See §A.2
`vendor_id`	UUID (FK→Vendor)	Y		
`subscription_type`	enum{perpetual, subscription, oem, volume, openlicense}	Y		Drives validation
`product_edition`	string(120)	N		e.g. "E3", "Enterprise"
`purchase_date`	date	Y		
`expiry_date`	date	N		Null for perpetual
`renewal_date`	date	N		Often = expiry − grace
`auto_renewal`	bool	Y		Default false
`seats_purchased`	int ≥0	Y		Immutable except via renewal/adjustment
`seats_used`	int ≥0	Y	✓	Materialized from active assignments
`seats_available`	int	—	✓	`= seats_purchased − seats_reserved`
`seats_reserved`	int ≥0	Y	✓	Atomic counter (see §A.7)
`unit_of_assignment`	enum{user, device, cpu, core, channel}	Y		From family default, overridable
`cost_amount`	decimal(14,2)	N		
`cost_currency`	char(3)	N		ISO-4217
`cost_model`	enum{per_seat, flat, per_cpu, per_core}	Y		
`invoice_id`	UUID (FK)	N		
`purchase_order_id`	UUID (FK)	N		
`status`	enum	Y		See lifecycle §A.13
`is_metered`	bool	N		For true-up families
`notes`	text	N		
`created_at` / `updated_at`	timestamptz	Y		
`row_version`	int / xmin	Y		Optimistic concurrency token

> **Invariant:** `0 ≤ seats_reserved ≤ seats_purchased` enforced by a DB `CHECK` constraint and an application guard. `seats_available = seats_purchased − seats_reserved` is computed, never stored authoritatively.

A.4.2 `SeatPool` (a.k.a. Entitlement)

A license may split its purchased seats into pools (e.g. by department, region, or cost center) to enable delegated administration and per-pool limits.

Field	Type	Req	Notes
`id`	UUID (PK)	Y	
`tenant_id`	UUID	Y	
`license_id`	UUID (FK)	Y	
`pool_name`	string(120)	Y	Default "Default Pool"
`seats_allocated`	int ≥0	Y	Σ of pool allocations ≤ `seats_purchased`
`seats_reserved`	int ≥0	Y	Atomic per-pool counter
`cost_center`	string(80)	N	
`department_id`	UUID (FK)	N	Delegated admin scope

> Single-pool licenses use one auto-created "Default Pool"; the enforcement logic operates on the pool the assignment targets, then rolls up to the license.

A.4.3 `LicenseAssignment`

Field	Type	Req	Notes
`id`	UUID (PK)	Y	
`tenant_id`	UUID	Y	
`license_id`	UUID (FK)	Y	
`seat_pool_id`	UUID (FK)	Y	
`license_key_id`	UUID (FK)	N	For key-bound families (Windows/Office)
`assignee_type`	enum{user, device}	Y	
`user_id`	UUID (FK)	Cond	Required if `assignee_type=user`
`device_id`	UUID (FK)	Cond	Required if `assignee_type=device`
`status`	enum{active, suspended, revoked, transferred}	Y	Only `active` consumes a seat
`assigned_by`	UUID (FK→User)	Y	
`assigned_at`	timestamptz	Y	
`revoked_at`	timestamptz	N	
`last_usage_at`	timestamptz	N	From reconciliation (sign-in/heartbeat)
`usage_source`	string(40)	N	e.g. "graph.signin", "agent.heartbeat"
`reservation_token`	UUID	Y	Ties to the atomic reservation

> **Unique constraints:** `(license_id, user_id) WHERE status='active'` and `(license_id, device_id) WHERE status='active'` — prevents double-consuming a seat by the same principal.

A.4.4 `LicenseKey`

Field	Type	Req	Notes
`id`	UUID (PK)	Y	
`tenant_id`	UUID	Y	
`license_id`	UUID (FK)	Y	
`key_value_encrypted`	bytea	Y	AES-256, envelope-encrypted; never returned in plaintext via list APIs
`key_fingerprint`	char(64)	Y	SHA-256 of normalized key — for duplicate detection without exposing the key
`key_type`	enum{product_key, mak, kms, activation_id, seat_range}	Y	
`max_activations`	int ≥0	N	For MAK-style keys
`activations_used`	int ≥0	Y	
`is_primary`	bool	Y	
`status`	enum{active, blocked, exhausted}	Y	

A.4.5 `LicenseRenewal`

Field	Type	Req	Notes
`id`	UUID (PK)	Y	
`tenant_id`	UUID	Y	
`license_id`	UUID (FK)	Y	
`renewal_type`	enum{manual, auto}	Y	
`previous_expiry`	date	Y	
`new_expiry`	date	Y	
`seats_before`	int	Y	

Field	Type	Req	Notes
`seats_after`	int	Y	Supports seat true-up on renewal
`cost_amount`	decimal(14,2)	N	
`invoice_id`	UUID (FK)	N	
`purchase_order_id`	UUID (FK)	N	
`status`	enum{planned, quoted, approved, completed, cancelled}	Y	
`renewed_by`	UUID (FK)	N	
`renewed_at`	timestamptz	N	

A.4.6 `LicenseAuditEvent` (immutable)

Field	Type	Notes
`id`	UUID (PK)	
`tenant_id`	UUID	
`license_id`	UUID	
`event_type`	enum{created, assigned, assign_blocked, revoked, transferred, bulk_assigned, bulk_removed, renewed, expired, reclaimed, key_added, seats_adjusted}	
`actor_id`	UUID	User or system principal
`subject_ref`	string	`user:&lt;id&gt;` / `device:&lt;id&gt;`
`before_json` / `after_json`	jsonb	Seat counters, status snapshots
`reason`	text	e.g. "License Limit Exceeded"
`created_at`	timestamptz	Append-only, no update/delete

A.5 Validations

#	Rule	Layer
V1	`seats_used ≤ seats_purchased` at all times (hard).	DB CHECK + service
V2	`Σ seat_pool.seats_allocated ≤ license.seats_purchased` .	Service
V3	Assignment principal type must match `unit_of_assignment` (user vs device).	Service
V4	No duplicate **active** assignment for same (license, principal).	DB unique index
V5	Key-bound families require a `license_key_id` with remaining activations.	Service
V6	`expiry_date ≥ purchase_date` ; `renewal_date ≤ expiry_date` .	Service
V7	Perpetual licenses must have null `expiry_date` ; subscription must have non-null.	Service
V8	Cannot assign against a license in `Expired` / `Retired` status.	Service
V9	Currency required when `cost_amount` present; ISO-4217 validation.	Service
V10	Duplicate key fingerprint across licenses raises a warning (not block) → duplicate detection queue.	Service

---

A.6 Permissions (RBAC)

Role	Capabilities
License Admin (tenant)	Full CRUD, seat adjustment, renewals, transfers, bulk ops, view keys (masked), override warnings.
License Manager (dept/pool)	Assign/revoke within delegated `SeatPool` s; view costs; cannot change `seats_purchased` .
Procurement	Create licenses, POs, invoices, renewals; no assignment.
Auditor / Read-only	View all license data & reports; no mutation; keys always masked.
Help Desk / Technician	Assign/revoke seats only within assigned scope; cannot delete licenses or view raw keys.
End User (self-service portal)	Request a license; view own assignments; no admin.
System / Reconciliation service	Update `last_usage_at` , `activations_used` , trigger reclamation proposals.

> Enforced via policy checks: `can(actor, "license.assign", pool)` etc. Keys are governed by a separate `license.key.reveal` permission with mandatory audit on every reveal.

A.7 SEAT-LIMIT ENFORCEMENT — The Key Rule

Requirement: With 10 M365 seats purchased and 10 assigned, assigning an 11th user **MUST be blocked**. The block is enforced across four coordinated layers.

(a) API-Level Guard — Transactional Seat Reservation

The seat is treated as a scarce resource acquired via an **atomic conditional decrement inside a serializable transaction**, immune to race conditions when two admins assign the last seat simultaneously.

Atomic reservation — pseudo-logic:

```
function assignLicense(licenseId, poolId, principal, actor, tenantId):
    BEGIN TRANSACTION  ISOLATION LEVEL SERIALIZABLE

    # 1. Atomic conditional increment of reserved seats.
    #    The WHERE clause is the guard: it only succeeds if a seat is free.
    rows = EXECUTE:
        UPDATE seat_pool
           SET seats_reserved = seats_reserved + 1
         WHERE id = :poolId
           AND tenant_id = :tenantId
           AND seats_reserved + 1 <= seats_allocated      # <-- HARD LIMIT
        RETURNING seats_reserved, seats_allocated

    # 2. If no row was updated, the pool is full -> BLOCK.
    if rows.affected == 0:
        ROLLBACK
        snapshot = readSeatSnapshot(licenseId)             # purchased/used/available
        writeAudit(event="assign_blocked",
                   reason="License Limit Exceeded",
                   after=snapshot)
        emitDashboardAlert(licenseId, level="critical")
        notify(actor, template="LIMIT_EXCEEDED", snapshot)
        raise SeatLimitExceededError(
            available = 0,
            purchased = snapshot.purchased,
            assigned  = snapshot.used,
            message   = "License Limit Exceeded / Available: 0 / "
                        "Purchased: 10 / Assigned: 10 / "
                        "Please purchase additional licenses.")

    # 3. Verify principal not already actively assigned (idempotency / V4).
    if activeAssignmentExists(licenseId, principal):
        ROLLBACK
        raise DuplicateAssignmentError

    # 4. Create the assignment row bound to this reservation.
    token = newUUID()
    INSERT INTO license_assignment(..., status='active',
                                   reservation_token=token)

    # 5. Roll the reservation up to the license aggregate and
    #    recompute materialized seats_used.
    UPDATE software_license
       SET seats_reserved = (SELECT SUM(seats_reserved) FROM seat_pool
                             WHERE license_id = :licenseId),
           seats_used     = (SELECT COUNT(*) FROM license_assignment
                             WHERE license_id = :licenseId
                               AND status = 'active')
     WHERE id = :licenseId AND tenant_id = :tenantId

    writeAudit(event="assigned", subject=principal, actor=actor)
    COMMIT
    return assignment
```

Race-condition safety: the single `UPDATE ... WHERE seats_reserved + 1 <= seats_allocated` is atomic — the database row lock serializes concurrent writers, so exactly one of two simultaneous "11th seat" requests wins the last free seat and the other's `WHERE` fails (`affected == 0`) and is blocked. `SERIALIZABLE` isolation + the `row_version` token additionally defend against phantom reads on the roll-up. No application-side "read-then-write" gap exists.

Release (revoke/transfer) is the mirror image — a guarded decrement:

```
UPDATE seat_pool
   SET seats_reserved = seats_reserved - 1
 WHERE id = :poolId AND seats_reserved > 0
```

(b) UI: Error + Warning + Dashboard Alert

Trigger	UI Behavior
Assign when `available = 0`	Modal **blocks** submit; red banner: "*License Limit Exceeded / Available: 0 / Purchased: 10 / Assigned: 10 / Please purchase additional licenses.*" Primary CTA becomes **"Request additional licenses."**
`available ≤ 10%` of purchased	Amber warning badge on the license row and assign dialog: "*Only N seats remaining.*"
Any block event	A live **dashboard alert card** ("License Limit Exceeded — M365 E3") appears with count of blocked attempts in last 24h.

(c) Notifications: Email + In-App

- **In-app toast + notification center entry** to the acting admin immediately on block.
- **Email** to License Admins and the license owner: subject "*Seat limit reached — Microsoft 365 E3 (0 of 10 available)*" with a one-click **Buy/Request more** deep link.
- **Digest**: repeated blocks within a window are coalesced to avoid noise.

(d) Audit-Log Entry

Every blocked attempt writes an immutable `LicenseAuditEvent{event_type: "assign_blocked", reason: "License Limit Exceeded", subject_ref: user:<id>, before/after: {purchased:10, used:10, available:0}}`, feeding compliance reports and the "over-demand" analytics used to justify procurement.

A.8 Bulk & Advanced Operations

Bulk Assignment / Bulk Remove

- Input: CSV/multi-select of principals + target license/pool.
- Executed as a **single transaction with per-row seat reservation**; if capacity is insufficient, the system assigns up to the limit and returns a **partial-success report** (`assigned: 7, blocked: 3`) with per-row reasons, or (configurable) all-or-nothing rollback.
- Bulk remove releases seats via guarded decrements and writes one `bulk_removed` audit event with a child manifest.

License Transfer

Type	Behavior
user → user	Revoke source assignment (release seat), create target assignment (re-reserve). Net seat delta = 0; single transaction; `transferred` status recorded.
device → device	Same, key re-binding for key-bound families (deactivate on old, activate on new, respecting `max_activations`).
user → device (cross-unit)	Allowed only if license `unit_of_assignment` permits both; otherwise blocked with validation V3.

Transfers are atomic — never release-then-fail leaving a seat leaked; if the target reservation fails, the source is restored on rollback.

License History

Per-license, per-seat, per-user, and per-device timeline built from `LicenseAuditEvent` , showing assign/revoke/transfer/renew/expire with actor, timestamp, and seat-count deltas. Exportable.

Expired-License Alerts

Daily job flags licenses past `expiry_date` → status `Expired` , suspends dependent assignments (configurable grace), and raises alerts.

Renewal Reminders (90 / 60 / 30 / 7 days)

Scheduler evaluates `renewal_date` (or `expiry_date`) daily; fires reminders at 90, 60, 30, 7 days and on day-0. Auto-renewal licenses generate a *heads-up* ("will auto-renew on <date> at ~<cost>"); manual ones generate an *action-required* task assigned to Procurement.

Unused-License Detection

A license is flagged **unused/reclaimable** when a seat is `active` but: - User family: `last_usage_at` (Graph sign-in) older than *N* days (default 30/60/90), **or** - Device family: device `last_seen` offline > *N* days.

Output = a **reclamation queue** with recovered-cost estimate; admin can bulk-reclaim (release seats) with one click. Feeds "unused spend recovery" report.

Duplicate-License Detection

Flags: identical `key_fingerprint` across licenses; same product+edition purchased multiple times with idle seats (over-purchase); and same principal holding overlapping entitlements. Surfaced as warnings + a consolidation report.

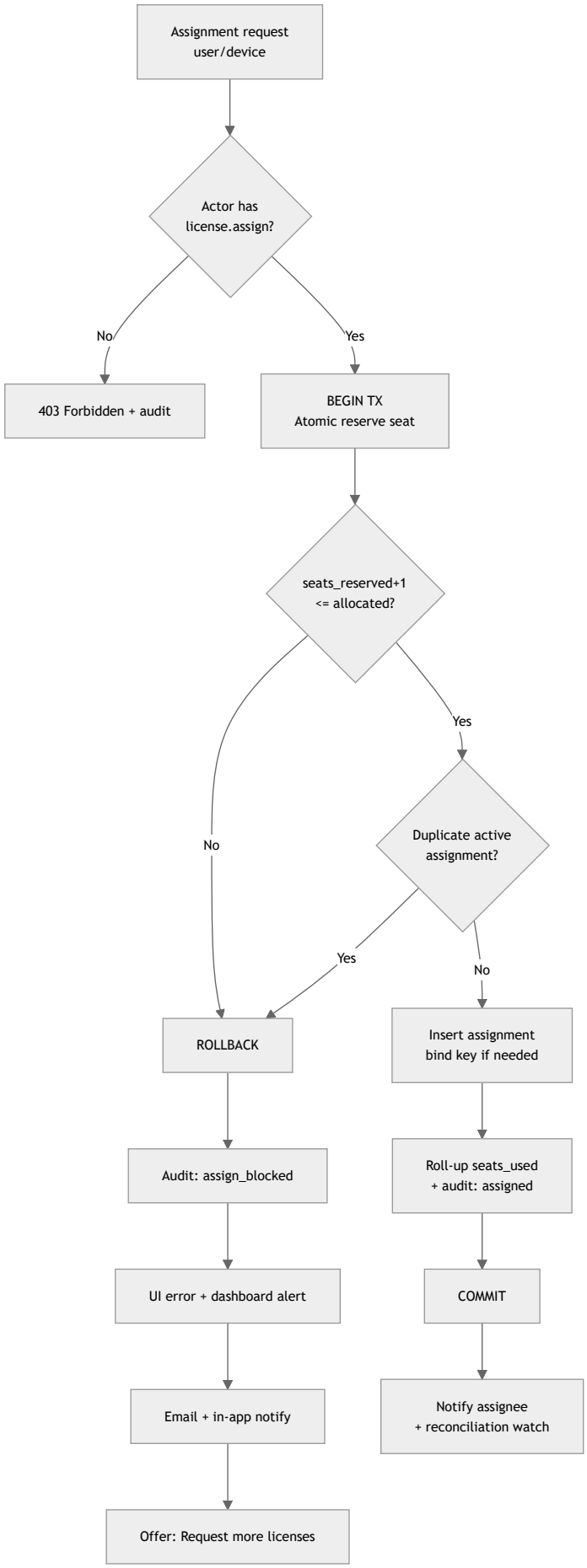
A.9 API — `/api/v1/licenses/...`

Method	Endpoint	Purpose
GET	`/api/v1/licenses`	List/filter (family, vendor, status, expiring). Paginated.
POST	`/api/v1/licenses`	Create license.
GET	`/api/v1/licenses/{id}`	Detail incl. derived `seats_available` .
PATCH	`/api/v1/licenses/{id}`	Update metadata / adjust seats.
DELETE	`/api/v1/licenses/{id}`	Soft-delete / retire.
GET	`/api/v1/licenses/{id}/seats`	Seat summary `{purchased, used, available, reserved}` .
POST	`/api/v1/licenses/{id}/assignments`	**Assign** (transactional guard §A.7). `409 Conflict` on limit.
DELETE	`/api/v1/licenses/{id}/assignments/{assignmentId}`	Revoke (release seat).
POST	`/api/v1/licenses/{id}/assignments:bulk`	Bulk assign; returns per-row result.
POST	`/api/v1/licenses/{id}/assignments:bulkRemove`	Bulk remove.
POST	`/api/v1/licenses/{id}/transfer`	Transfer seat between principals.
GET	`/api/v1/licenses/{id}/keys`	List keys (masked).
POST	`/api/v1/licenses/{id}/keys`	Add key.
POST	`/api/v1/licenses/{id}/keys/{keyId}:reveal`	Reveal key (audited, permissioned).
GET	`/api/v1/licenses/{id}/history`	Audit timeline.
POST	`/api/v1/licenses/{id}/renewals`	Create/plan renewal.
POST	`/api/v1/licenses/{id}/renewals/{rid}:complete`	Complete renewal (roll expiry + seats).
GET	`/api/v1/licenses/reports/expiring?days=90`	Expiring licenses.
GET	`/api/v1/licenses/reports/unused`	Reclamation candidates.
GET	`/api/v1/licenses/reports/duplicates`	Duplicate/over-purchase.
GET	`/api/v1/licenses/reports/compliance`	Over-deployment / true-up risk.

Block response (HTTP 409):

```
{
  "error": "SeatLimitExceeded",
  "message": "License Limit Exceeded / Available: 0 / Purchased: 10 / Assigned: 10 / Please purchase additional licenses.",
  "details": { "purchased": 10, "assigned": 10, "available": 0, "licenseId": "…" },
  "remediation": { "action": "request_additional", "url": "/portal/licenses/…/request" }
}
```

A.10 Workflow


Assignment request
user/device
Actor has
license.assign?
No
Yes
403 Forbidden + audit
BEGIN TX
Atomic reserve seat
seats_reserved+1
<= allocated?
No
Yes
Duplicate active
assignment?
Yes
No
ROLLBACK
Insert assignment
bind key if needed
Audit: assign_blocked
Roll-up seats_used
+ audit: assigned
UI error + dashboard alert
COMMIT
Email + in-app notify
Notify assignee
+ reconciliation watch
Offer: Request more licenses

A.11 Notifications Matrix

Event	In-App	Email	Recipient
Seat limit block	✓	✓	Acting admin, License owner
Low seats (≤10%)	✓	Digest	License Admin
Renewal 90/60/30/7d	✓	✓	Procurement, License owner
Auto-renewal upcoming	✓	✓	License Admin, Finance
License expired	✓	✓	License Admin
Unused seat detected	✓	Digest	License Manager
Duplicate detected	✓	Digest	License Admin
Key reveal	✓ (audit)	—	Security/Auditor

A.12 Reports & Dashboard

Reports

- **License Compliance** (entitled vs deployed; over-deployment/true-up risk).
- **Seat Utilization** (used vs purchased vs available per license/pool/dept).
- **Renewal Forecast** (12-month calendar + projected cost).
- **Unused-Spend Recovery** (reclaimable seats × unit cost).
- **Expiry Report** (expired + expiring buckets).
- **Cost by Vendor / Family / Cost Center.**
- **Duplicate & Over-Purchase.**
- **Audit / Chain-of-custody** per license or principal.

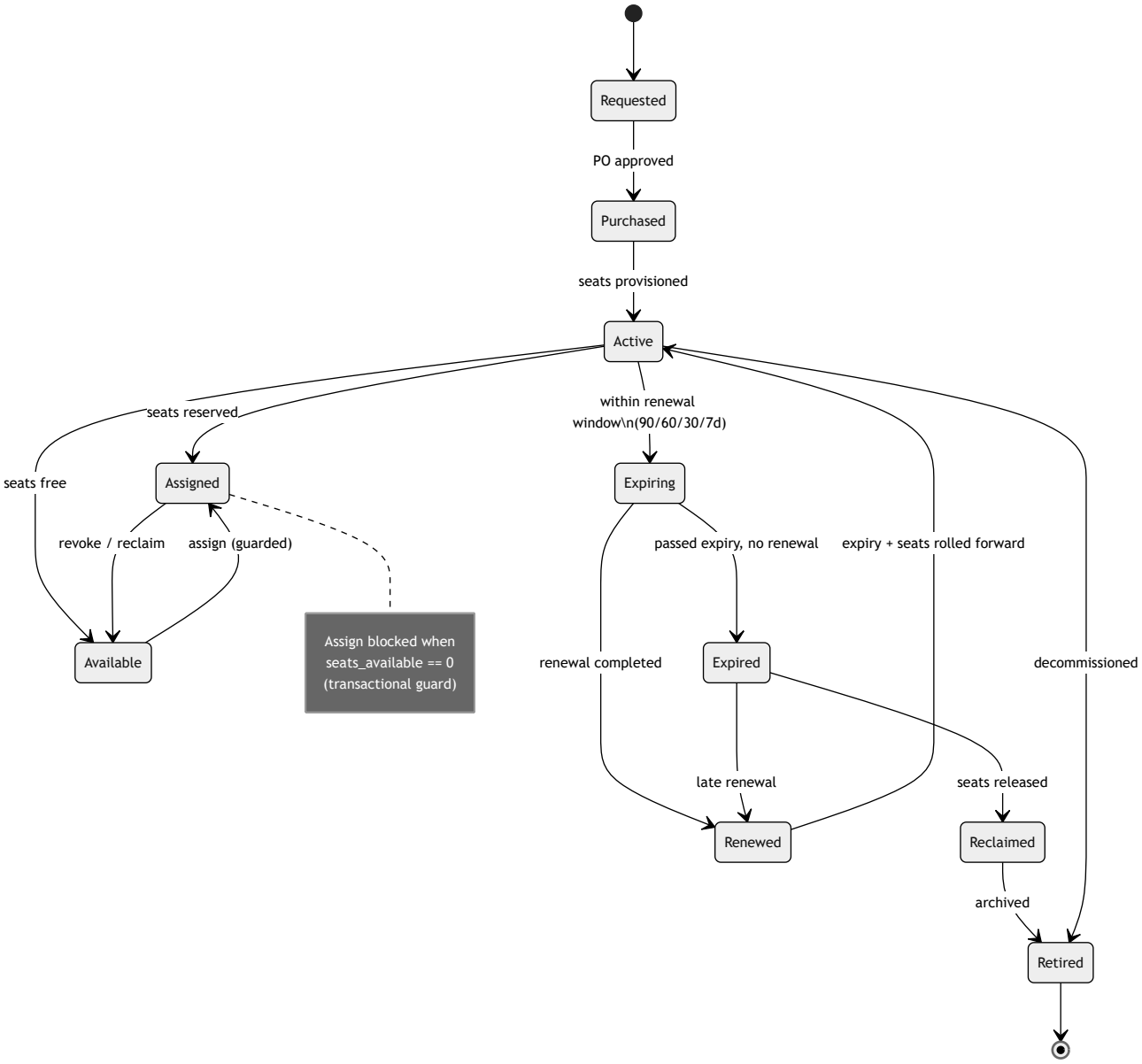
Dashboard Widgets

Widget	Content
Seat Utilization Gauge	Aggregate used/purchased with red zones per license
Expiring Soon	Next 90 days, sorted by date & cost
Seat-Limit Alerts	Live blocked-assignment count (fed by `assign_blocked`)
Reclaimable Seats	Count + $ recoverable
Spend by Family	Donut chart
Renewal Calendar	Timeline of upcoming renewals
Compliance Score	% licenses within entitlement

Mobile Support

- Responsive PWA + native views: view licenses, seat availability, and **assign/revoke on the go**.
- Full seat-limit enforcement applies identically (same API guard) — mobile shows the same block modal.
- Push notifications for renewals, limit blocks, and approvals.
- Offline read cache; mutations require connectivity (reservation must be transactional server-side).

A.13 License Lifecycle (State Diagram)



PART B — Asset Details / Hardware Information Redesign

The redesigned asset-detail page replaces the thin manufacturer/model/serial/OS/CPU/RAM/disk view with a **tabbed, source-attributed profile**. Every field carries a **source badge** (Manual · Agent · Intune · Discovery · Entra/Graph) and a **last-synced** timestamp, so users know provenance and freshness.

B.0 Data Population Model

Source-precedence & merge rules (per field): the reconciliation engine applies a **priority order** and **staleness window** per attribute: - **Authoritative-source-wins**: security/compliance fields (BitLocker, Defender, TPM, activation) prefer **Intune/Agent** over manual; identity/license fields prefer **Entra/Graph**. - **Manual override lock**: procurement fields (asset tag, purchase date, warranty, owner, location) prefer **Manual** unless empty, then discovery fills. - **Freshest-wins** within same tier; a field older than its staleness window is flagged "stale" in the UI. - Every write records `source`, `collected_at`, and `confidence`, enabling the source badges.

Entity mapping: Sections 1, 3 (system + hardware) map to `HardwareProfile`; Section 2 (OS) maps to `OperatingSystemInfo`; Section 2's security fields + parts of Sections 3/5 map to `SecurityPosture`; Section 4 → `InstalledSoftware`; Section 5 → `M365Profile`; Section 6 → `WarrantyContract`; Section 7 → computed `AssetHealth`.

B.1 Tab: System Information → `HardwareProfile`

Field	Source	Notes
Manufacturer	Agent / Discovery / Manual	CIM `Win32_ComputerSystem`
Model	Agent / Discovery	
Serial Number	Agent / Manual	BIOS serial; primary correlation key
Asset Tag	Manual	Procurement/CMDB
Hostname	Agent / Discovery	
Device Name (friendly)	Intune / Manual	
BIOS Version	Agent	`Win32_BIOS`
BIOS Date	Agent	
UEFI Mode	Agent	UEFI vs Legacy BIOS
Secure Boot	Agent / Intune	Enabled/Disabled
TPM Present / Version	Agent / Intune	1.2 / 2.0
Motherboard	Agent	`Win32_BaseBoard`
Chassis Type	Agent	Laptop/Desktop/VM/Tablet
Domain / Workgroup	Agent	
Azure AD / Entra Joined	Intune / Entra	Joined / Hybrid / Registered
Intune Enrolled	Intune	MDM enrollment state
Autopilot Registered	Intune	Autopilot device
Location / Site	Manual / Discovery	Subnet-inferred fallback
Assigned User	Manual / Intune / Entra	Primary user
Department / Office	Manual / Entra	
Purchase Date	Manual	
Warranty (summary)	Manual / Vendor API	See §B.6
Age	Derived	`now − purchase_date`
Lifecycle Stage	Derived / Manual	Ordered/In-stock/Deployed/Repair/Retired
Availability	Derived	In-use / Available / Reserved
Condition	Manual	New/Good/Fair/Damaged
Owner (cost center)	Manual	

B.2 Tab: Operating System → `OperatingSystemInfo` (+ `SecurityPosture`)

Field	Source	Notes
OS Name	Agent / Intune	e.g. Windows 11
Edition	Agent	Pro/Enterprise/Education
Version	Agent	e.g. 24H2
Build Number	Agent	e.g. 26100.x
Architecture	Agent	x64 / ARM64
Install Date	Agent	
Last Boot Time	Agent	Uptime derived
Update Level / Patch Status	Agent / Intune	Latest CU; missing-patch count
Activation Status	Agent	Activated / Not activated
Windows License Key	Agent / Manual	Masked; links to License Module
License Type	Agent	OEM/Retail/Volume/Subscription
License Expiry	Intune / License Module	For subscription (E3/E5)
Windows Defender Status	Agent / Intune	→ `SecurityPosture`
BitLocker Status	Agent / Intune	On/Off per volume → `SecurityPosture`
BitLocker Recovery Key (status)	Intune / Entra	**Escrow present? yes/no**; key itself never shown here (permissioned reveal)

Field	Source	Notes
Firewall Status	Agent	Per-profile
Antivirus Product	Agent	Defender or 3rd-party (from Security Center)
Local Admin Accounts	Agent	Count + list (LAPS-aware)

B.3 Tab: Hardware → `HardwareProfile`

Field	Source	Notes
CPU (model/cores/threads/speed)	Agent / Discovery	`Win32_Processor`
RAM Total	Agent	
Memory Slots (used/total)	Agent	`Win32_PhysicalMemoryArray`
Disk(s) (model/capacity)	Agent	
Disk Usage (free/used)	Agent	Per volume
Disk Health (SMART)	Agent	Healthy/Warning/Failing
SSD vs HDD	Agent	Media type
GPU	Agent / Discovery	
Display (panel/resolution)	Agent	
Battery Health	Agent	Design vs full-charge capacity %
Battery Cycle Count	Agent	
Power Adapter	Agent / Manual	Wattage
Network Adapters	Agent	NIC list
WiFi / Bluetooth	Agent	Present + driver
MAC Address(es)	Agent / Discovery	
IP Address(es)	Agent / Discovery	v4/v6, DHCP/static
Docking Station	Agent / Manual	
USB Devices	Agent	Connected peripherals
Monitors (external)	Agent	EDID model/serial
Printer / Scanner	Agent / Discovery	
Camera / Mic	Agent	Present/enabled

B.4 Tab: Installed Software → `InstalledSoftware`

Field	Source	Notes
Software Name	Agent / Intune	Registry uninstall + MDM inventory
Version	Agent	
Install Date	Agent	
Publisher	Agent	
License Link	Reconciliation	FK → `SoftwareLicense` (Part A); shows entitlement/compliance state

Common apps surfaced with quick-status: Microsoft 365 / Office, Windows OS, Adobe Acrobat/Creative Cloud, browsers (Chrome/Edge/Firefox), Teams/Zoom, antivirus/EDR, remote tools (TeamViewer/AnyDesk), Visual Studio, AutoCAD, 7-Zip/utilities. Each installed entry is **reconciled to an entitlement**: ✅ licensed / ⚠️ unlicensed / 🔵 unmanaged.

B.5 Tab: Microsoft 365 Information → `M365Profile` (via Entra/Graph)

Field	Source	Notes
Tenant	Entra	Tenant name/ID
Assigned M365 License	Graph	`assignedLicenses` → links to Part A
Exchange Online	Graph	Enabled + plan
OneDrive	Graph	Provisioned + storage
SharePoint	Graph	Access/sites
Teams	Graph	Enabled
Defender for O365	Graph / Intune	Plan level
Conditional Access	Graph	Applied policies
MFA Status	Graph	Enabled/enforced/registered methods
Mailbox Size	Graph	Used/quota
Last Sign-In	Graph signIns	Feeds unused-license detection (Part A §A.8)
User Status	Entra	Active / Disabled / Blocked

B.6 Tab: Warranty & Contracts → `WarrantyContract`

Field	Source	Notes
Vendor	Manual / Vendor API	Dell/HP/Lenovo warranty lookup
Purchase Date	Manual	
Warranty Start	Manual / Vendor API	
Warranty End	Manual / Vendor API	
Remaining Days	Derived	Drives health + alerts
AMC (Annual Maintenance Contract)	Manual	Provider, coverage
Support Contact	Manual	Phone/email/portal
Invoice	Manual	Linked document
Contract Documents	Manual	Uploaded attachments

B.7 Tab: Asset Health → computed `AssetHealth`

An **overall health score (0–100)** rolled up from weighted sub-scores. Each sub-score is 0–100; the overall is a weighted average, and any sub-score in a critical band caps the overall grade.

Sub-score model

Sub-score	Weight	Inputs	Scoring logic (examples)
Battery	15%	Health %, cycle count	≥80% & <500 cycles = 100; 60–79% = 70; <50% or >1000 cycles = 30
Storage	20%	SMART status, free space %	SMART healthy & >20% free = 100; <10% free = 50; SMART warning = 40; failing = 0
Memory	10%	RAM vs role baseline, slots free	Meets baseline = 100; below baseline = 50
Warranty	15%	Remaining days	>180d = 100; 90–180 = 75; 30–89 = 50; <30 = 25; expired = 0
Security	25%	BitLocker, Defender, firewall, TPM, activation, local-admin count	All enabled = 100; each disabled control −20; unencrypted disk caps at 40
Updates	15%	Missing patches, OS support status	Fully patched = 100; each missing critical −10; unsupported OS caps at 30

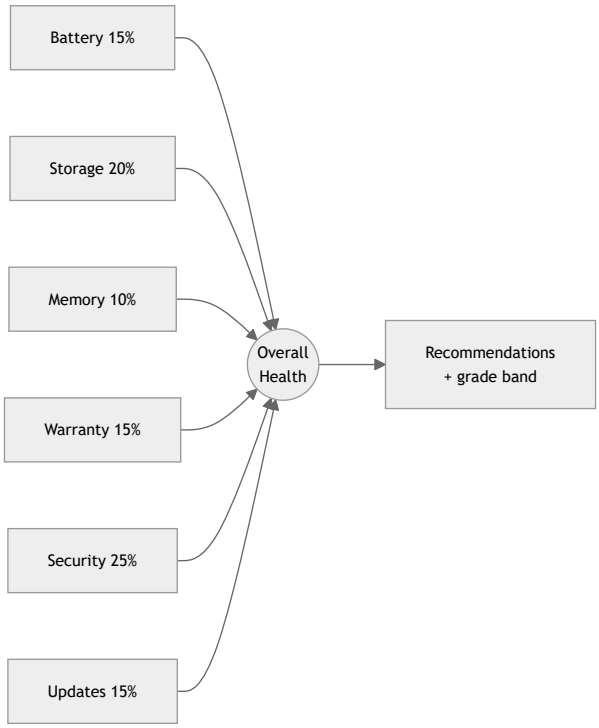
Overall grade bands: 90–100 Excellent · 75–89 Good · 60–74 Fair · 40–59 Poor · <40 Critical. **Capping rule:** if Security or Storage < 40 (e.g. unencrypted disk or failing SMART), overall grade cannot exceed "Poor" regardless of weighted average — safety-critical dimensions dominate.

```
overall = round( Σ(subScore_i × weight_i) )
if security < 40 or storage < 40:
    overall = min(overall, 59)   # cannot exceed "Poor"
```

Recommendations engine

Generates prioritized, actionable items from the sub-scores, e.g.: - Storage < 50 & free < 10% → "*Disk nearly full and SMART warning — schedule replacement / free space.*" - Battery < 50 → "*Battery degraded (health 42%, 1,120 cycles) — replace battery.*" - Security: BitLocker off → "*Enable BitLocker; drive currently unencrypted (compliance risk).*" - Warranty < 30d → "*Warranty expires in 21 days — renew AMC or plan refresh.*" - Updates: unsupported OS → "*Windows build past end-of-support — schedule upgrade.*"

Health widget



B.8 Entity Mapping Summary

Tab / Section	Primary Entity	Secondary
1 System Information	`HardwareProfile`	—
2 Operating System	`OperatingSystemInfo`	`SecurityPosture`
3 Hardware	`HardwareProfile`	—
4 Installed Software	`InstalledSoftware`	`SoftwareLicense` (Part A)
5 Microsoft 365	`M365Profile`	`SoftwareLicense`
6 Warranty & Contracts	`WarrantyContract`	—
7 Asset Health	`AssetHealth` (computed)	HardwareProfile · OperatingSystemInfo · SecurityPosture · WarrantyContract

Each entity stores per-field `source`, `collected_at`, and `confidence` so the redesigned UI can render **source badges, freshness indicators, and "stale data" warnings** consistently across all seven tabs — the hallmark that lifts TechpioAsset's asset-detail experience to ServiceNow/Intune class.

PART 5

Procurement & Inventory — Entities, Validations & APIs

MODULE 1 — PROCUREMENT

1.1 Purpose

Provide a closed-loop, source-to-pay procurement engine that turns an approved asset request (or a standalone need) into a Purchase Request, competitively sources it through multi-vendor RFQ/quotation, converts the winning quote into a budget-checked and threshold-approved Purchase Order, tracks the shipment, receives goods into inventory, enforces a 3-way match (PO ↔ GRN ↔ Invoice) before payment, and closes the record — all with per-tenant cost-center/budget governance and a full audit trail. It is the upstream feeder of the Inventory module: every accepted Goods Receipt line increments stock.

1.2 Features

- **Purchase Requests (PR):** created from an approved Asset Request (auto-populated) or standalone; multi-line; needed-by dates; cost-center + budget tagging; attachments; requester justification.
- **RFQ / Quotation capture:** issue an RFQ to N vendors from a PR; capture quotes manually or via vendor portal / email intake; per-line pricing, lead time, warranty, validity.
- **Quote comparison:** side-by-side matrix across vendors; weighted scoring (price, lead time, rating, compliance, warranty); auto-recommendation + manual override with reason.
- **Purchase Orders (PO):** generated from winning quote or directly; header + lines; automatic budget availability check; multi-level approval routed by monetary threshold; PDF generation; revisions with version history.
- **Vendor Management:** vendor profiles, multiple contacts, addresses, tax/bank details, contracts (with expiry + renewal alerts), category tags, performance ratings (OTIF, quality, price competitiveness), blacklist/preferred flags.
- **Goods Receipt (GRN):** partial or full receipt against PO lines; over/under-receipt tolerance; QC accept/reject/quarantine; serial/asset-tag capture; feeds Inventory.
- **Invoice matching (3-way):** match invoice ↔ PO ↔ GRN on quantity and price within tolerance; flag/hold exceptions; supports 2-way fallback for service lines.
- **Payment status tracking:** scheduled → approved → paid → reconciled; partial payments; ties to existing invoice module and finance export.
- **Delivery / shipment tracking:** carrier, tracking number, ETA, milestones, exceptions; optional carrier-webhook ingestion.
- **Budget & cost-center allocation:** budgets per cost-center/period/GL; commitment accounting (PR = soft commit, PO = hard commit, GRN/Invoice = actual); overspend blocks or escalates.
- **Procurement reports:** spend analytics, cycle time, vendor scorecards, budget vs actual, open PO/GRN aging, 3-way match exceptions.

1.3 Key Fields

1.3.1 Purchase Request (`purchase_requests` / `purchase_request_lines`)

Field	Type	Required	Validation	Notes
id	UUID	Y	system	PK
tenant_id	UUID	Y	FK tenants	RLS isolation
pr_number	string	Y	unique per tenant, pattern `PR-{YYYY}-{seq}`	auto-generated
title	string(160)	Y	3–160 chars	short need description
source_type	enum	Y	`asset_request` | `standalone`	drives auto-fill
asset_request_id	UUID	N	FK, required when source_type=asset_request	link to approved request
requester_id	UUID	Y	FK users	defaults to current user
department_id	UUID	N	FK	reporting
cost_center_id	UUID	Y	FK, must be active	budget anchor
needed_by	date	Y	≥ today	drives priority/SLA
priority	enum	Y	`low|medium|high|critical`	default medium
estimated_total	money	N	≥ 0	soft commitment
currency_code	char(3)	Y	ISO-4217	
status	enum	Y	see workflow	default `draft`
justification	text	N	≤ 4000	required if over policy threshold
attachments	file[]	N	≤ 25MB each, allow-listed MIME	quotes, specs
line: item_name	string	Y	2–200	free text or catalog ref
line: catalog_item_id	UUID	N	FK product catalog	links to stock item template
line: category_id	UUID	Y	FK category	
line: quantity	decimal(12,3)	Y	> 0	fractional allowed (consumables)
line: uom	string	Y	FK unit-of-measure	ea, box, m, L
line: estimated_unit_price	money	N	≥ 0	
line: gl_account_id	UUID	N	FK	posting hint

1.3.2 RFQ & Quotation (`rfqs` , `quotations` , `quotation_lines`)

Field	Type	Required	Validation	Notes
rfq.id / rfq.rfq_number	UUID / string	Y	unique `RFQ-{YYYY}-{seq}`	
rfq.purchase_request_id	UUID	Y	FK	source PR
rfq.vendor_ids	UUID[]	Y	≥ 1 active vendor	invited vendors
rfq.due_date	timestamptz	Y	> now	quote submission deadline
rfq.status	enum	Y	`draft|sent|receiving|closed|cancelled`	
quotation.id / quote_number	UUID / string	Y	unique per tenant	
quotation.rfq_id	UUID	Y	FK	
quotation.vendor_id	UUID	Y	FK	one quote per vendor per RFQ (revisable)
quotation.valid_until	date	Y	≥ today	expiry drives comparison eligibility
quotation.lead_time_days	int	Y	≥ 0	scoring input
quotation.total_amount	money	Y	= Σ lines	
quotation.warranty_months	int	N	≥ 0	scoring input
quotation.score	decimal(5,2)	N	0–100, system-computed	weighted
quotation.is_selected	bool	Y	one selected per RFQ	award flag
quotation.selection_reason	text	N	required if not top-scored	override justification
qline.item_name / quantity / unit_price / uom	mixed	Y	qty>0, price≥0	maps to PR lines

1.3.3 Purchase Order (`purchase_orders` , `purchase_order_lines`)

Field	Type	Required	Validation	Notes
po_number	string	Y	unique `PO-{YYYY}-{seq}`	
purchase_request_id	UUID	N	FK	traceability
quotation_id	UUID	N	FK	winning quote
vendor_id	UUID	Y	FK, not blacklisted	
cost_center_id	UUID	Y	FK active	budget check anchor
budget_id	UUID	Y	FK, sufficient available	hard commitment
order_date	date	Y	≤ today	
expected_delivery	date	Y	≥ order_date	
ship_to_location_id	UUID	Y	FK inventory location	receiving warehouse
billing_address_id	UUID	Y	FK	
subtotal / tax / shipping / discount / total	money	Y	total = subtotal+tax+shipping−discount	
currency_code / fx_rate	char(3)/decimal	Y	ISO-4217, fx>0	
status	enum	Y	see workflow	default `draft`
approval_state	enum	Y	`not_required|pending|approved|rejected`	threshold-driven
payment_terms	string	Y	e.g. Net-30	from vendor default
revision_no	int	Y	≥ 0	version history
poline: quantity_ordered	decimal(12,3)	Y	> 0	
poline: quantity_received	decimal(12,3)	Y	0 ≤ recv ≤ ordered×(1+tol)	maintained by GRN
poline: quantity_invoiced	decimal(12,3)	Y	≤ received (match)	maintained by invoice
poline: unit_price	money	Y	≥ 0	
poline: tax_rate	decimal(5,2)	N	0–100	

1.3.4 Vendor (`vendors` , `vendor_contacts` , `vendor_contracts` , `vendor_ratings`)

Field	Type	Required	Validation	Notes
vendor.name / legal_name	string	Y	unique legal_name per tenant	
vendor.code	string	Y	unique per tenant	
vendor.status	enum	Y	`active|inactive|blacklisted|onboarding`	blacklisted blocks POs
vendor.tax_id	string	N	tax-format validation by country	
vendor.categories	UUID[]	N	FK	sourcing filter
vendor.payment_terms_default	string	N		
vendor.bank_details	encrypted json	N	encrypted at rest	PCI-ish handling
vendor.preferred	bool	N		ranking boost
vendor.overall_rating	decimal(3,2)	N	0–5, system rollup	
contact.name/email/phone/role	mixed	Y	valid email/phone	≥1 primary contact
contract.contract_number	string	Y	unique	

Field	Type	Required	Validation	Notes
contract.start_date/end_date	date	Y	end > start	
contract.value	money	N	≥ 0	
contract.renewal_alert_days	int	N	≥ 0	drives expiry notification
contract.document	file	N	PDF, ≤ 25MB	
rating.otif_score / quality_score / price_score	decimal	N	0–5	per-PO captured, rolled up

1.3.5 Goods Receipt (`goods_receipts` , `grn_lines`)

Field	Type	Required	Validation	Notes
grn_number	string	Y	unique `GRN-{YYYY}-{seq}`	
purchase_order_id	UUID	Y	FK, PO status = acknowledged/shipped	
received_by	UUID	Y	FK users	
received_date	timestamptz	Y	≤ now	
warehouse_id	UUID	Y	FK, = PO ship_to	
receipt_type	enum	Y	`partial|full`	
status	enum	Y	`draft|inspecting|accepted|rejected|posted`	posted → inventory
grnline: po_line_id	UUID	Y	FK	
grnline: quantity_received	decimal(12,3)	Y	≤ remaining × (1+over_tol)	tolerance policy
grnline: quantity_accepted	decimal(12,3)	Y	≤ received	
grnline: quantity_rejected	decimal(12,3)	Y	received − accepted	routes to quarantine
grnline: qc_status	enum	Y	`pass|fail|partial`	
grnline: serials	string[]	N	count = accepted qty for serialized items	asset tag seeding
grnline: bin_location_id	UUID	Y	FK bin	put-away target

1.3.6 Invoice Match & Payment (`vendor_invoices` , `invoice_match_results` , `payments`)

(extends existing invoice module)

Field	Type	Required	Validation	Notes
invoice.invoice_number	string	Y	unique per vendor	external number
invoice.purchase_order_id	UUID	Y	FK	
invoice.grn_ids	UUID[]	N	FK	matched receipts
invoice.amount / tax / total	money	Y	≥ 0	
invoice.match_status	enum	Y	`unmatched|matched|exception|overridden`	
match.type	enum	Y	`three_way|two_way`	2-way for services
match.qty_variance / price_variance	decimal	Y	within tolerance → pass	
match.tolerance_pct	decimal	Y	tenant policy	
payment.status	enum	Y	`scheduled|approved|paid|partial|failed|reconciled`	
payment.method	enum	Y	`bank|card|check|wire|other`	
payment.paid_amount / paid_date	money/date	N	≤ invoice.total	partials allowed

1.3.7 Shipment (`shipments`)

Field	Type	Required	Validation	Notes
purchase_order_id	UUID	Y	FK	
carrier	string	N		
tracking_number	string	N		
status	enum	Y	`pending|in_transit|out_for_delivery|delivered|exception`	drives in-transit stock
eta	date	N	≥ order_date	
milestones	json[]	N		carrier webhook events

1.3.8 Budget (`budgets` , `budget_allocations` , `commitments`)

Field	Type	Required	Validation	Notes
budget.cost_center_id	UUID	Y	FK	
budget.period	string	Y	e.g. `2026-Q3` / fiscal-year	
budget.gl_account_id	UUID	N	FK	
budget.amount_allocated	money	Y	≥ 0	
budget.amount_committed	money	Y	system-maintained	PR soft + PO hard
budget.amount_actual	money	Y	system-maintained	GRN/invoice actuals

Field	Type	Required	Validation	Notes
budget.amount_available	money	Y	= allocated − committed − actual	check target
commitment.type	enum	Y	`soft|hard|actual`	

1.4 Validations & Business Rules

- **PR→PO traceability:** a PO created from a quotation must inherit PR + quotation IDs; a standalone PO requires elevated permission and mandatory justification.
- **Budget check (hard gate):** on PO submit-for-approval, `amount_available ≥ PO.total` at the target budget; otherwise block with overspend reason and route to budget-owner escalation. Approving a PO converts soft commitment to hard commitment.
- **Threshold approval matrix (tenant-configurable):** e.g. `< 1,000` auto-approve; `1,000–10,000` manager; `10,000–50,000` department head; `> 50,000` CFO + procurement lead (parallel). Approval level recalculated on any PO revision that increases total.
- **Quote comparison eligibility:** only quotes with `valid_until ≥ today` and status `submitted` enter scoring. Awarding a non-top-scored quote requires `selection_reason`.
- **Scoring formula (weights tenant-configurable, default):** `score = 0.40·price_norm + 0.20·lead_time_norm + 0.20·vendor_rating_norm + 0.10·warranty_norm + 0.10·compliance_norm`, each normalized 0–100; lowest price and shortest lead time score highest.
- **Vendor guardrails:** blacklisted/inactive vendors cannot receive RFQs or POs; contract-required categories block PO issue if no active contract.
- **GRN tolerance:** over-receipt beyond `over_receipt_tolerance_pct` blocked; under-receipt allowed and leaves PO line open. Rejected quantity must be routed to quarantine, never to available stock.
- **3-way match:** invoice passes only if `|qty_variance| ≤ qty_tolerance` AND `|price_variance| ≤ price_tolerance` for every matched line against PO price and GRN accepted qty; else `exception` and payment blocked until resolved or overridden (override requires finance role + reason).
- **Payment guard:** no payment can be `approved` while `match_status ∈ {unmatched, exception}`. Sum of payments ≤ invoice total.
- **PO closure:** PO auto-closes when all lines fully received, fully invoiced, and fully paid; manual short-close permitted with reason and reversal of residual commitment.
- **Currency:** cross-currency PO vs invoice compared at captured `fx_rate`; FX variance beyond tolerance is a match exception.
- **Immutability:** approved POs are versioned; edits create a new `revision_no` and re-enter approval if total increases.

1.5 Permissions (roles)

Capability	Requester	Procurement Officer	Procurement Manager	Budget/Finance	Receiver/Warehouse	Vendor (portal)	Tenant Admin
Create/submit PR	✓	✓	✓	–	–	–	✓
Approve PR	–	–	✓	(budget)	–	–	✓
Issue RFQ / capture quotes	–	✓	✓	–	–	submit own quote	✓
Run comparison / award	–	✓	✓	–	–	–	✓
Create PO	–	✓	✓	–	–	–	✓
Approve PO (by threshold)	–	–	✓	✓	–	–	✓
Manage vendors/contracts	–	✓	✓	–	–	edit own profile	✓
Post Goods Receipt	–	–	–	–	✓	–	✓
3-way match / override	–	✓ (view)	✓	✓ (override)	–	–	✓
Record/approve payment	–	–	–	✓	–	–	✓
Configure budgets/thresholds	–	–	–	✓	–	–	✓
Procurement reports	scoped to own	✓	✓	✓	scoped	own only	✓

All permissions are tenant-scoped; vendor portal users are external identities restricted to their own vendor records and RFQs they were invited to.

1.6 API Endpoints (REST, `/api/v1/...`)

Purchase Requests - `GET /api/v1/procurement/purchase-requests` - `POST /api/v1/procurement/purchase-requests` - `GET|PATCH|DELETE /api/v1/procurement/purchase-requests/{id}` - `POST /api/v1/procurement/purchase-requests/{id}/submit` - `POST /api/v1/procurement/purchase-requests/{id}/approve` - `POST /api/v1/procurement/purchase-requests/{id}/reject` - `POST /api/v1/procurement/purchase-requests/from-asset-request/{assetRequestId}`

RFQ / Quotations - `POST /api/v1/procurement/rfqs` (from PR) · `POST /rfqs/{id}/send` - `GET /api/v1/procurement/rfqs/{id}/quotations` - `POST /api/v1/procurement/rfqs/{id}/quotations` (capture / vendor submit) - `GET /api/v1/procurement/rfqs/{id}/comparison` (scored matrix) - `POST /api/v1/procurement/rfqs/{id}/award` `{quotation_id, reason?}`

Purchase Orders - `GET|POST /api/v1/procurement/purchase-orders` - `GET|PATCH /api/v1/procurement/purchase-orders/{id}` - `POST /api/v1/procurement/purchase-orders/{id}/budget-check` - `POST /api/v1/procurement/purchase-orders/{id}/submit` - `POST /api/v1/procurement/purchase-orders/{id}/approve|reject` - `POST /api/v1/procurement/purchase-orders/{id}/acknowledge` (vendor) - `POST /api/v1/procurement/purchase-orders/{id}/revise` - `GET /api/v1/procurement/purchase-orders/{id}/pdf`

Vendors - `GET|POST /api/v1/procurement/vendors` · `GET|PATCH /vendors/{id}` - `POST /api/v1/procurement/vendors/{id}/contacts` - `GET|POST /api/v1/procurement/vendors/{id}/contracts` - `GET|POST /api/v1/procurement/vendors/{id}/ratings` - `POST /api/v1/procurement/vendors/{id}/blacklist`

Goods Receipt - `GET|POST /api/v1/procurement/goods-receipts` - `GET /api/v1/procurement/purchase-orders/{id}/receipts` - `POST /api/v1/procurement/goods-receipts/{id}/inspect` - `POST /api/v1/procurement/goods-receipts/{id}/post` → emits `inventory.stock.received` event

Invoice match & payment - `POST /api/v1/procurement/invoices/{id}/match` (3-way) - `GET /api/v1/procurement/invoices/{id}/match-result` - `POST /api/v1/procurement/invoices/{id}/match/override` - `POST /api/v1/procurement/invoices/{id}/payments` - `GET /api/v1/procurement/payments/{id}`

Shipments & budgets - `GET|POST /api/v1/procurement/purchase-orders/{id}/shipments` - `POST /api/v1/procurement/shipments/{id}/events` (carrier webhook) - `GET|POST /api/v1/procurement/budgets` · `GET /budgets/{id}/utilization`

Reports - `GET /api/v1/procurement/reports/{spend|cycle-time|vendor-scorecard|budget-vs-actual|open-po-aging|match-exceptions}`

1.7 Workflow

1. **PR raised** (from approved asset request or standalone) → **PR approval** (policy/threshold).
2. **RFQ issued** to selected vendors → **quotations received** → **comparison & scoring** → **award**.
3. **PO created** from winning quote → **budget check** → **threshold approval** → **PO issued** to vendor.
4. **Vendor acknowledges** → **shipment** (in-transit stock created) → **Goods Receipt** (partial/full, QC) → **posted to Inventory**.
5. **Vendor invoice** received → **3-way match** → pass → **payment scheduled/approved/paid** → **reconciled**.
6. **PO closed** when fully received, invoiced, and paid (or short-closed with reason).

1.8 Notifications

- PR submitted/approved/rejected → requester + approver.
- RFQ sent → invited vendors; RFQ closing soon → non-responding vendors.
- Quote received → procurement officer; award decision → all bidders.
- PO pending approval → approver (with SLA reminder escalation); PO issued/acknowledged → buyer + vendor.
- Budget overspend attempt → budget owner.
- Shipment milestones/exceptions → buyer + receiver.
- Goods receipt posted / partial / rejection-to-quarantine → buyer + inventory manager.
- 3-way match exception → procurement + finance.
- Payment scheduled/paid → finance + vendor.
- Contract expiring within `renewal_alert_days` → vendor manager.
- Channels: in-app, email, and webhook/Slack-Teams per tenant preference; digest option for high-volume roles.

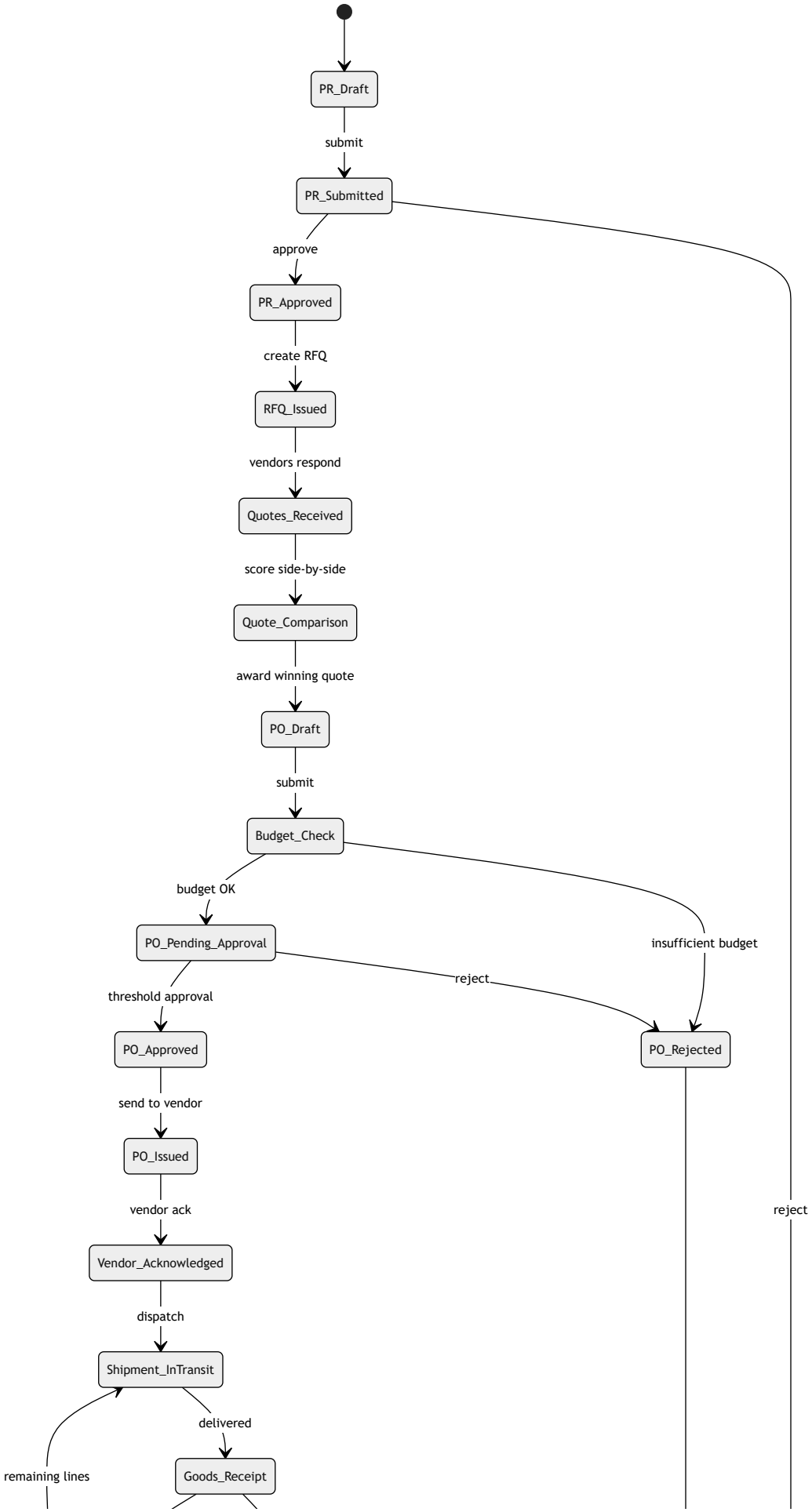
1.9 Reports

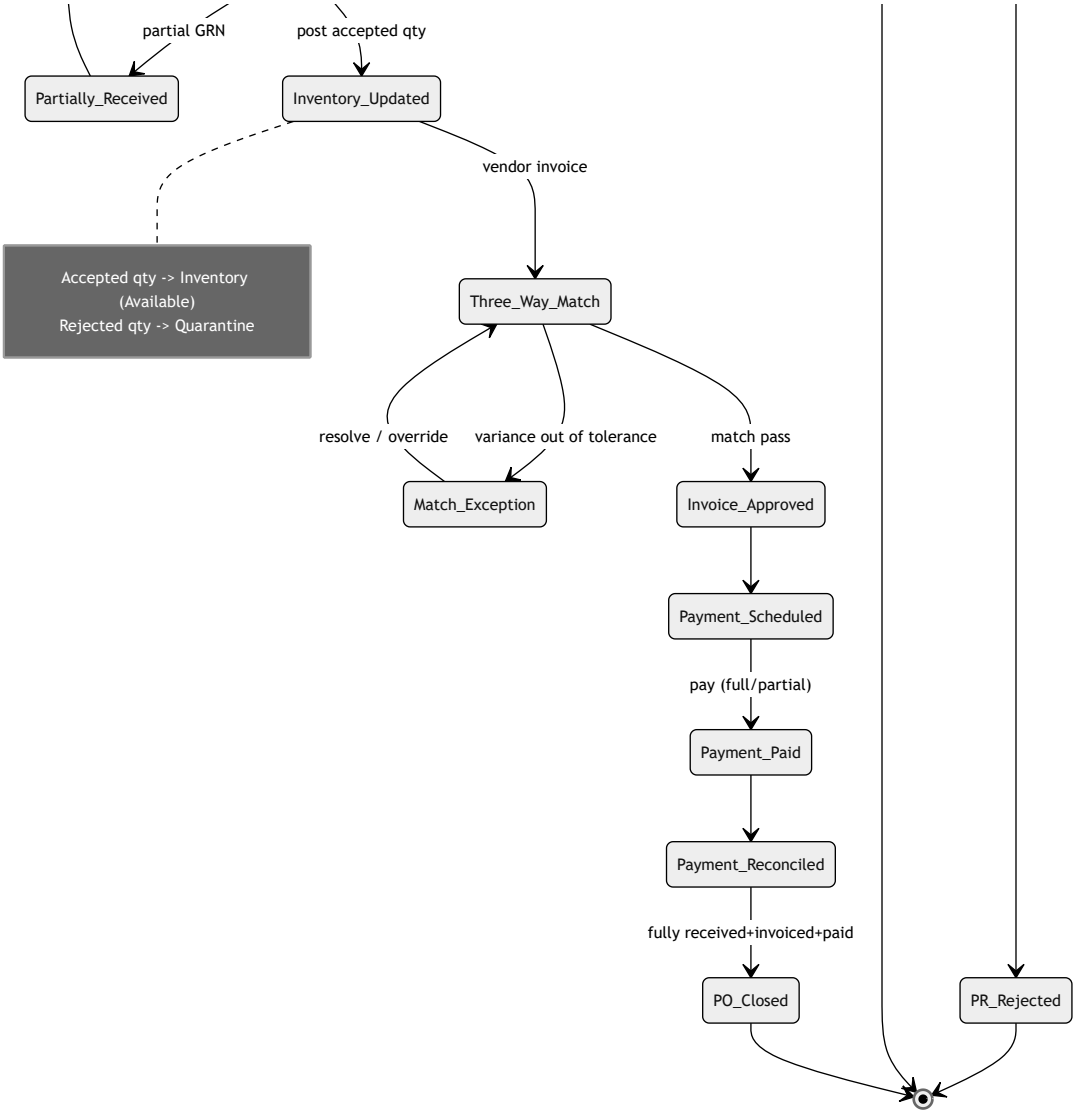
- **Spend analysis** by category / vendor / cost-center / period; top-N vendors; maverick (off-contract) spend.
- **Procurement cycle time** (PR→PO, PO→GRN, GRN→payment) with bottleneck breakdown.
- **Vendor scorecard**: OTIF, quality reject rate, price competitiveness, responsiveness, overall rating trend.
- **Budget vs actual / commitment utilization** with burn-down.
- **Open PO & GRN aging**; **3-way match exception log**; **savings realized** (quoted vs awarded vs list).
- Export: CSV/XLSX/PDF; scheduled email delivery; API for BI tools.

1.10 Mobile Support

- Approvers: push-notified PR/PO approvals with one-tap approve/reject + reason; view attachments and budget impact.
- Receivers: mobile Goods Receipt with camera barcode/QR scan of packing slips and serials, offline-capable receipt draft, photo capture of damage for quarantine.
- Buyers: read-only PO status, shipment tracking, vendor contact quick-dial; responsive web + native app parity for approvals and receipts.

1.11 Procurement Lifecycle — Mermaid


PR_Draft
submit
PR_Submitted
approve
PR_Approved
create RFQ
RFQ_Issued
vendors respond
Quotes_Received
score side-by-side
Quote_Comparison
award winning quote
PO_Draft
submit
Budget_Check
budget OK
insufficient budget
PO_Pending_Approval
reject
threshold approval
PO_Approved
PO_Rejected
send to vendor
PO_Issued
vendor ack
reject
Vendor_Acknowledged
dispatch
Shipment_InTransit
delivered
remaining lines
Goods_Receipt



MODULE 2 — INVENTORY (WAREHOUSE)

2.1 Purpose

Provide an enterprise warehouse inventory system that tracks every stock item across a hierarchical location model (site → building → room → warehouse → bin), maintaining real-time on-hand / reserved / available / in-transit balances, reorder governance, damaged/quarantine handling, consumable issue/consume with fractional quantities, barcode/QR identification, an immutable stock-movement ledger, cycle-count auditing, inter-warehouse transfers, and offline mobile counts with reconciliation. It receives stock from procurement Goods Receipts and, on deployment, converts a stock unit into a tracked Asset (with generated QR).

2.2 Features

- **Warehouses & hierarchical locations:** unlimited depth (site/building/room/warehouse/bin); capacity and pickable/receivable flags.
- **Stock items:** catalog-linked SKUs; serialized vs non-serialized; consumable vs durable; UoM + conversions; batch/lot & expiry support.
- **Stock levels:** on-hand, reserved, available (= on-hand − reserved), in-transit, quarantine — per item per location.
- **Low-stock thresholds & reorder points:** min/max, reorder point, reorder quantity; auto Purchase-Request suggestion when available ≤ reorder point.
- **Damaged / quarantine stock:** segregated non-available buckets; disposition workflow (return-to-vendor, repair, scrap/dispose).
- **Consumables:** issue and consume with fractional quantities; consumption to cost-center/user/asset.
- **Barcode & QR:** generate and scan Code-128/QR for items, bins, and asset tags; label printing.
- **Stock movements ledger:** append-only record of every receipt, issue, transfer, adjustment, reservation, deployment.
- **Stock audit / cycle count:** full and cyclic counts; blind counts; variance capture and posting.
- **Inventory transfer:** inter-warehouse and inter-location with in-transit tracking and two-step confirm (dispatch/receive).
- **Offline mobile stock count:** download count sheet, scan offline, sync with server-side reconciliation and conflict resolution.

2.3 Key Fields

2.3.1 Location (`locations`)

Field	Type	Required	Validation	Notes
id	UUID	Y	system	PK
tenant_id	UUID	Y	FK	RLS
parent_id	UUID	N	FK self, no cycles	hierarchy
type	enum	Y	`site|building|room|warehouse|bin`	level
code	string	Y	unique path per tenant	scannable
name	string	Y	2–120	
barcode	string	N	unique	bin label
is_receivable	bool	Y		can accept put-away
is_pickable	bool	Y		can issue from
capacity	decimal	N	≥ 0	optional constraint
status	enum	Y	`active|inactive`	

2.3.2 Stock Item (`stock_items`)

Field	Type	Required	Validation	Notes
sku	string	Y	unique per tenant	
catalog_item_id	UUID	N	FK product catalog	shared with procurement
name	string	Y	2–200	
category_id	UUID	Y	FK	
item_type	enum	Y	`durable|consumable`	drives fractional issue
is_serialized	bool	Y		true → unique unit tracking
uom	string	Y	FK unit-of-measure	base UoM
uom_conversions	json	N	positive factors	box↔ea
track_batch	bool	Y		lot/expiry
unit_cost	money	N	≥ 0	valuation (avg/FIFO per policy)
reorder_point	decimal(12,3)	N	≥ 0	triggers reorder
reorder_quantity	decimal(12,3)	N	≥ 0	suggested PR qty
min_level / max_level	decimal(12,3)	N	max ≥ min	
barcode / qr_code	string	N	unique	
status	enum	Y	`active|discontinued`	

2.3.3 Stock Level (`stock_levels`) — one row per (item, location, batch)

Field	Type	Required	Validation	Notes
stock_item_id	UUID	Y	FK	
location_id	UUID	Y	FK	
batch_no	string	N	required if track_batch	
expiry_date	date	N	≥ today at receipt	FEFO picking
qty_on_hand	decimal(12,3)	Y	≥ 0	physical
qty_reserved	decimal(12,3)	Y	0 ≤ reserved ≤ on_hand	soft allocation
qty_available	decimal(12,3)	Y	= on_hand − reserved (computed)	issuable
qty_in_transit	decimal(12,3)	Y	≥ 0	transfers/POs en route
qty_quarantine	decimal(12,3)	Y	≥ 0	not available
unique constraint	—	Y	(item, location, batch)	

2.3.4 Stock Movement Ledger (`stock_movements`) — append-only

Field	Type	Required	Validation
id	UUID	Y	system
movement_type	enum	Y	`receipt|issue|transfer_out|transfer_in|adjustment|reservation|release|deployment|quarantine|disposal|cou`
stock_item_id	UUID	Y	FK
from_location_id	UUID	N	FK
to_location_id	UUID	N	FK
quantity	decimal(12,3)	Y	> 0
batch_no	string	N	
reference_type	enum	Y	`grn|transfer|issue|audit|asset_deploy|manual`

Field	Type	Required	Validation
reference_id	UUID	N	FK
cost_center_id	UUID	N	FK
performed_by	UUID	Y	FK users
occurred_at	timestamptz	Y	≤ now

2.3.5 Transfer (`stock_transfers` , `transfer_lines`)

Field	Type	Required	Validation	Notes
transfer_number	string	Y	unique `TRF-{YYYY}-{seq}`	
from_location_id / to_location_id	UUID	Y	different, both active	
status	enum	Y	`draft|dispatched|in_transit|received|cancelled`	two-step
line: stock_item_id / quantity	mixed	Y	qty ≤ available at source	reserves on dispatch
line: quantity_received	decimal	Y	≤ dispatched	shortage → variance

2.3.6 Stock Audit / Cycle Count (`stock_audits` , `audit_lines`)

Field	Type	Required	Validation	Notes
audit_number	string	Y	unique	
type	enum	Y	`full|cycle|spot`	
location_scope	UUID[]	Y	FK	counted area
is_blind	bool	Y		hide expected qty
status	enum	Y	`planned|counting|review|posted|cancelled`	
line: expected_qty	decimal	Y	snapshot	
line: counted_qty	decimal	N	≥ 0	from device
line: variance	decimal	Y	counted − expected (computed)	
line: variance_reason	text	N	required if variance ≠ 0 on post	

2.3.7 Mobile Count Session (`count_sessions`) — offline

Field	Type	Required	Validation	Notes
audit_id	UUID	Y	FK	
device_id	string	Y		offline origin
synced_at	timestamptz	N		
conflict_status	enum	Y	`none|detected|resolved`	reconciliation
offline_entries	json[]	Y	each: item, location, qty, scanned_at	client-captured

2.4 Validations & Business Rules

- **Non-negative invariants:** `qty_on_hand ≥ 0` , `0 ≤ qty_reserved ≤ qty_on_hand` , `qty_available = qty_on_hand − qty_reserved` . Any movement violating these is rejected atomically.
- **Location hierarchy:** parent chain must be acyclic; stock can only rest in `receivable` / `pickable` bins; a location with stock cannot be deleted (soft-delete only).
- **Reservation:** reserving decreases available (not on-hand); issue/deploy converts reserved→out and reduces on-hand; releasing reservation restores available.
- **Fractional issue:** allowed only when `item_type = consumable` ; serialized/durable items require whole-unit movements and per-serial tracking.
- **FEFO/FIFO:** picking suggests earliest-expiry (batch) or FIFO by cost policy; expired batches block issue and route to disposition.
- **Quarantine isolation:** quarantine/damaged quantities are excluded from available and cannot be reserved, issued, or deployed until disposition (return/repair/scrap).
- **Reorder trigger:** when `qty_available ≤ reorder_point` , raise a low-stock alert and optionally auto-create a draft Purchase Request for `reorder_quantity` (feeds Procurement).
- **Transfer atomicity:** dispatch reserves and moves source→in-transit; receive moves in-transit→destination on-hand; short-receipt creates a variance requiring reason and adjustment.
- **Audit posting:** posting a count writes `count_adjust` movements for each variance and updates levels in one transaction; variance beyond tenant tolerance requires supervisor approval.
- **Offline reconciliation:** on sync, server compares device snapshot version to current; conflicting concurrent changes flagged, last-write-wins is disallowed — conflicts resolved by reviewer before posting.
- **Ledger immutability:** `stock_movements` are append-only; corrections are new reversing movements, never edits/deletes.
- **Barcode uniqueness:** item/bin/asset barcodes unique per tenant; QR generated at asset conversion is globally unique.
- **GRN→stock:** posting a Goods Receipt creates `receipt` movements for accepted qty (→ available) and `quarantine` movements for rejected qty; batch/serial captured at receipt.
- **Stock-out → Asset:** deploying a serialized durable unit consumes exactly one unit (`deployment` movement, on-hand−1) and creates one Asset record with the unit's serial + newly generated QR; non-serialized consumables cannot be converted to assets.

2.5 Permissions (roles)

Capability	Warehouse Operator	Inventory Manager	Auditor	Asset Manager	Requester/Employee	Tenant Admin
View stock levels	✓ (assigned WH)	✓	✓	✓	own requests	✓
Receive / put-away (post GRN)	✓	✓	–	–	–	✓
Issue / consume stock	✓	✓	–	–	request only	✓
Reserve / release	✓	✓	–	✓	–	✓
Create/receive transfer	✓	✓	–	–	–	✓
Manual adjustment	–	✓	–	–	–	✓
Manage locations/items/reorder	–	✓	–	–	–	✓
Plan/count audit	✓ (count)	✓	✓	–	–	✓
Post audit / approve variance	–	✓	✓ (recommend)	–	–	✓
Quarantine disposition	–	✓	–	✓ (return-to-repair)	–	✓
Deploy stock → Asset	–	✓	–	✓	–	✓
Inventory reports	scoped	✓	✓	✓	–	✓

2.6 API Endpoints (REST, `/api/v1/...`)

Locations & items - `GET|POST /api/v1/inventory/locations` · `GET|PATCH /locations/{id}` · `GET /locations/tree` - `GET|POST /api/v1/inventory/stock-items` · `GET|PATCH /stock-items/{id}` - `GET /api/v1/inventory/stock-items/{id}/barcode` · `POST /stock-items/{id}/qr`

Stock levels & movements - `GET /api/v1/inventory/stock-levels?item=&location=&batch=` - `GET /api/v1/inventory/stock-items/{id}/levels` - `GET /api/v1/inventory/movements` (ledger, filterable) - `POST /api/v1/inventory/adjustments` (manual, reason required) - `POST /api/v1/inventory/reservations` · `DELETE /reservations/{id}` (release) - `POST /api/v1/inventory/issues` (issue/consume, fractional for consumables)

Receiving (from procurement) - `POST /api/v1/inventory/receipts` (internal, called by GRN post) → `receipt` + `quarantine` movements

Quarantine / disposition - `GET /api/v1/inventory/quarantine` - `POST /api/v1/inventory/quarantine/{id}/disposition` `{action: return|repair|scrap, reason}`

Transfers - `GET|POST /api/v1/inventory/transfers` - `POST /api/v1/inventory/transfers/{id}/dispatch` - `POST /api/v1/inventory/transfers/{id}/receive` - `POST /api/v1/inventory/transfers/{id}/cancel`

Audit / cycle count - `GET|POST /api/v1/inventory/audits` - `GET /api/v1/inventory/audits/{id}/count-sheet` - `POST /api/v1/inventory/audits/{id}/counts` (submit counts) - `POST /api/v1/inventory/audits/{id}/post` (writes variance movements)

Offline mobile count - `GET /api/v1/inventory/audits/{id}/sync-package` (download) - `POST /api/v1/inventory/count-sessions/{id}/sync` (upload offline entries) - `GET /api/v1/inventory/count-sessions/{id}/conflicts` - `POST /api/v1/inventory/count-sessions/{id}/resolve`

Reorder & deployment - `GET /api/v1/inventory/reorder-suggestions` - `POST /api/v1/inventory/reorder-suggestions/{id}/create-pr` → Procurement PR - `POST /api/v1/inventory/stock-items/{id}/deploy` `{serial, assignee, location}` → creates Asset + QR

Reports - `GET /api/v1/inventory/reports/{stock-summary|valuation|movement-ledger|low-stock|aging|audit-variance|transfer-status|consumption}`

2.7 Workflow

1. **Receive:** GRN post creates stock → accepted to **Available**, rejected to **Quarantine**.
2. **Store & manage:** put-away to bins; levels maintained; reorder monitoring runs continuously.
3. **Allocate:** reserve stock against a request/deployment (Available→Reserved).
4. **Fulfil:** issue/consume (consumables) or **deploy** (durable serialized) → on-hand decreases; deployment spawns an Asset with QR.
5. **Move:** inter-warehouse transfer with dispatch (→in-transit) and receive (→destination).
6. **Damage handling:** quarantine → disposition (return/repair/scrap-dispose).
7. **Audit:** plan → count (incl. offline) → reconcile → post variance adjustments.
8. **Replenish:** low-stock/reorder-point breach → suggestion → draft Purchase Request → Procurement loop.

2.8 Notifications

- Low-stock / reorder-point reached → inventory manager (+ optional auto-PR notice to procurement).
- Expiry approaching / expired batch → inventory manager.
- Goods received / put-away complete → requester + manager.
- Reservation created/released; issue/consume posted → requester + cost-center owner.
- Transfer dispatched / received / shortage → source + destination managers.
- Quarantine created and disposition due → inventory + asset manager.
- Audit assigned / count due / variance beyond tolerance → counters + manager + auditor.
- Offline sync conflict detected → reviewer.
- Deployment → Asset created → asset manager + assignee.
- Channels: in-app, email, push; per-warehouse subscription routing.

2.9 Reports

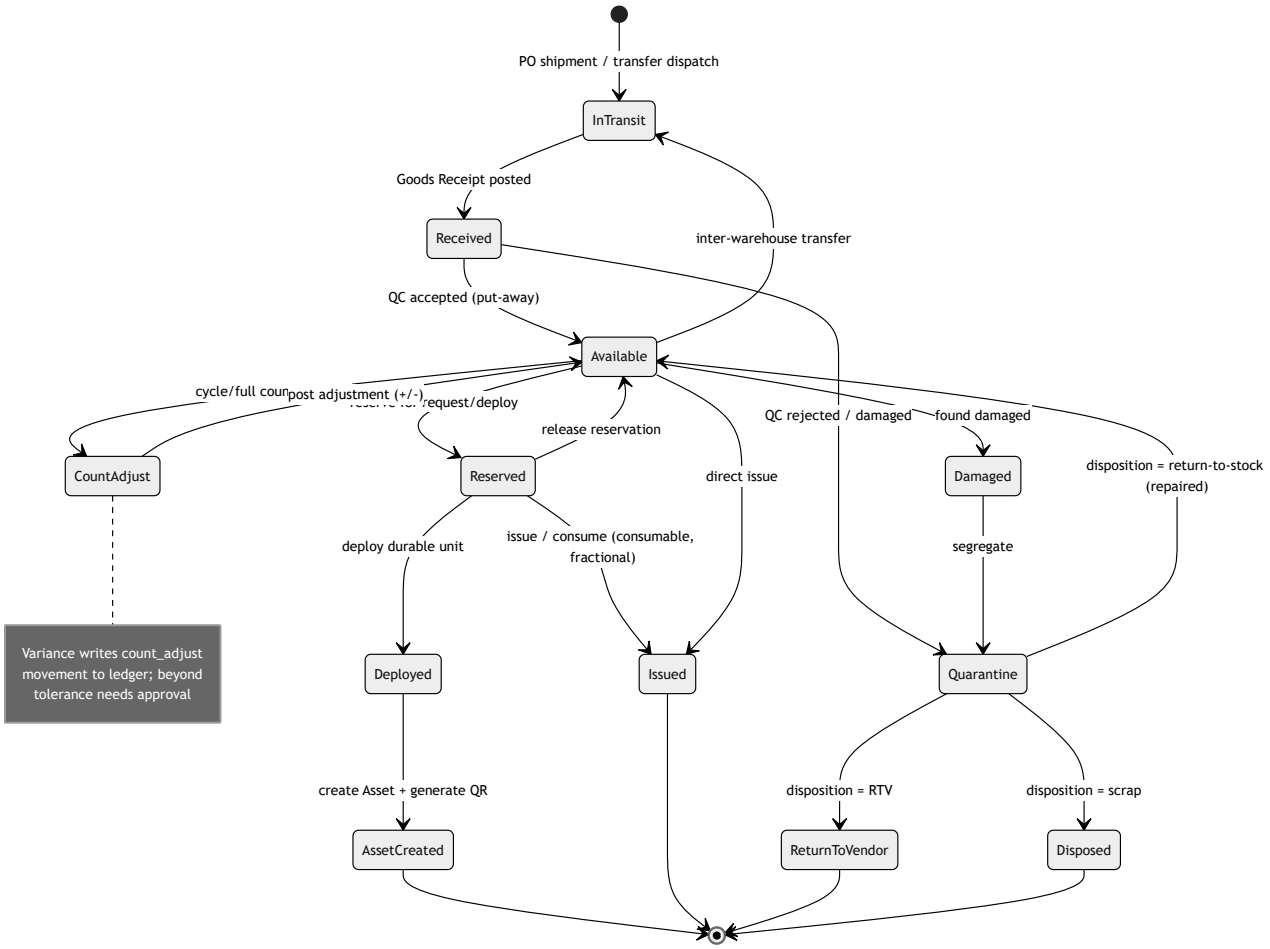
- **Stock summary & valuation** (on-hand/reserved/available/in-transit/quarantine) by item/location/category; valuation by avg/FIFO.
- **Movement ledger** (full auditable history, filter by type/reference).
- **Low-stock & reorder** report with suggested PR quantities.
- **Inventory aging / slow-moving / dead stock**; **batch expiry** report.
- **Audit variance** (planned vs counted, shrinkage %); **transfer status/aging**.

- **Consumption** by cost-center/user/asset; **quarantine & disposition** log.
- Export CSV/XLSX/PDF; scheduled delivery; BI API.

2.10 Mobile Support

- Native scanner (camera + Bluetooth ring scanner) for barcode/QR on items, bins, and assets.
- **Offline-first counting:** download count sheet, scan and enter quantities with no connectivity, queue locally, sync on reconnect with conflict reconciliation.
- Mobile receive/put-away, issue/consume, and transfer dispatch/receive with scan validation.
- On-device **deploy-to-Asset**: scan stock unit, capture assignee/location, generate and print/display QR label for the new asset.
- Photo capture for damage/quarantine; push notifications for tasks (counts, transfers, low-stock); responsive web parity for supervisors.

2.11 Inventory Lifecycle / Stock-Movement — Mermaid



CROSS-MODULE INTEGRATION

3.1 Goods Receipt → Inventory

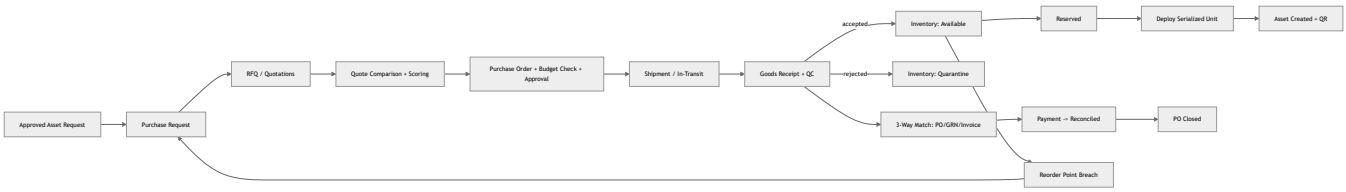
When a GRN is **posted** (`POST /goods-receipts/{id}/post`), the system emits `inventory.stock.received` carrying `{grn_id, po_id, warehouse_id, lines[]}` . For each line: **accepted qty** creates a `receipt` movement into the target bin (Received → Available, with batch/serial captured), and **rejected qty** creates a `quarantine` movement (Received → Quarantine). The PO line `quantity_received` is updated, the budget `amount_actual` is incremented, and item valuation is recomputed. The GRN document ID is stored as `reference_id` on every movement, giving end-to-end PO ↔ GRN ↔ stock traceability.

3.2 Inventory Stock-Out → Asset (deployment + QR)

When a serialized durable unit is **deployed** (`POST /stock-items/{id}/deploy` with `{serial, assignee, location}`): 1. Validate one available/reserved unit for that serial; reject fractional or non-serialized deployment. 2. Write a `deployment` movement (on-hand − 1; Reserved/Available → Deployed). 3. **Create an Asset record** in the ITAM asset registry: inherit SKU, category, unit_cost (as acquisition cost), serial, PO/GRN provenance (warranty start = receipt date), assign to the user/location. 4. **Generate a globally-unique QR code** bound to the new asset ID; make it printable from web/mobile. 5. Emit `asset.created.from_inventory` for lifecycle, warranty, and audit linkage.

This makes procurement, inventory, and asset lifecycle a single traceable chain: **PR → RFQ → PO → Shipment → GRN → Stock → Deployment → Asset (QR)**, with budget commitments and 3-way match governing spend and every quantity change captured in the immutable movement ledger.

End-to-End Traceability Chain (Mermaid)



Prepared for the TechpioAsset Enterprise ITAM Redesign Blueprint — Procurement & Inventory volume. All entities are multi-tenant, audit-logged, and API-first; existing invoice/vendor functionality is retained and extended by the vendor, invoice-match, and payment specifications above.

PART 6

Maintenance, Notification Matrix & Audit Matrix

PART A — MAINTENANCE MODULE (Full FRD)

A.1 Purpose

The Maintenance module governs the full service lifecycle of any managed asset (hardware, peripherals, infrastructure, facility equipment) from fault detection through return-to-inventory. It unifies **reactive (corrective)**, **scheduled (preventive)**, **telemetry-driven (predictive)**, **contract-based (AMC)**, and **warranty-claim** work under one work-order model, with cost governance, SLA timers, spare-parts consumption tied to Inventory, technician e-signature capture, and MTTR/uptime analytics.

Business objectives - Reduce mean-time-to-repair (MTTR) and unplanned downtime via structured triage → diagnosis → repair flow. - Enforce **cost-threshold approvals** so no spend occurs without the right authority. - Maintain a defensible, immutable service history per asset (audit, warranty disputes, depreciation adjustment, resale value). - Automate preventive schedules and predictive triggers to shift the reactive/proactive ratio. - Give vendors a scoped external portal to receive, update, and close service tickets without internal system access.

A.2 Features

#	Feature	Description
F-01	Work Order (WO) engine	Central entity for all maintenance types; state-machine driven.
F-02	Maintenance types	Preventive, Corrective, Predictive, AMC, Warranty-Claim.
F-03	Triage & inspection	Structured intake, severity/priority scoring, initial photos.
F-04	Diagnosis capture	Fault codes, root cause, recommended action, estimated cost/parts.
F-05	Cost-threshold approval	Multi-tier approval routing based on estimated cost bands.
F-06	Technician assignment	Skill/queue/load-based assignment; reassignment; escalation.
F-07	Repair paths	Internal repair, vendor service dispatch, warranty claim — mutually branching.
F-08	Spare-parts consumption	Reserve → consume parts from Inventory; auto stock decrement + low-stock trigger.
F-09	Checklist templates	Reusable inspection/repair/QC checklists per asset category.
F-10	Photos & attachments	Before/during/after media, invoices, warranty docs, e-sign PDFs.
F-11	Technician e-signature	Captured on completion; frozen into the WO record + PDF report.
F-12	SLA timers	Response & resolution SLAs by priority; pause on approval/vendor/parts waits.
F-13	Vendor service tickets	External ticket ref, dispatch, RMA, vendor SLA, cost reconciliation.
F-14	AMC management	Contract coverage lookup, entitlement check, visit scheduling, PM calendar.
F-15	Warranty engine	Coverage validation, claim submission, RMA tracking, replacement receipt.
F-16	Out-of-service cascade	Auto status change, un-assign user, reclaim licenses, loaner issuance.
F-17	Preventive scheduler	Recurrence rules (RRULE), meter-based (usage), auto-WO generation.
F-18	Predictive triggers	Health-score / telemetry thresholds auto-raise WOs.
F-19	Quality check gate	Independent QC sign-off before return-to-inventory.
F-20	Cost & downtime ledger	Labor + parts + vendor cost roll-up; downtime clock; MTTR/MTBF.
F-21	Reports & dashboards	MTTR, uptime, cost per asset, PM compliance, SLA attainment.
F-22	Mobile execution	Technician mobile app: queue, checklist, photo, part-scan, e-sign, offline.
F-23	Notifications	Event-driven multichannel alerts (see Part B).
F-24	Full audit trail	Immutable event log of every decision/field change (see Part C).

A.3 Data Model (Field Tables)

A.3.1 `maintenance_work_order`

Field	Type	Req	Notes / Validation
id	UUID (PK)	Y	System-generated.
tenant_id	UUID (FK)	Y	RLS scope.
wo_number	String	Y	Human ref, e.g. `WO-2026-004521` ; unique per tenant.
asset_id	UUID (FK→asset)	Y	Must be an active, non-retired asset.
type	Enum	Y	`PREVENTIVE` | `CORRECTIVE` | `PREDICTIVE` | `AMC` | `WARRANTY_CLAIM` .
category	Enum	Y	Derived from asset category (hardware/peripheral/infra/facility).
title	String(200)	Y	Non-empty.
description	Text	Y	Min 10 chars for corrective.
priority	Enum	Y	`P1` .. `P4` ; drives SLA.

Field	Type	Req	Notes / Validation
severity	Enum	Y	`Critical/High/Medium/Low` .
status	Enum	Y	See A.5 state machine.
reported_by	UUID (FK→user)	Y	Auto = actor unless on behalf.
reported_channel	Enum	Y	`Portal/Email/Phone/Auto-Predictive/Scheduler` .
reported_at	Timestamp	Y	Server time.
triaged_by	UUID	N	Set at triage.
assigned_technician_id	UUID (FK→user)	N	Required to leave `ASSIGNED` .
assigned_vendor_id	UUID (FK→vendor)	N	Required for vendor path.
repair_path	Enum	N	`INTERNAL` | `VENDOR` | `WARRANTY` .
estimated_cost	Decimal(12,2)	N	≥ 0; drives approval tier.
approved_cost	Decimal(12,2)	N	Set on approval.
actual_cost	Decimal(12,2)	N	Roll-up (labor+parts+vendor).
currency	ISO-4217	Y	Default tenant currency.
downtime_start	Timestamp	N	Set on out-of-service.
downtime_end	Timestamp	N	Set at return-to-inventory.
sla_response_due	Timestamp	N	Computed from priority policy.
sla_resolution_due	Timestamp	N	Computed; pause-aware.
sla_breached	Boolean	Y	Default false.
checklist_template_id	UUID (FK)	N	Applied at assignment.
amc_contract_id	UUID (FK)	N	Required for AMC type.
warranty_id	UUID (FK)	N	Required for warranty type.
esignature_id	UUID (FK)	N	Required to reach `COMPLETE` .
root_cause_code	Enum/String	N	Set at diagnosis.
resolution_code	Enum	N	`Repaired/Replaced/NoFaultFound/Retired/Warranty` .
created_at / updated_at	Timestamp	Y	Audit.

A.3.2 `maintenance_diagnosis`

Field	Type	Req	Notes
id, tenant_id, work_order_id	UUID	Y	FK to WO.
fault_code	String	N	Category-specific dictionary.
root_cause	Text	Y	
recommended_action	Enum	Y	`Repair/ReplacePart/VendorService/Warranty/Retire` .
estimated_labor_hours	Decimal(6,2)	N	≥ 0.
estimated_parts_cost	Decimal(12,2)	N	≥ 0.
diagnosed_by	UUID	Y	
diagnosed_at	Timestamp	Y	

A.3.3 `maintenance_part_consumption` (links to Inventory)

Field	Type	Req	Notes
id, tenant_id, work_order_id	UUID	Y	
inventory_item_id	UUID (FK→inventory)	Y	Must be a stocked spare.
quantity_reserved	Int	Y	≤ available stock.
quantity_consumed	Int	Y	≤ reserved; decrements stock on commit.
unit_cost	Decimal(12,2)	Y	Snapshot at consumption.
serial_numbers	String[]	N	Required for serialized parts.
consumed_by	UUID	Y	
consumed_at	Timestamp	Y	

A.3.4 `maintenance_checklist_result`

Field	Type	Req	Notes
id, tenant_id, work_order_id	UUID	Y	
template_id	UUID (FK)	Y	
item_key	String	Y	
item_label	String	Y	
result	Enum	Y	`Pass/Fail/NA` .

Field	Type	Req	Notes
value	String	N	For measurement items.
note	Text	N	Required when `Fail`.
photo_attachment_id	UUID	N	
completed_by	UUID	Y	

A.3.5 `maintenance_attachment`

Field	Type	Req	Notes
id, tenant_id, work_order_id	UUID	Y	
kind	Enum	Y	`Photo_Before/Photo_During/Photo_After/Invoice/WarrantyDoc/ESignPDF/Other`.
file_url	String	Y	Object-store key; virus-scanned.
mime_type	String	Y	Whitelist: image/*, application/pdf.
size_bytes	Int	Y	≤ 25 MB.
uploaded_by	UUID	Y	

A.3.6 `maintenance_esignature`

Field	Type	Req	Notes
id, tenant_id, work_order_id	UUID	Y	
signer_id	UUID	Y	Technician (or vendor on portal).
signer_role	Enum	Y	
signature_image_url	String	Y	
signed_hash	String(64)	Y	SHA-256 of WO snapshot at sign time — tamper evidence.
signed_at	Timestamp	Y	
ip_address	String	Y	

A.3.7 `maintenance_vendor_ticket`

Field	Type	Req	Notes
id, tenant_id, work_order_id, vendor_id	UUID	Y	
external_ref	String	N	Vendor's own ticket number.
rma_number	String	N	For part/unit return.
dispatch_at / eta / closed_at	Timestamp	N	
vendor_cost	Decimal(12,2)	N	Reconciled to actual_cost.
vendor_sla_due	Timestamp	N	
status	Enum	Y	`Dispatched/InService/AwaitingParts/Closed`.

A.3.8 `maintenance_schedule` (Preventive / AMC recurrence)

Field	Type	Req	Notes
id, tenant_id	UUID	Y	
asset_id or asset_group_id	UUID	Y	One required.
rrule	String	N	iCal RRULE (calendar-based).
meter_type / meter_threshold	Enum/Int	N	Usage-based (e.g. 500 print-hrs).
checklist_template_id	UUID	Y	
lead_time_days	Int	Y	WO auto-created N days before due.
next_due_at	Timestamp	Y	
active	Boolean	Y	

A.3.9 `maintenance_health_signal` (Predictive)

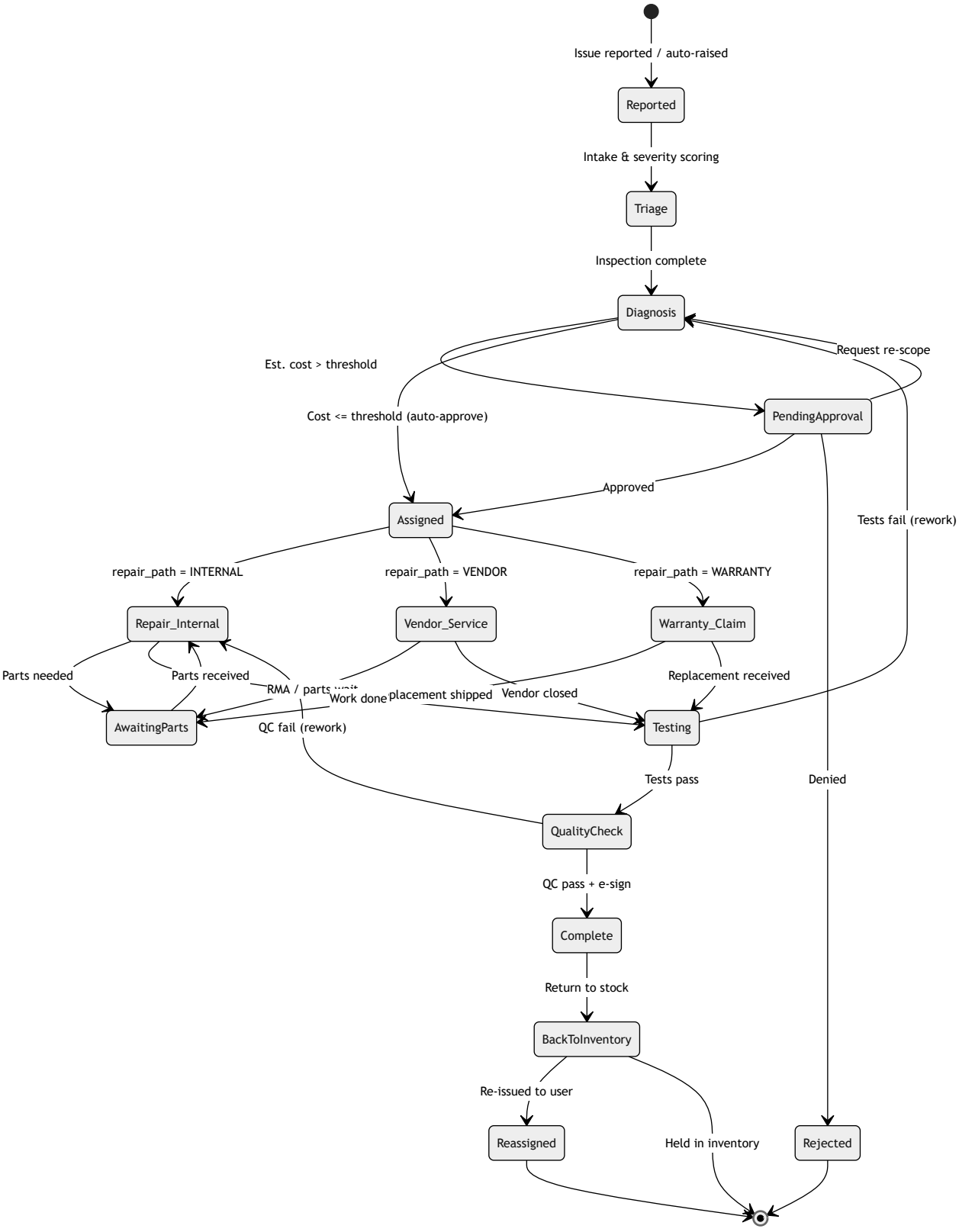
Field	Type	Req	Notes
id, tenant_id, asset_id	UUID	Y	
metric	Enum	Y	`disk_smart/battery_health/temp/error_rate/uptime`.
value / threshold	Decimal	Y	
health_score	Int (0–100)	Y	
triggered_wo_id	UUID	N	Set when auto-raised.
received_at	Timestamp	Y	

A.4 Validations (Business Rules)

ID	Rule
V-01	An asset with an **open** WO cannot be assigned/transferred to a user (blocked by cascade).
V-02	`type = AMC` requires a valid, in-force `amc_contract_id` covering the asset.
V-03	`type = WARRANTY_CLAIM` requires `warranty_id` with `warranty_end >= today`. Expired → block with "warranty lapsed".
V-04	Cannot leave `PENDING_APPROVAL` without an approval record when `estimated_cost > tier-0 threshold`.
V-05	`quantity_consumed ≤ quantity_reserved ≤ available_inventory`. Consumption commits an atomic stock decrement.
V-06	Serialized parts must supply `serial_numbers` count == `quantity_consumed`.
V-07	Cannot reach `TESTING` until all `Fail` checklist items have a note + corrective action.
V-08	Cannot reach `QUALITY_CHECK` unless a valid technician e-signature exists.
V-09	QC must be performed by a user **different** from `assigned_technician_id` (segregation of duties) when tenant policy `qc_independent = true`.
V-10	Cannot reach `COMPLETE` with an open vendor ticket (`status != Closed`).
V-11	`downtime_end ≥ downtime_start`; MTTR computed only on valid pairs.
V-12	SLA clock **pauses** in `PENDING_APPROVAL`, `AWAITING_PARTS`, `VENDOR_SERVICE`, `WARRANTY_CLAIM`; resumes on exit.
V-13	`actual_cost` must reconcile = Σ(labor) + Σ(part consumption) + vendor_cost within tolerance ±configured %.
V-14	Vendor portal actor may only edit WOs where `assigned_vendor_id` = their vendor org.
V-15	`estimated_cost/approved_cost/actual_cost ≥ 0`; currency must match tenant or contract currency.
V-16	Priority P1 auto-sets severity Critical and forces immediate triage SLA.
V-17	Return-to-inventory only allowed from `COMPLETE`; sets asset status `Available` and clears out-of-service flag.

A.5 Maintenance Lifecycle

A.5.1 State Diagram

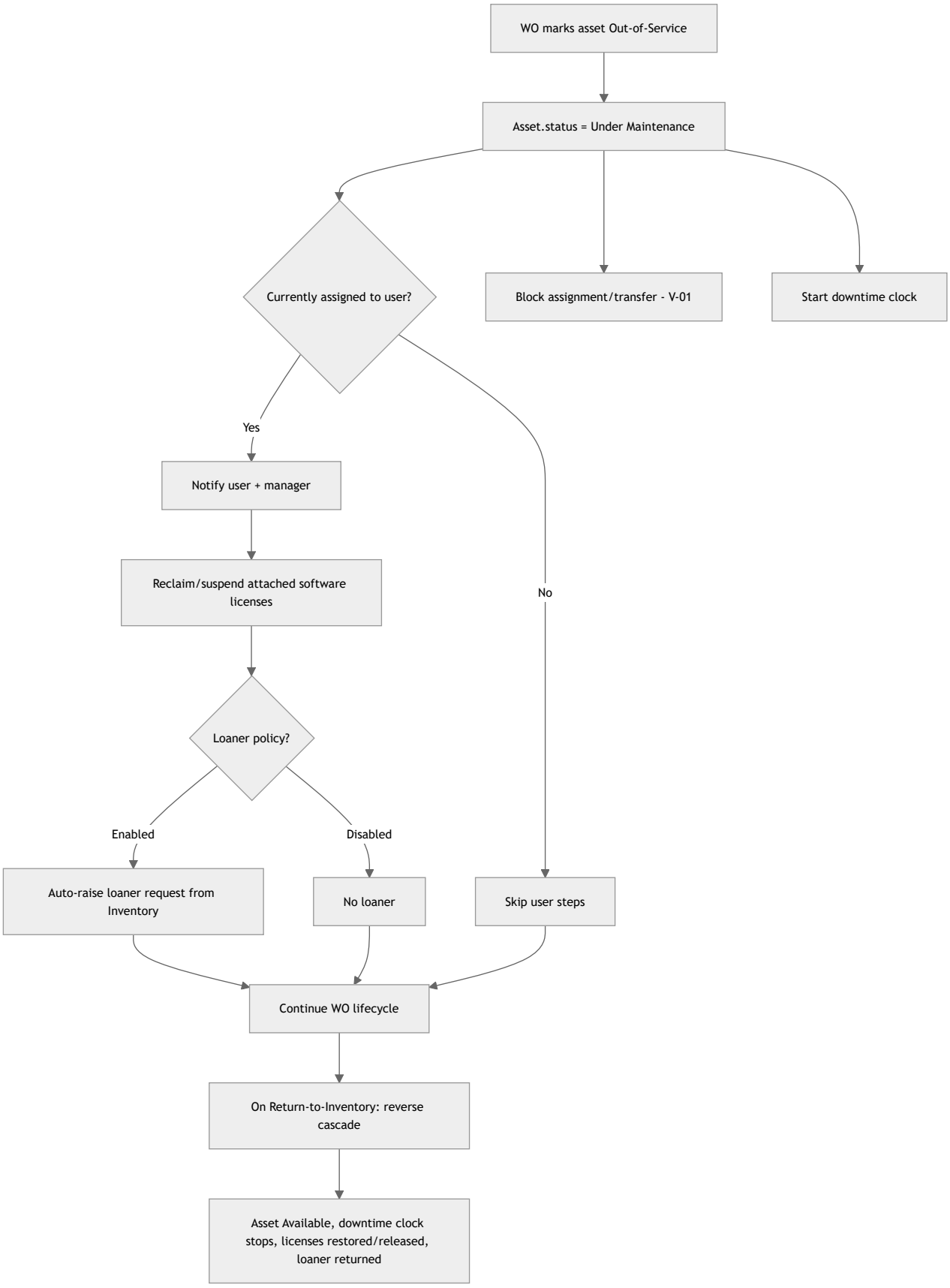


A.5.2 Lifecycle Step Table

Stage	Entry trigger	Actor	Key actions	Exit condition	SLA clock
Reported	User portal / email / predictive signal / scheduler	Any / System	Create WO, capture description, initial photos	Assigned to triage	Response starts
Triage / Inspection	WO created	IT Technician	Set priority/severity, verify fault, out-of-service decision	Diagnosis begins	Running
Diagnosis	Triage done	IT Technician	Root cause, fault code, recommended action, estimate parts/labor	Estimate produced	Running

Stage	Entry trigger	Actor	Key actions	Exit condition	SLA clock
Approval (cost threshold)	Est. cost > tier threshold	IT Manager / Office Admin	Review estimate, approve/reject/re-scope	Approved / Rejected	**Paused**
Assign Technician	Approved or auto-approved	IT Manager / auto-router	Choose repair path, assign tech/vendor, apply checklist	Path selected	Running
Repair (Internal)	Path = INTERNAL	IT Technician	Execute checklist, consume parts, photos	Work complete	Running
Vendor Service	Path = VENDOR	Vendor + IT Tech liaison	Dispatch, external ticket, RMA, vendor cost	Vendor closes	**Paused (vendor)**
Warranty Claim	Path = WARRANTY	IT Technician	Validate coverage, submit claim, track RMA/replacement	Replacement received	**Paused**
Parts / Spare	Parts required	IT Technician	Reserve → consume from Inventory, decrement stock	Parts fitted	Paused (awaiting)
Testing	Repair/service/warranty done	IT Technician	Functional test, record measurements	Pass → QC; Fail → re-diagnose	Running
Quality Check	Testing pass	IT Manager / QC tech (independent)	QC checklist, verify e-signature	Pass → Complete	Running
Complete	QC pass + e-sign	System	Freeze record, roll-up actual cost, stop downtime	Return-to-inventory	Resolution stops
Back to Inventory	Complete	System / Office Admin	Set asset Available, clear out-of-service, close loaner	Held or reissued	—
Re-assigned	Inventory available	IT Manager / Office Admin	Assign to user, deliver	—	—

A.5.3 Asset Out-of-Service Cascade



A.6 Permissions Matrix (RBAC)

Legend: **C**reate · **R**ead · **U**pdate · **A**pprove · **X** none

Capability	IT Manager	IT Technician	Office Admin	Vendor (portal)
Create WO	C	C	C	X

Capability	IT Manager	IT Technician	Office Admin	Vendor (portal)
View WO (tenant-wide)	R	R (assigned/queue)	R	R (own vendor only)
Triage / set priority	RU	RU	X	X
Diagnosis	RU	RU	X	X
Approve cost (Tier 1)	A	X	A (≤ admin cap)	X
Approve cost (Tier 2/3)	A	X	X	X
Assign technician	U	X	U	X
Consume parts / Inventory	U	U	R	X
Update vendor ticket	U	U	R	U (own)
Upload attachments	CU	CU	CU	CU (own WO)
Technician e-sign	X	Sign	X	Sign (own WO)
Quality check sign-off	A	A (if independent)	X	X
Return-to-inventory	U	X	U	X
Re-assign asset	U	X	U	X
Edit checklist templates	CU	R	R	X
Edit SLA / cost thresholds	U	X	R	X
View cost fields	R	R (est only, config)	R	R (vendor cost only)
Reports / dashboards	R (all)	R (own KPIs)	R (ops)	X

Vendor scope is enforced at the query layer by `assigned_vendor_id = actor.vendor_org_id`; vendors never see cross-tenant or unrelated WOs.

A.7 API Surface (`/api/v1/maintenance/...`)

All endpoints: tenant-scoped via `X-Tenant-Id` + JWT; RBAC-guarded; idempotency-key on writes; standard envelope `{data, meta, errors}`.

Method	Path	Purpose	Roles
POST	`/work-orders`	Create WO	Mgr/Tech/Admin
GET	`/work-orders`	List/filter (status,type,priority,asset,assignee,SLA)	Mgr/Tech/Admin/Vendor(own)
GET	`/work-orders/{id}`	Retrieve WO detail	scoped
PATCH	`/work-orders/{id}`	Update fields	scoped
POST	`/work-orders/{id}/transition`	State transition `{to_state, payload}`	scoped
POST	`/work-orders/{id}/triage`	Submit triage (priority/severity/OOS)	Tech/Mgr
POST	`/work-orders/{id}/diagnosis`	Add diagnosis + estimate	Tech/Mgr
POST	`/work-orders/{id}/approvals`	Approve/reject/re-scope	Mgr/Admin
POST	`/work-orders/{id}/assign`	Assign tech/vendor + path	Mgr/Admin
POST	`/work-orders/{id}/parts`	Reserve/consume parts	Tech
DELETE	`/work-orders/{id}/parts/{pid}`	Release reservation	Tech
POST	`/work-orders/{id}/checklist`	Submit checklist results	Tech
POST	`/work-orders/{id}/attachments`	Upload media/doc (multipart)	scoped
POST	`/work-orders/{id}/esign`	Capture e-signature	Tech/Vendor
POST	`/work-orders/{id}/vendor-ticket`	Create/update vendor ticket	Tech/Mgr/Vendor(own)
POST	`/work-orders/{id}/warranty-claim`	Submit warranty claim	Tech/Mgr
POST	`/work-orders/{id}/qc`	QC sign-off	Mgr/QC-Tech
POST	`/work-orders/{id}/complete`	Finalize + cost roll-up	system/Mgr
POST	`/work-orders/{id}/return-to-inventory`	Reverse OOS cascade	Mgr/Admin
GET	`/work-orders/{id}/history`	Immutable audit timeline	scoped
GET/POST/PATCH	`/schedules`	Preventive/AMC recurrence CRUD	Mgr/Admin
POST	`/health-signals`	Ingest telemetry (predictive)	service/agent
GET	`/reports/mttr` · `/reports/uptime` · `/reports/cost` · `/reports/sla` · `/reports/pm-compliance`	Analytics	Mgr/Admin
GET	`/checklist-templates` (+CRUD)	Template management	Mgr

Webhooks: `maintenance.wo.created`, `.approved`, `.assigned`, `.sla_breached`, `.completed`, `.parts_low_stock` → tenant integrations (Teams/Slack/ITSM).

A.8 Workflow Automation Rules

Rule	Behavior
Auto-approval	`estimated_cost ≤ Tier-0 threshold` → skip PENDING_APPROVAL.

Rule	Behavior
Approval routing	Tier by cost band (see B.2 & config); escalate if approver SLA missed.
Auto-assignment	Round-robin/skill-match within technician queue for asset category.
SLA escalation	50% → notify assignee; 80% → notify manager; 100% → breach flag + escalation.
Preventive generation	Scheduler creates WO `lead_time_days` before `next_due_at`; recomputes next.
Predictive trigger	Health score < threshold or metric breach → auto-create Predictive WO (dedupe open).
Low-stock cascade	Part consumption crossing reorder point → Inventory low-stock event.
Rework loop	Testing/QC fail returns to Diagnosis/Repair, increments `rework_count`.

A.9 Notifications (module-level; see full Part B)

Core maintenance events: WO created, assigned, approval requested/granted/denied, awaiting parts, vendor dispatched, SLA at-risk/breached, testing failed, QC result, completed, returned-to-inventory, asset out-of-service (to affected user), preventive due, predictive alert.

A.10 Reports & Metrics

Report	Metric definition
MTTR	Σ(downtime_end − downtime_start) / count(completed WOs), by asset/category/type/period.
MTBF	Σ(uptime between failures) / failure count.
Uptime %	(Total period − downtime) / total period × 100, per asset/fleet.
SLA attainment	% WOs resolved within SLA (response & resolution split).
PM compliance	Preventive WOs completed on-time / scheduled.
Cost per asset	Lifetime maintenance cost vs. asset value (repair-vs-replace signal).
First-time-fix rate	WOs completed without rework (rework_count = 0).
Warranty recovery	Cost avoided via warranty vs. total repairs.
Vendor performance	Vendor SLA attainment, avg cost, reopen rate.
Backlog / aging	Open WOs by age bucket & priority.

Dashboards: Operations (queue, SLA, backlog), Executive (uptime, cost trend, PM ratio), Asset 360 (per-asset service history + e-signed PDFs).

A.11 Mobile Support (Technician App)

Capability	Detail
My Queue	Assigned WOs, priority-sorted, SLA countdown badges.
Guided execution	Step-through checklist with pass/fail + measurement inputs.
Camera capture	Before/during/after photos attach directly to WO.
Barcode/QR scan	Scan asset tag to open WO; scan part to record consumption.
Offline mode	Cache assigned WOs; queue actions; sync with conflict resolution on reconnect.
E-signature	On-device signature pad → generates signed hash + PDF.
Push notifications	New assignment, reassignment, SLA at-risk, approval granted.
Location/timestamp	Optional geo + time stamp on completion for field service.
Parts lookup	Real-time Inventory availability before reserving.

PART B — NOTIFICATION MATRIX

B.1 Channels & Conventions

Channels: **In-app** · **Email** · **Push** (mobile) · **SMS** · **Webhook** (Teams/Slack/ITSM). Mandatory = cannot be disabled by user preference (governance/compliance). Reminder cadences default **90/60/30/7-day** for expiries; configurable per tenant.

B.2 Comprehensive Notification Matrix

#	Module	Event	Recipient role(s)	In-app	Email	Push	SMS	Webhook	Mandatory	Trigger / Timing	Template summary
N-01	Request	New asset request submitted	Requester, IT Manager	✓	✓	–	–	✓	Immediate	"Request {id} for {item} submitted, pending approval."	
N-02	Request	Approval needed	Approver (Mgr/Admin)	✓	✓	✓	–	✓	Y	Immediate + reminder @24h	"Action: approve request {id} ({cost})."
N-03	Request	Approved / Rejected	Requester	✓	✓	✓	–	–	Immediate	"Your request {id} was {decision}."	
N-04	Procurement	PO created / issued	Requester, Finance, Vendor	✓	✓	–	–	✓	Immediate	"PO {po} issued to {vendor}, total {amt}."	

#	Module	Event	Recipient role(s)	In-app	Email	Push	SMS	Webhook	Mandatory	Trigger / Timing	Template summary
N-05	Procurement	PO approval threshold	Finance approver	✓	✓	–	–	✓	Y	Immediate	"PO {po} {amt} exceeds {tier}, approve."
N-06	Goods Receipt	GRN posted / partial	Requester, IT Manager, Store	✓	✓	–	–	✓	Immediate	"Goods for PO {po} received ({qty}/{ordered})."	
N-07	Goods Receipt	Receipt discrepancy	IT Manager, Procurement	✓	✓	–	–	✓	Y	Immediate	"Discrepancy on GRN {id}: {detail}."
N-08	Asset	Assigned to user	Assignee, Manager	✓	✓	✓	–	–	Immediate	"{asset} assigned to you; acknowledge receipt."	
N-09	Asset	Return requested	Current holder, Manager	✓	✓	✓	–	–	Immediate + reminder	"Please return {asset} by {date}."	
N-10	Asset	Transfer to new user/site	From-user, To-user, Managers	✓	✓	✓	–	✓	Immediate	"{asset} transferred from {a} to {b}."	
N-11	License	Seat limit reached	IT Manager, Software Admin	✓	✓	–	–	✓	Y	Immediate	"{software} at {used}/{total} seats."
N-12	License	Expiry approaching	IT Manager, Finance	✓	✓	–	–	✓	Y	90/60/30/7-day	"{software} expires {date}; renew."
N-13	License	Renewal due / auto-renew	IT Manager, Finance	✓	✓	–	–	✓	30/7-day	"Renewal for {software} due {date}, {amt}."	
N-14	License	Unused / reclaimable seats	Software Admin	✓	✓	–	–	–	Weekly digest	"{n} unused {software} seats reclaimable."	
N-15	Maintenance	WO created	Assigned tech, Manager	✓	✓	✓	–	✓	Immediate	"WO {id} for {asset} created ({priority})."	
N-16	Maintenance	Approval (cost threshold)	Manager / Office Admin	✓	✓	✓	–	✓	Y	Immediate	"Approve WO {id} est. {cost}."
N-17	Maintenance	Assigned / reassigned	Technician	✓	–	✓	–	–	Immediate	"WO {id} assigned to you."	
N-18	Maintenance	Awaiting parts	Technician, Store	✓	✓	–	–	✓	Immediate	"WO {id} awaiting part {sku}."	
N-19	Maintenance	Vendor dispatched	Vendor, Manager	✓	✓	–	–	✓	Immediate	"Vendor {v} dispatched for WO {id}, ETA {eta}."	
N-20	Maintenance	SLA at-risk (50/80%)	Assignee then Manager	✓	✓	✓	–	✓	Y	At threshold	"WO {id} SLA at {pct}%."
N-21	Maintenance	SLA breached	Manager, Assignee	✓	✓	✓	✓(P1)	✓	Y	On breach	"WO {id} SLA BREACHED."
N-22	Maintenance	Testing/QC failed	Technician, Manager	✓	✓	✓	–	–	Immediate	"WO {id} failed {stage}, rework."	
N-23	Maintenance	Completed	Requester, Manager	✓	✓	–	–	✓	Immediate	"WO {id} completed; asset restored."	
N-24	Maintenance	Asset out-of-service	Affected user, Manager	✓	✓	✓	–	–	Y	Immediate	"{asset} under maintenance; loaner {status}."
N-25	Maintenance	Preventive due	Assigned tech, Manager	✓	✓	–	–	✓	Lead-time (e.g. 7-day)	"Scheduled PM for {asset} due {date}."	
N-26	Maintenance	Predictive health alert	IT Manager, Technician	✓	✓	✓	–	✓	On threshold	"{asset} health {score}; {metric} degrading."	
N-27	Warranty	Warranty expiry	IT Manager, Finance	✓	✓	–	–	✓	Y	90/60/30/7-day	"Warranty for {asset} expires {date}."
N-28	Warranty	AMC contract expiry	IT Manager, Finance	✓	✓	–	–	✓	Y	90/60/30-day	"AMC {contract} expires {date}."
N-29	Inventory	Low stock (reorder point)	Store, Procurement	✓	✓	–	–	✓	Y	Immediate	"Part {sku} below reorder ({qty})."
N-30	Inventory	Stock count variance	Store Manager, Auditor	✓	✓	–	–	✓	Y	On count post	"Variance {n} units for {sku}."
N-31	Offboarding	Return-required checklist	Leaver's manager, IT	✓	✓	✓	–	✓	On offboard trigger	"Collect {n} assets from {employee}."	
N-32	Offboarding	Return overdue	Manager, IT, Security	✓	✓	✓	✓	✓	Y	Due+1, then daily	"OVERDUE: {asset} not returned by {employee}."
N-33	Security	Suspicious/security alert	Security team, IT Manager	✓	✓	✓	✓	✓	Y	Immediate	"Security event: {detail} on {entity}."
N-34	Audit	Config / permission change	Tenant Admin, Security	✓	✓	–	–	✓	Y	Immediate	"{actor} changed {setting}."
N-35	Audit	Data export performed	Tenant Admin, DPO	✓	✓	–	–	✓	Y	Immediate	"{actor} exported {entity} ({rows})."
N-36	Vendor	Vendor portal action on WO	IT Manager liaison	✓	✓	–	–	✓	Immediate	"Vendor updated WO {id}: {change}."	

PART C — AUDIT MATRIX

C.1 Principles

- **Append-only ledger:** every write emits an immutable event (`audit_event`) — no update/delete on the ledger; corrections are new compensating events.
- **Tamper evidence:** each event carries `prev_hash` + `event_hash` (hash chain) per tenant partition.
- **Actor context:** `actor_id` , `actor_role` , `on_behalf_of` , `source_ip` , `user_agent` , `session_id` , `timestamp (UTC)` , `tenant_id` , `request_id` .
- **Before/After:** JSON diffs of changed fields captured for state-changing actions.
- **Retention** honors regulatory minimums; **PII minimization** — sensitive values referenced by ID, not duplicated, to satisfy GDPR.

C.2 Auditable Actions Matrix

#	Action	Entity	Actor	Captured before/after	Retention	Immutable / append-only	Compliance mapping
A-01	Login success/failure, MFA, logout	Auth/Session	User/System	ip, device, method, result (no password)	1 yr	Append-only	SOC2 CC6.1; ISO A.9; GDPR 32
A-02	Password/MFA change, role switch	Identity	User/Admin	before/after auth factors (hashed refs)	1 yr	Append-only	SOC2 CC6; ISO A.9.2
A-03	Role / permission grant/revoke	RBAC	Admin	role set before→after, scope	3 yr	Append-only	SOC2 CC6.3; ISO A.9.2.3
A-04	Asset create / edit / retire / dispose	Asset	IT Mgr/Admin	full field diff, status, value	Life+7 yr	Append-only	ISO A.8.1; SOX/finance
A-05	Asset assign / return	Assignment	Mgr/Admin	holder before→after, dates	Life+7 yr	Append-only	ISO A.8.1.3
A-06	Asset transfer (user/site)	Assignment	Mgr/Admin	from→to user/location	Life+7 yr	Append-only	ISO A.8.1.3
A-07	Cost / valuation change	Asset/Finance	Mgr/Finance	old→new cost, reason	7 yr	Append-only	SOX; SOC2 CC3
A-08	License assign / remove / transfer	License	Sw Admin	seat holder before→after	3 yr	Append-only	SOC2 CC6; vendor-audit
A-09	License key / entitlement edit	License	Sw Admin	masked key ref, counts	3 yr	Append-only	ISO A.18.1.2
A-10	PO create / approve / cancel	Procurement	Mgr/Finance	amounts, approver, status	7 yr	Append-only	SOX; SOC2 CC3
A-11	GRN posting / discrepancy	Procurement	Store	received vs ordered, variance	7 yr	Append-only	SOX
A-12	Invoice match / payment ref	Procurement	Finance	invoice→PO match, amount	7 yr	Append-only	SOX
A-13	Maintenance decision (approve/reject/re-scope)	Maintenance WO	Mgr/Admin	est/approved cost, decision, reason	5 yr	Append-only	SOC2 CC8; ISO A.12
A-14	WO state transition	Maintenance	Tech/Mgr/System	from_state→to_state	5 yr	Append-only	SOC2 CC8
A-15	Parts consumption	Maint/Inventory	Tech	qty, serials, stock before→after	5 yr	Append-only	ISO A.8; SOX (inventory)
A-16	Technician e-signature	Maintenance	Tech/Vendor	signer, signed_hash, ip, ts	7 yr	Immutable (frozen)	SOC2 CC7; e-sign integrity
A-17	QC sign-off	Maintenance	Mgr/QC	result, signer, independence flag	5 yr	Append-only	SOC2 CC8; SoD
A-18	Config change (SLA, thresholds, workflows)	Config	Admin	setting before→after	3 yr	Append-only	SOC2 CC8.1; ISO A.12.1.2
A-19	Notification rule / integration change	Config	Admin	rule/endpoint before→after	3 yr	Append-only	SOC2 CC7
A-20	Data export / report download	Data	Any authorized	entity, filter, row count, format	3 yr	Append-only	GDPR 30/15; SOC2 CC6.7
A-21	Bulk import / mass update	Data	Admin	record count, field diffs summary	3 yr	Append-only	SOC2 CC8; GDPR 5
A-22	Vendor-portal action on WO/ticket	Vendor	Vendor user	field diff, external ref, cost	5 yr	Append-only	SOC2 CC6 (3rd-party)
A-23	PII access / DSAR fulfillment	Personal data	Admin/DPO	subject id, purpose, scope	3 yr	Append-only	GDPR 15/17/30
A-24	Retention / deletion job execution	Data lifecycle	System/Admin	entity, records purged, policy id	7 yr (log of deletion)	Append-only	GDPR 17; ISO A.18
A-25	Security policy / alert action	Security	Security/Admin	alert, action taken	3 yr	Append-only	SOC2 CC7.2; ISO A.16

C.3 Compliance Notes

- **SOC 2 (CC6 access, CC7 monitoring, CC8 change mgmt):** A-01–03, A-13/14/17/18/25 provide the access, monitoring, and change-management evidence set.
- **ISO/IEC 27001 Annex A:** A.8 (asset mgmt) → A-04–07, A-15; A.9 (access) → A-01–03; A.12/A.16 (operations/incidents) → A-13/14/25; A.18 (compliance) → A-24.

- **GDPR:** Art. 30 records-of-processing → A-20/23; Art. 15/17 subject rights → A-23/24; Art. 5/32 minimization & security → hash chain, PII-by-reference, retention jobs.
- **Retention governance:** financial-linked records (A-04, A-07, A-10–12, A-16) follow the longest applicable statutory clock (asset life + 7 yr / 7 yr); the deletion log itself (A-24) is retained even after underlying PII is purged, satisfying the accountability principle.

End of Volume III. Contents are implementation-ready specifications: entity/field tables map directly to schema, the state machine to the WO engine, the matrices to the notification-rules and audit-ledger services. All entities are tenant-partitioned with row-level security in the multi-tenant deployment.

Deliverable notes for the parent agent: This is the complete Part A (Maintenance FRD) + Part B (Notification Matrix, 36 events) + Part C (Audit Matrix, 25 auditable actions) content, formatted with tables and three Mermaid diagrams (lifecycle state diagram, out-of-service cascade flowchart, approval branch). It is ready to drop into the PDF blueprint. No files were written and no repository code was inspected — this was a pure authoring task per the request to "return ONLY the content."

PART 7

Technical SRS — Architecture, API, Security, Data + Module/Screen Trees

PART 1 — SOFTWARE REQUIREMENTS SPECIFICATION (Technical)

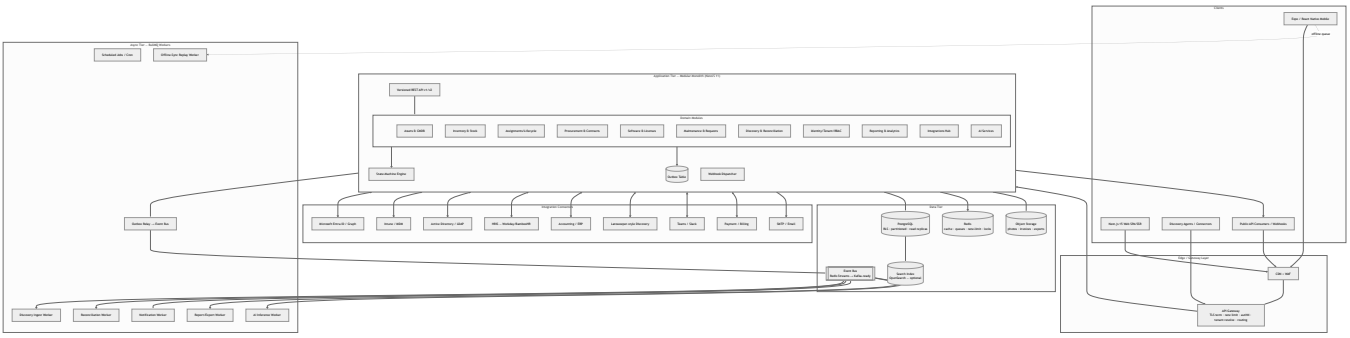
1.1 Architectural Philosophy

TechpioAsset evolves from a **monolith-modular NestJS app** into a **modular monolith with an event/outbox backbone**, designed so that high-volume subsystems (discovery ingest, audit/event stream, notifications, reporting) can be **extracted into independent services** without rewriting domain logic. The rule: *one deployable today, clean seams for tomorrow.*

Design tenets:

Tenet	Implementation
Modular monolith first	Nest domain modules with enforced boundaries (no cross-module Prisma access; only via module service interfaces / domain events).
Extract by pressure, not fashion	Discovery ingest, event/audit, notifications, and reporting are the first extraction candidates (write-heavy / bursty).
Event-driven core	Transactional outbox → relay → Redis Streams / Kafka-ready bus. Every state change emits a domain event.
Multi-tenant by default	Tenant context is a first-class request dimension enforced at data, cache, queue, and storage layers.
Provider abstractions preserved	Existing storage/AI/mail/queue/cache/push/SSO provider interfaces stay; new integrations register as providers.
Zero-trust	Every internal call is authenticated + authorized + tenant-scoped; no implicit trust between modules or pods.

1.2 Target System Architecture



Extraction roadmap (later): `Discovery &amp; Reconciliation`, `Reporting &amp; Analytics`, `Notifications`, and `AI Services` become independent services communicating over the event bus + gRPC/REST, sharing the same tenant-context contract.

1.3 Database & Multi-Tenancy Strategy

1.3.1 Isolation model comparison

Model	Isolation	Cost/Scale	Ops Complexity	Noisy-Neighbor	Per-tenant customization	Verdict
Shared schema + `tenant_id`	Logical (row)	Excellent (1 DB, thousands of tenants)	Low	Medium (mitigated by RLS + partition)	Hard (schema shared)	**Default**
Schema-per-tenant	Strong (namespace)	Moderate (hundreds of tenants)	Medium (migrations × N schemas)	Low	Easy	**Tier-up for large/regulated**
DB-per-tenant	Strongest	Poor (dozens)	High	None	Easiest	**Enterprise/on-prem/data-residency only**

1.3.2 Recommendation — **Hybrid, RLS-anchored**

1. **Default: shared-schema-with-`tenant_id`**, enforced by **PostgreSQL Row-Level Security (RLS)**. Every tenant-scoped table carries `tenant_id uuid not null`; a `SET app.tenant_id` session GUC drives an RLS policy so *even a buggy query cannot cross tenants.*
2. **Prisma integration:** a per-request Prisma middleware/extension opens a transaction that executes `SELECT set_config('app.tenant_id', $1, true)` before any query — RLS is the backstop, the middleware is the convenience layer.
3. **Tier-up path:** premium/regulated tenants promoted to **schema-per-tenant** via the same connection-routing abstraction; the app code is isolation-agnostic (resolves a `TenantDataSource` from tenant metadata).
4. **DB-per-tenant** reserved for data-residency / on-prem contracts, provisioned by the same tenant-provisioning workflow.

1.3.3 Scale mechanics

- **Partitioning:** time-partition high-volume tables — `audit_events`, `discovery_scans`, `asset_history`, `notifications` — by month (declarative range partitions) + `tenant_id` sub-hashing for the largest tenants.
- **Read replicas:** reporting/analytics + list endpoints routed to replicas via Prisma read/write split; writes to primary.
- **Indexing:** composite `(tenant_id, &lt;lookup&gt;)` on every hot path; GIN indexes for JSONB custom-field search and tag arrays.

- **Soft-delete + retention:** `deleted_at` everywhere; hard-purge governed by tenant retention policy job.
- **Optimistic concurrency:** `version int` column + `updated_at` for conflict detection (feeds mobile offline sync).

1.4 API Design

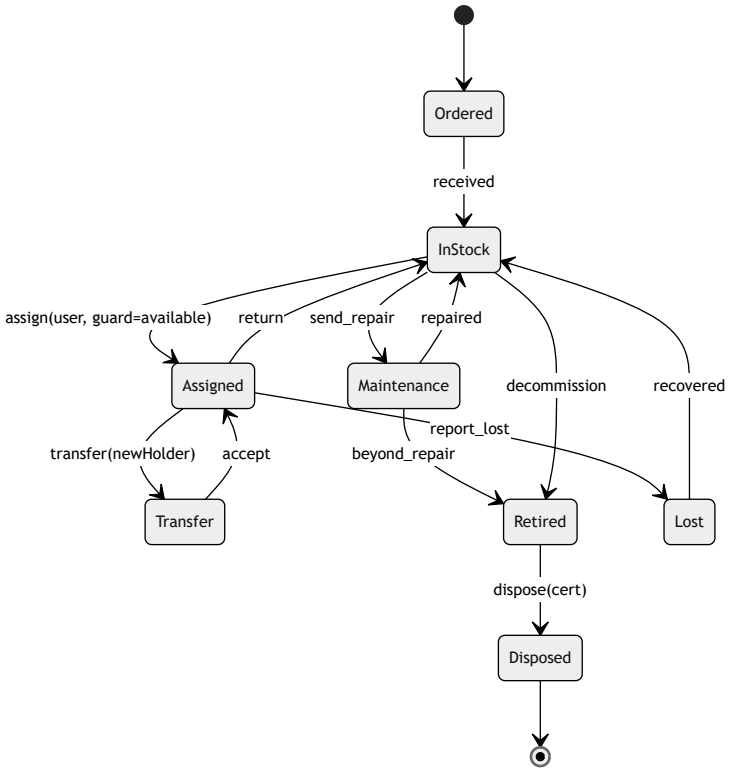
Aspect	Specification
Style	Versioned REST (`/api/v1` , `/api/v2`), URI-versioned; additive within a version, breaking changes bump version.
Envelope	Keep existing response envelope `{ data, meta, links }` ; cursor pagination (`?cursor=` , opaque, stable ordering).
Errors	RFC-9457 `application/problem+json` (already in place) with `type` , `title` , `status` , `detail` , `instance` , `errors[]` , `traceId` .
Filtering	RSQL/FIQL-style `?filter=status==active;category==laptop` + saved-filter resources.
Public API	Separate `/public/v1` surface, scoped API keys (per-tenant), documented via OpenAPI 3.1 + generated SDKs.
Webhooks	Tenant-configurable subscriptions to domain events; HMAC-SHA256 signed payloads, retry with exponential backoff + DLQ, delivery log, replay endpoint.
Rate limits	Redis sliding-window per `(tenant, key, route-class)` ; tiered by plan; `RateLimit-*` + `Retry-After` headers; separate buckets for interactive vs. bulk/import vs. agent-ingest.
Idempotency	`Idempotency-Key` header on all POST/PUT (mutations) — stored 24h, returns first result on replay. Critical for mobile offline replay.
Bulk	`/bulk` endpoints (import/export/patch) that enqueue jobs and return a job resource to poll or webhook.
AuthN transport	Bearer JWT (interactive), API key (public), mTLS or signed-token (discovery agents).

Webhook signature contract:

```
X-Techpio-Signature: t=<unix>,v1=<hex hmac_sha256(secret, `${t}.${rawBody}`)>
X-Techpio-Event: asset.assigned
X-Techpio-Delivery: <uuid>
```

1.5 State-Machine Framework

A single declarative engine (e.g. XState-style config or a lightweight internal FSM) governs every lifecycle. Transitions are **guarded**, **audited**, **event-emitting**, and **permission-gated**.



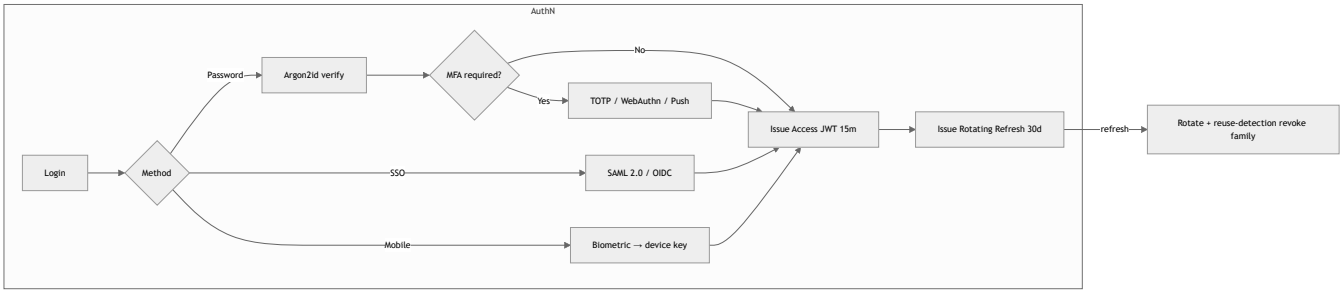
Framework requirements: - **Config-driven** state graphs per entity type (Asset, License, PurchaseOrder, MaintenanceRequest, StocktakeSession) — tenants may add custom states/transitions on top of a locked core. - **Guards** = permission check + business rule + data precondition. - **Side-effects** run *after* commit via outbox events (notify, webhook, integration sync). - **Every transition** writes an immutable `asset_history` row and an `audit_event` .

1.6 Security (Zero-Trust)

Control	Implementation
Zero-trust posture	Every request (external and internal) authenticated + authorized + tenant-scoped; no network-implicit trust; short-lived tokens.

Control	Implementation
Encryption in transit	TLS 1.3 everywhere; mTLS between app↔workers↔DB where supported; HSTS.
Encryption at rest	DB + object storage volume encryption (AES-256); column-level encryption (pgcrypto/app-layer) for PII & secrets (serial numbers optionally, IMEI, purchase costs on request).
Secrets	External secrets manager (Vault / cloud KMS / Doppler); no secrets in env files or images; automatic rotation; per-tenant integration credentials encrypted with envelope encryption (KMS DEK/KEK).
RLS	Postgres RLS as the tenant-isolation backstop (see 1.3.2).
Input/output	Strict DTO validation (class-validator), output serialization allow-lists, parameterized queries (Prisma), file-type/size validation + AV scan on uploads.
Headers	CSP, X-Content-Type-Options, Referrer-Policy, Permissions-Policy; CORS allow-list per tenant domain.
Data residency	Region-pinned deployments; tenant metadata declares residency; storage buckets region-scoped.
Threat protections	WAF, bot/rate protection, brute-force lockout, anomaly alerts (see AI §1.14).

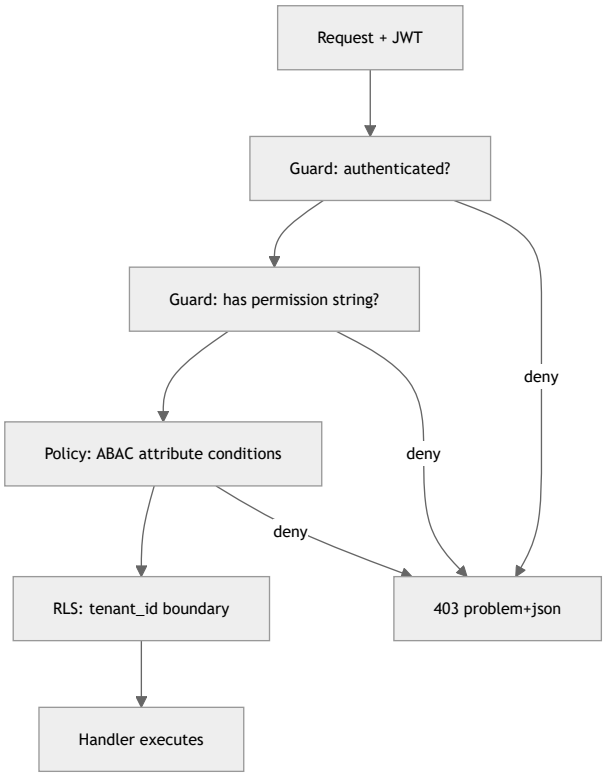
1.7 Authentication



Feature	Spec
Access token	JWT, ~15 min TTL, contains `sub`, `tenant_id`, `roles`, `perms` (or perm-hash), `scope`, `jti`.
Refresh rotation	Rotating refresh tokens with **reuse detection** → revoke entire token family (already partly in baseline; harden). Stored hashed, per-device.
MFA	TOTP, WebAuthn/FIDO2 passkeys, push-approval; per-tenant enforcement policy; step-up MFA for sensitive ops (disposal, bulk delete, cost export).
SSO	SAML 2.0 + OIDC; IdP-initiated + SP-initiated; per-tenant IdP config (Entra ID, Okta, Google).
SCIM 2.0	Inbound user/group provisioning & deprovisioning from IdP → auto-map to roles; JIT provisioning on first SSO login.
Device trust (mobile)	Device registration, biometric unlock bound to a device-scoped key in secure enclave/keystore; remote device revoke.
Sessions	Per-device session registry, admin force-logout, concurrent-session limits.

1.8 Authorization (RBAC + ABAC)

- **RBAC core:** permission strings (`asset.assign`, `license.manage`, `report.export`) grouped into roles. Preserve baseline permission-string + scope model.
- **ABAC overlay:** policy conditions on attributes — `location`, `department`, `assetCategory`, `costCenter`, `ownership`. Example: "*Site Manager may assign only assets where* `asset.LocationId ∈ user.managedLocations`."
- **Scopes:** OAuth-style scopes constrain API keys and integration tokens (`read:assets`, `write:inventory`).
- **Per-tenant custom roles:** tenants compose roles from the permission catalog; system roles are locked templates; role changes audited.
- **Enforcement:** Nest guards (route-level perm) + policy layer (record-level ABAC) + RLS (tenant boundary) — **defense in depth**.



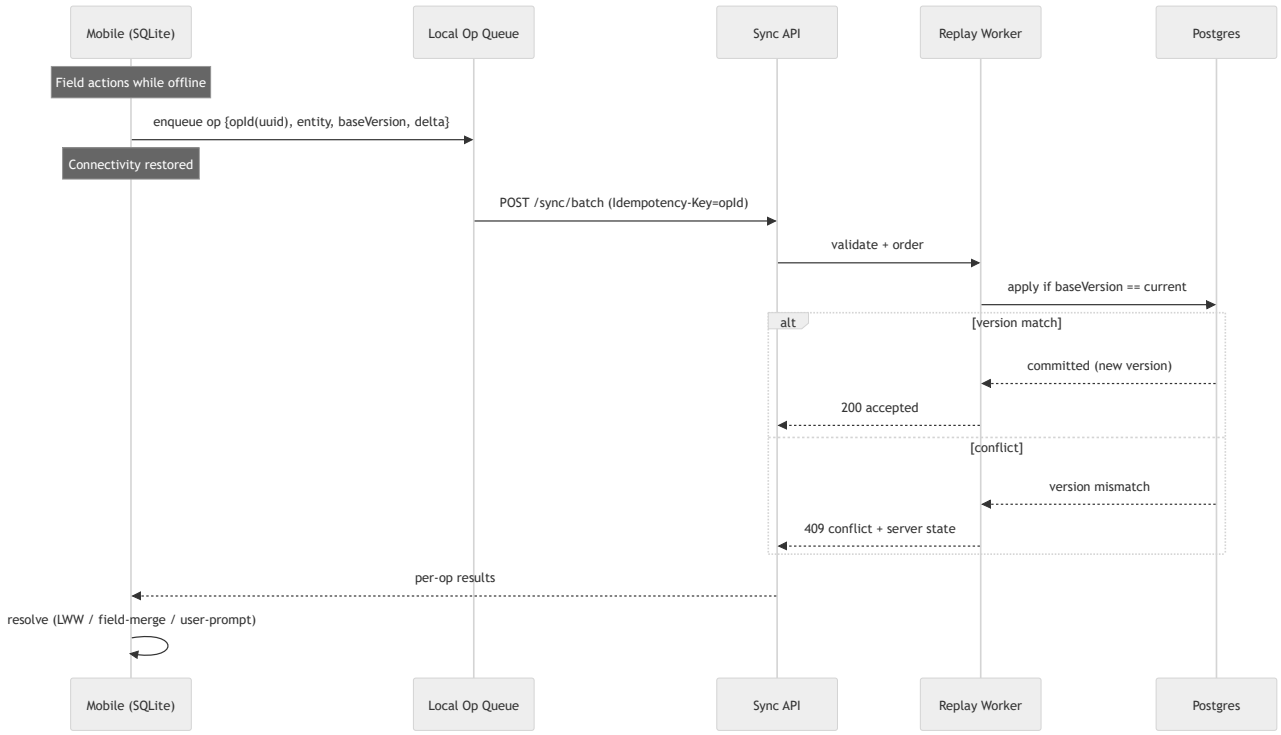
1.9 Integrations Hub

All integrations implement a common **Connector interface** (auth, sync, webhook-in, health, mapping) and register as providers. Sync is **bidirectional where meaningful**, event-driven, with conflict rules and a per-connector sync log.

Connector	Direction	Primary Data	Mechanism
Microsoft Entra ID / Graph	In	Users, groups, devices, license SKUs	OAuth2 app + Graph delta queries + change notifications
Intune / MDM	In	Managed device inventory, compliance, hardware/OS	Graph `deviceManagement` APIs, scheduled + webhook
Active Directory / LDAP	In	Users, OUs, computer objects	LDAP sync agent / on-prem connector
HRIS (Workday/BambooHR)	In	Employees, departments, onboarding/offboarding events	REST + webhooks → auto assign/reclaim on offboard
Accounting / ERP	Both	POs, invoices, cost centers, depreciation	REST/SFTP; push asset costs, pull invoices
Discovery agents / Lansweeper-style	In	Network scan, installed software, hardware specs	Agent → signed ingest API → reconciliation
SMTP / Email	Out	Notifications, approvals, digests	Provider abstraction (existing)
Teams / Slack	Out+In	Alerts, approvals (actionable cards), request bot	Incoming webhooks + bot/app + slash actions
Payment / Billing	Out	Subscription/seat billing per tenant	Provider abstraction; Stripe-class

Reconciliation engine (discovery): dedup + match against CMDB by identity keys (serial, MAC, hostname, asset tag), fuzzy match with confidence scoring (AI-assisted), auto-merge above threshold, human-review queue below.

1.10 Offline Sync (Mobile)



Concern	Strategy
Local store	SQLite (baseline), mutation queue table with monotonic client sequence.
Idempotency	Each op carries a client-generated `opId` used as `Idempotency-Key` → safe replay.
Ordering	Server orders by entity + client-sequence; dependent ops (create→assign) preserved.
Conflict detection	Optimistic `baseVersion` vs server `version`.
Conflict resolution	Default **field-level last-writer-wins** with server-authority for state transitions; **user-prompt** for irreconcilable field clashes; certain ops (assignments, counts) are additive/CRDT-friendly.
Delta pull	`/sync/changes?since=&lt;cursor&gt;` returns changed records for the device's scoped assets/locations.
Attachments	Photos/invoices queued separately, uploaded to object storage via resumable upload; record links reconciled after upload.

1.11 Queue, Outbox & Scheduled Jobs

Component	Detail
Queue	BullMQ on Redis (baseline). Named queues per concern: `discovery`, `reconcile`, `notify`, `report`, `ai`, `sync-replay`, `integration-sync`, `scheduled`.
Transactional outbox	Domain writes + outbox row in **one DB transaction**; relay worker publishes to event bus and marks sent → **exactly-once effect, no dual-write loss**.
DLQ	Failed jobs → dead-letter with reason; admin retry/inspect UI; alert on DLQ growth.
Concurrency & priority	Per-queue concurrency, rate-limited groups (per tenant) to prevent one tenant starving others; priority lanes for interactive vs. bulk.
Scheduled jobs	Cron: license true-up, warranty/contract expiry alerts, depreciation run, stale-discovery flag, retention purge, digest emails, integration delta polls, backup verification.
Idempotent workers	All handlers idempotent (keyed by event id) for safe retries.

1.12 Caching (Redis)

Layer	Cached	Invalidation
Reference data	Categories, statuses, custom-field schemas, permission catalog	Event-driven bust on config change; short TTL fallback.
Entity read cache	Hot asset/user lookups	Write-through / event-bust on entity update (`asset.updated`).
List/aggregate	Dashboard counters, report summaries	TTL + tag-based invalidation on relevant events.
Auth	Permission sets, JWT denylist (`jti`), refresh families	Bust on role change / logout / revoke.
Rate-limit & locks	Sliding-window counters, distributed locks (Redlock) for reconciliation/import	N/A / lock TTL.
Tenant scoping	All keys prefixed `t:{tenantId}:`	Enables per-tenant flush.

Strategy: **cache-aside for reads, event-driven invalidation for correctness**; every cache key namespaced by tenant + schema version to survive deploys/migrations.

1.13 Audit & Compliance

- **Append-only** `audit_events` (partitioned, immutable; DB `INSERT`-only role, no `UPDATE/DELETE`). Optional hash-chaining (`prev_hash`) for tamper-evidence.

- Captures: actor, tenant, action, entity, before/after diff, IP, device, request `traceId`, timestamp.
- **Retention policy** per tenant (e.g. 1–7 yrs); archival to cold object storage before purge; legal-hold flag blocks purge.
- **Exports** for auditors (CSV/JSON, signed); SIEM streaming (webhook/syslog) for enterprise tenants.
- Separate **security audit** (logins, MFA, permission changes, token reuse) surfaced in an admin security console.

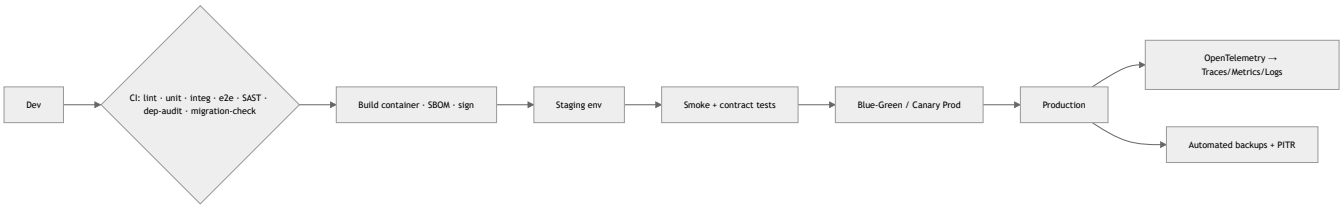
1.14 AI Services (Human-in-the-Loop, Governed)

Preserve the AI provider abstraction. AI is **assistive, never autonomous** for state changes.

Capability	Function	Guardrail
Assisted extraction	OCR invoices/POs/receipts → structured line items (vendor, cost, serials, warranty); asset photo → make/model/spec suggestion	Confidence score; low-confidence → human review; user confirms before commit.
Reconciliation matching	Fuzzy-match discovery records to CMDB with confidence	Auto-merge only above threshold; queue for review below.
Asset health scoring	Predict failure/EOL risk from age, maintenance history, warranty, usage	Advisory; drives suggestions, not auto-retire.
Anomaly detection	Unusual assignment/transfer patterns, license over-usage, ghost assets, cost outliers	Alerts to admins; no auto-action.
Natural-language search / assistant	"Show unassigned laptops in Delhi past warranty" → query	Read-only; respects RBAC/ABAC/RLS.
Smart suggestions	Suggested assignee, reorder point, category tagging	Suggestion UI with accept/reject.

Governance: per-tenant AI opt-in; data-use policy (no cross-tenant training; provider config controls whether data leaves boundary); every AI suggestion logged with model/version + confidence; human decision recorded in audit; PII redaction before external inference where required.

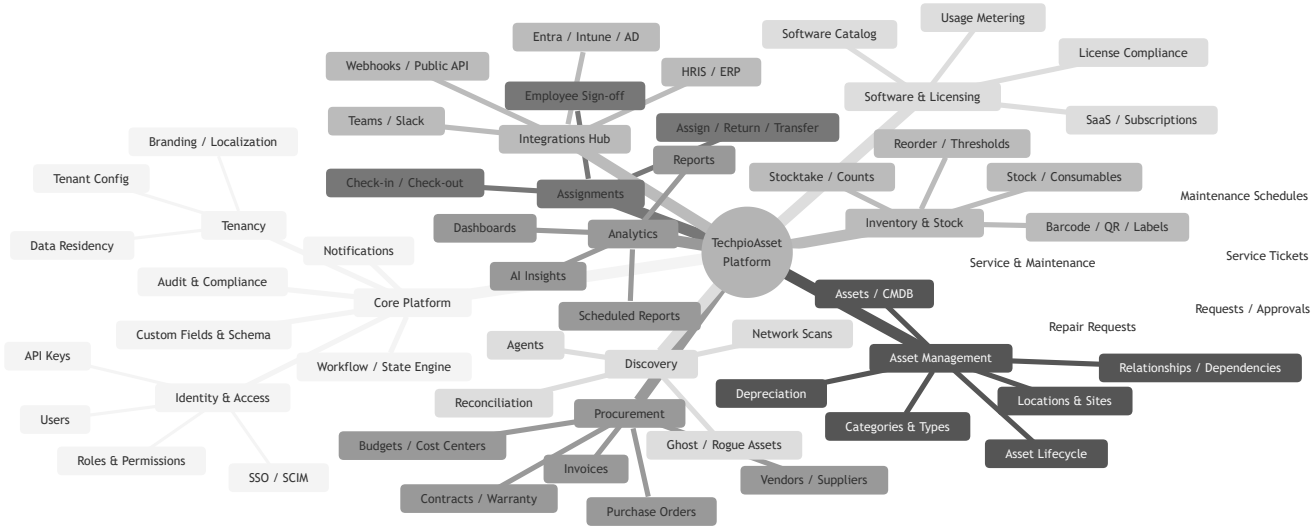
1.15 Deployment, Observability & DR



Area	Spec
Packaging	Containers (Docker); Kubernetes/Helm or ECS; separate deployables for API, workers, and (future) extracted services.
Environments	dev → staging → prod; ephemeral PR preview envs; infra as code (Terraform).
CI/CD	Build once, promote artifact; DB migrations gated + reversible (expand/contract pattern); feature flags for progressive rollout.
Release	Blue-green or canary; automated rollback on SLO breach; zero-downtime migrations.
Observability	**OpenTelemetry** traces (request→queue→worker→DB correlation via `traceId`), RED/USE metrics, structured logs, per-tenant dashboards, error tracking (Sentry-class), synthetic monitoring, SLO/alerting.
Scaling	Stateless API pods (HPA on CPU/RPS), worker pods scaled per-queue depth (KEDA), PG read replicas + connection pooler (PgBouncer), Redis cluster.
Backups / DR	Automated daily + WAL/PITR; cross-region replica; **tested restore drills**; documented RTO/RPO (target RPO ≤ 5 min, RTO ≤ 1 h); object-storage versioning; per-tenant export/backup for portability.
Multi-region	Region-pinned for residency; global control plane, regional data planes.

PART 2 — MODULE, SCREEN & NAVIGATION TREES (WEB)

2.1 Redesigned Module Map



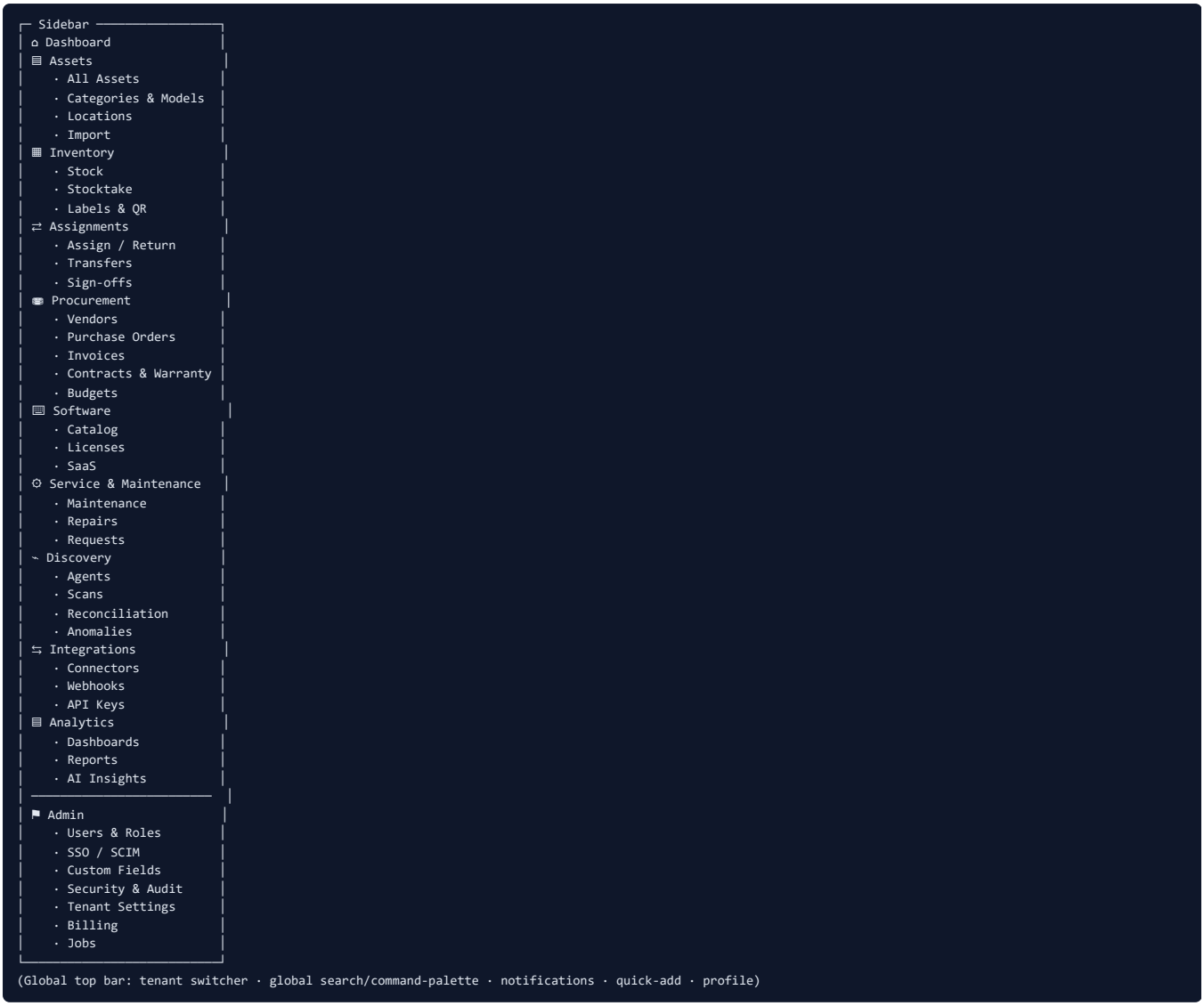
2.2 Full Module Tree (indented)

```
TechpioAsset/
├─ core/
│  ├─ identity/          (auth, MFA, sessions, SSO, SCIM, API keys)
│  ├─ tenancy/           (tenant provisioning, config, branding, residency)
│  ├─ rbac/              (roles, permissions, ABAC policies, custom roles)
│  ├─ audit/             (append-only events, security console, exports)
│  ├─ notifications/     (channels, templates, digests, preferences)
│  ├─ custom-fields/     (per-entity schema builder, validation)
│  ├─ workflow/          (state-machine engine, approvals, guards)
│  └─ files/             (upload, AV scan, object storage, thumbnails)
├─ assets/
│  ├─ cmdb/              (asset records, statuses, tags)
│  ├─ categories/        (types, models, manufacturers)
│  ├─ locations/         (sites, buildings, rooms, geo)
│  ├─ lifecycle/         (states, history, disposal certificates)
│  ├─ depreciation/      (methods, schedules, book value)
│  └─ relationships/     (dependency graph, parent/child, CI links)
├─ inventory/
│  ├─ stock/             (consumables, quantities, batches)
│  ├─ stocktake/         (count sessions, variance, reconciliation)
│  ├─ replenishment/     (reorder points, POs trigger)
│  └─ labeling/          (QR/barcode generation, print templates)
├─ assignments/
│  ├─ assign-return/     (checkout, checkin, transfer)
│  ├─ signoff/           (employee e-signature, acknowledgment)
│  └─ custody/           (chain of custody, responsibility)
├─ procurement/
│  ├─ vendors/           (suppliers, contacts, ratings)
│  ├─ purchase-orders/   (PO lifecycle, receiving)
│  ├─ invoices/          (capture, OCR, matching)
│  ├─ contracts/         (warranty, SLA, renewals, expiry)
│  └─ budgets/           (cost centers, spend tracking)
├─ software/
│  ├─ catalog/           (software titles, versions)
│  ├─ licenses/          (entitlements, compliance, true-up)
│  ├─ saas/              (subscriptions, seats)
│  └─ metering/          (usage, reclaim unused)
├─ service/
│  ├─ maintenance/       (preventive schedules, tasks)
│  ├─ repairs/           (repair tickets, vendor RMA)
│  └─ requests/          (asset requests, approvals workflow)
├─ discovery/
│  ├─ agents/            (agent registry, health, tokens)
│  ├─ scans/             (ingest, raw records)
│  ├─ reconciliation/    (matching, merge, review queue)
│  └─ anomalies/         (ghost/rogue detection)
├─ integrations/
│  ├─ connectors/        (Entra, Intune, AD, HRIS, ERP, Teams/Slack)
│  ├─ webhooks/          (subscriptions, delivery log)
│  └─ public-api/        (keys, scopes, usage)
├─ analytics/
│  ├─ dashboards/        (role-based, widgets)
│  ├─ reports/           (builder, scheduled, exports)
│  └─ ai-insights/       (health, anomalies, forecasts)
└─ admin/
   ├─ settings/          (system, security policies)
   ├─ billing/          (plan, seats, usage)
   └─ jobs/              (queue monitor, DLQ, scheduled jobs)
```

2.3 Web Screen Tree

```
Web App/
├─ Auth
│  ├─ Login (password / SSO)
│  ├─ MFA Challenge (TOTP/WebAuthn/Push)
│  ├─ Forgot / Reset Password
│  └─ Accept Invite / SSO First-login
├─ Dashboard (role-based)
│  ├─ Admin Dashboard
│  ├─ Asset Manager Dashboard
│  ├─ IT Ops Dashboard
│  └─ Finance Dashboard
├─ Assets
│  ├─ Asset List (filters, saved views, bulk actions)
│  ├─ Asset Detail (overview · history · relationships · docs · maintenance · financials)
│  ├─ Asset Create / Edit
│  ├─ Bulk Import (wizard + mapping + preview)
│  └─ Category / Model / Manufacturer manager
├─ Inventory
│  ├─ Stock List
│  ├─ Stocktake Sessions
│  ├─ Count Session Detail (variance)
│  └─ Label / QR Print Studio
├─ Assignments
│  ├─ Assign / Checkout Wizard
│  ├─ Return / Checkin
│  ├─ Transfer
│  └─ Sign-off / Acknowledgment view
├─ Procurement
│  ├─ Vendors
│  ├─ Purchase Orders (list · detail · receive)
│  ├─ Invoices (capture · OCR review · matching)
│  ├─ Contracts & Warranty (expiry calendar)
│  └─ Budgets / Cost Centers
├─ Software
│  ├─ Software Catalog
│  ├─ License Compliance (over/under-licensed)
│  ├─ SaaS Subscriptions
│  └─ Reclaim / Optimization
├─ Service
│  ├─ Maintenance Schedules & Calendar
│  ├─ Repair Tickets
│  └─ Requests & Approvals inbox
├─ Discovery
│  ├─ Agents (registry · health)
│  ├─ Scan Results
│  ├─ Reconciliation Review Queue
│  └─ Anomalies / Ghost Assets
├─ Integrations
│  ├─ Connectors Catalog & Config
│  ├─ Sync Logs
│  ├─ Webhooks
│  └─ Public API Keys
├─ Analytics
│  ├─ Dashboards (builder)
│  ├─ Report Library & Builder
│  ├─ Scheduled Reports
│  └─ AI Insights
└─ Admin
   ├─ Users
   ├─ Roles & Permissions (custom role builder)
   ├─ SSO / SCIM
   ├─ Custom Fields & Workflow designer
   ├─ Security (audit, sessions, policies)
   ├─ Notifications templates
   ├─ Tenant Settings / Branding / Localization
   ├─ Billing
   └─ Jobs / Queue Monitor
```

2.4 Web Navigation (Sidebar) Tree



Sidebar items render conditionally by permission/ABAC; collapsible groups; pinned favorites; command-palette (`Ctrl+K`) for power users.

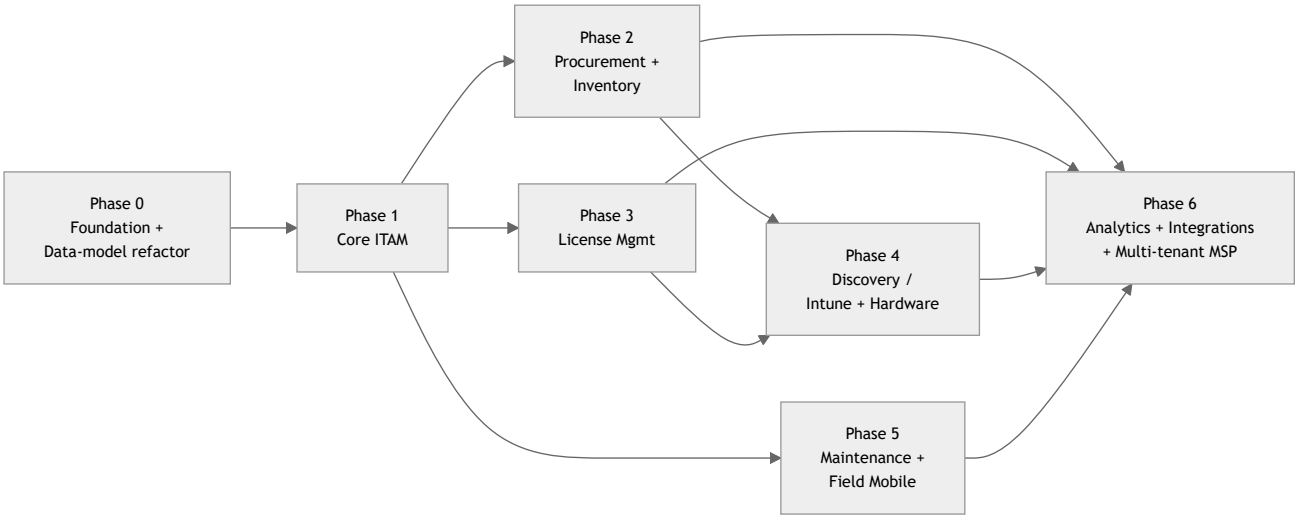
PART 8

Delivery Roadmap & Phase Plan

4. DEVELOPMENT ROADMAP + PHASE-WISE IMPLEMENTATION PLAN

Assumptions: existing TechpioAsset codebase (provider pattern for storage/AI/mail/queue/chat, hardened, 448 tests green). This roadmap is the **redesign delivery**, not a greenfield. Effort in engineer-weeks (EW) assuming a 4–5 dev + 1 designer + 1 QA squad. "S/M/L/XL" = 1–2 / 3–5 / 6–10 / 11+ EW.

Phase dependency flow



Phase 0 — Foundation & Data-Model Refactor

Goal: Establish the redesigned shell, design system, tenancy model, and normalized data schema before building features on top.

Aspect	Detail
Key deliverables	Design system + component library (tokens, grid, drawer, charts, badges); app shell (nav/topbar/drawer/⌘K); tenancy model (tenant isolation, RBAC scopes, SoD); normalized asset/license/inventory schema + migration; audit-log spine; dark mode; a11y baseline
Effort	**L** (6–10 EW)
Dependencies	None (must precede all)
Exit criteria	Component storybook published; new shell renders with sample data across ≥2 tenants; RBAC scopes enforced in middleware; migration runs green with rollback; WCAG-AA audit passes on shell; existing 448 tests still green

Phase 1 — Core ITAM

Goal: Redesigned asset lifecycle: register, assign, transfer, retire + Employee self-service + role dashboards v1.

Aspect	Detail
Key deliverables	Asset list (grid/filters/saved views/bulk); Asset detail (tabbed overview/hardware/software/warranty); assignment & transfer workflows; QR tagging; Employee self-service (my assets/requests); Approval inbox; dashboards for IT Manager, Technician, Employee, Dept Manager, Company Admin; Asset + Employee + Warranty reports
Effort	**XL** (11+ EW)
Dependencies	Phase 0
Exit criteria	Full asset lifecycle demoable end-to-end; approval routing works with SoD; QR scan resolves asset on web; role dashboards live; reports export CSV/XLSX/PDF; regression + new tests green

Phase 2 — Procurement + Inventory

Goal: PR→PO→receive pipeline and stockroom control feeding the asset register.

Aspect	Detail
Key deliverables	PR creation + approval chain; PO board (kanban) + list; vendor records + vendor portal (scoped); goods-receipt/GRN → auto-create assets/stock; inventory levels, reorder points, reservations, cycle counts; Procurement, Inventory, Vendor reports; dashboards for Procurement, Inventory, Office Admin, Vendor
Effort	**XL** (11+ EW)
Dependencies	Phase 1 (asset register, approvals)
Exit criteria	PR→PO→receipt creates assets/stock automatically; low-stock triggers reorder suggestion; vendor portal isolated to own POs; cycle-count variance report accurate; tests green

Phase 3 — License Management

Goal: Software entitlement, seat pools, compliance, reclamation.

Aspect	Detail
Key deliverables	License catalog + entitlements; seat-pool cards; assignment grid; over/under-deployment detection + limit-exceeded error state; idle-seat reclamation; renewal forecast; license↔asset↔user linkage; License + Finance(license cost) reports; IT Manager license widgets
Effort	**L** (6–10 EW)
Dependencies	Phase 1 (users/assets)
Exit criteria	Compliance position accurate vs entitlement; assigning past limit blocked with error UX; reclaim flow frees seats; renewal forecast scheduled email works; tests green

Phase 4 — Discovery / Intune + Hardware

Goal: Auto-populate hardware/software truth from device management + agents.

Aspect	Detail
Key deliverables	Intune / MDM connector; agent or network discovery ingest; hardware spec + installed-software sync into Asset detail (Hardware/Software/Health tabs); device health score; unauthorized-software flags; reconcile discovered vs registered; discovery job monitoring
Effort	**L** (6–10 EW)
Dependencies	Phase 1 (asset), Phase 3 (license reconciliation)
Exit criteria	Intune sync populates hardware/software on schedule; health scores render; unauthorized software surfaced; discovered-but-unregistered assets queued for reconciliation; connector failures alert; tests green

Phase 5 — Maintenance + Field Mobile

Goal: Ticketing, SLA, repairs, and full field-technician mobile app.

Aspect	Detail
Key deliverables	Maintenance ticket (create/assign/SLA/parts/time/cost); SLA timers + escalation; preventive-maintenance schedules; warranty-claim flow; mobile app (home dashboard, QR scan→asset, assign/return sheet with photo+signature, technician queue, log time offline); Maintenance reports; HR onboarding/offboarding pipelines
Effort	**XL** (11+ EW)
Dependencies	Phase 1 (asset/approvals); benefits from Phase 2 (parts from inventory)
Exit criteria	Ticket lifecycle + SLA breach alerts work; mobile app ships to stores/TestFlight; offline capture syncs; onboarding kit → asset assignment automated; MTTR report accurate; tests green

Phase 6 — Analytics + Integrations + Multi-tenant MSP

Goal: Executive analytics, external integrations, and Super-Admin MSP control plane.

Aspect	Detail
Key deliverables	Executive dashboard pack + Trend board; Auditor workspace + immutable audit trail + exception register + SoD review; scheduled report engine (all categories); integrations (SSO/SCIM, HRIS, finance/ERP, ITSM, webhooks); Super-Admin cross-tenant dashboard (tenants, MRR, usage, system health, impersonation, broadcast, feature flags); tenant provisioning/billing
Effort	**XL** (11+ EW)
Dependencies	Phases 1–5 (data to aggregate)
Exit criteria	Exec pack + all scheduled reports deliver on cadence; audit trail immutable + exportable; SSO/SCIM provisioning works; Super-Admin manages ≥N tenants with isolation proven; MRR/usage metrics reconcile with billing; full regression green

---

Roadmap summary (Now / Next / Later)

Horizon	Phases	Theme
Now	0, 1	Foundation + core asset lifecycle & dashboards
Next	2, 3	Procurement/inventory + license compliance
Later	4, 5, 6	Discovery, field mobile, analytics & MSP scale
